Signal Processing

Ranim Tom

December 7, 2025

Abstract

This document represent a ***course reader*** for my signal processing work. Mainly it contains theoretical explanation.
Simulation validating the theory are presented in a separate document.

Contents

1 Introduction to Digital Signal Processing **9**

2 Sampling **11**

2.1 References . . . 11
2.2 Introduction to Sampling and Reconstruction . . . 11
2.3 Sampling Continuous Time Signals . . . 12
2.3.1 Impulse Sampling Analysis using Fourier Transform . . . 12
2.3.2 Intuition and Interpretation . . . 12
2.3.3 Example: Sampling a Cosine . . . 13
2.3.4 Predicting Aliasing frequency . . . 15
2.4 Examples . . . 17
2.5 Other Practical Considerations . . . 17
2.5.1 Time Limited Signals . . . 17
2.5.2 Anti-Aliasing Filter . . . 19
2.5.3 Practical Sampling with finite width pulse . . . 20
2.6 Ideal Reconstruction . . . 24
2.6.1 Sampling theorem and the Uniqueness of Reconstruction . . . 24
2.6.2 Ideal (Sinc) Interpolation . . . 25
2.7 Practical Sampling and Reconstruction . . . 29
2.7.1 Mathematical Formulation for ZOH . . . 30
2.7.2 Reconstruction for ZOH . . . 31
2.8 Spectral Sampling . . . 32
2.9 Applications . . . 33
2.9.1 PAM, PWM and PPM . . . 34
2.9.2 Quantization and A/D Process . . . 34
2.9.3 PCM . . . 36
2.10 Practical Digital Filtering and Oversampling . . . 37
2.10.1 Building Analog filters . . . 37
2.10.2 Example of Oversampling . . . 39
2.10.3 Oversampling in D/A . . . 41
2.11 Quantization and A/D Process . . . 42
2.11.1 Dynamic Range of an Ideal ADC . . . 44
2.11.2 Error Analysis of the Quantization Error . . . 44
2.11.3 More Source of Errors . . . 48
2.11.4 Specs for ADC . . . 49
2.12 Oversampling and $\Sigma - \Delta$ Converter . . . 51

3 Bandpass Sampling **53**
3.1 Integer Band Positioning . . . 53
3.2 Non-Integer Band Positioning . . . 53
3.3 Quadrature Sampling . . . 54
3.3.1 Review of Phasor Notation . . . 54
3.3.2 Equivalent Low Pass Signal . . . 55
3.3.3 Recap . . . 57
3.4 Generation of I and Q components . . . 58
3.5 Sampling Dilemma using I and Q Components . . . 60
3.5.1 Integer band positioning . . . 62

I DSP Time Domain Tools **67**

4 Convolution **69**
4.1 Introduction . . . 69
4.2 Definition . . . 69
4.3 Properties . . . 70
4.4 Example in Continuous Time . . . 71
4.4.1 Example 1 . . . 71
4.4.2 Example 2 . . . 73
4.5 Examples in Discrete Time . . . 75
4.5.1 Finite Short Signals . . . 75
4.5.2 Long or Infinite Signal Approach . . . 75

5 Correlation **83**
5.1 Definition . . . 83

6 Discrete Time Fourier Transform **85**
6.1 Relation between DTFT and impulse sampled CTFT . . . 85

7 The Discrete Fourier Transform **87**
7.1 Introduction . . . 87
7.2 Formula and Direct Results . . . 87
7.2.1 Discrete time case: DFT of DTFT . . . 87
7.2.2 Continuous time case: DFT of CTFT . . . 88
7.3 Examples . . . 88
7.3.1 Continuous time case: Infinite Time . . . 88
7.3.2 Discrete Time: Finite Length Signal . . . 93
7.3.3 Discrete Time: Infinite Length Signal . . . 96
7.4 Resolution and Zero Padding . . . 98
7.4.1 1 Sinusoidal Analysis . . . 100

8 The Discrete Fourier Transform (Enhanced) **101**
8.1 Introduction . . . 101
8.2 Finite Duration Case . . . 101
8.3 Infinite Duration Case . . . 102
8.4 DFT for Spectral Analysis . . . 103
8.4.1 Modeling the Truncation Problem . . . 104

8.5 Example . . . 106

9 DFT V3 **107**
9.1 Introduction . . . 107
9.2 Notes . . . 107

10 Windowing **109**
10.1 Design Criteria of a Window . . . 109
10.2 Window Comparison . . . 109
10.3 Derivative,Continuity and Rate of Decay . . . 110

11 Discrete Time System Implementation **111**
11.1 Introduction . . . 111
11.2 IIR and Recursion . . . 112
11.3 Block Diagrams . . . 113
11.3.1 First Order Difference Equation and Direct Form . . . 114
11.3.2 General Direct Form . . . 116
11.3.3 Recursion and Non Recursion . . . 116
11.4 General Filter Design Concept . . . 116

12 FIR Design **119**
12.1 Linear Phase Revisited . . . 119
12.2 Linear Phase for FIR . . . 120
12.2.1 Recap for FIR Equations . . . 120
12.2.2 Linear Phase definition for FIR . . . 121
12.2.3 Zero Pattern . . . 122
12.2.4 Use case for Symmetric and Anti-symmetric . . . 122
12.3 Window Design Technique . . . 122
12.3.1 The usage of other window . . . 124
12.4 Frequency Sampling Technique . . . 125
12.5 Optimum Equiripple . . . 127
12.5.1 MATLAB Example . . . 127
12.5.2 Revisiting the Optimum Equiripple . . . 128

13 IIR Filter **129**
13.1 Introduction . . . 129
13.2 Notation . . . 130
13.3 Difference Equation Approach . . . 131
13.4 Impulse Invariance Approach . . . 132
13.4.1 Example . . . 133
13.5 Bilinear Transformation . . . 134

14 System Realization **139**

15 Multirate Signal Processing **141**
15.1 Reference Used . . . 141
15.2 Downsampling . . . 141
15.2.1 Implementing Downsampling . . . 142
15.3 Upsampling . . . 143

16 Multirate Sampling by Dr.Michael Lustig Berkely **145**
16.1 Motivation . . . 145
16.2 Downsampling . . . 145
16.3 Aliasing Revisited . . . 146
16.4 Upsampling . . . 146

17 Fred Harris talk about Multirate 2020 **147**

18 Compressed Sensing **149**
18.1 Introduction and Motivation . . . 149
18.2 Example of Compression and Interpretation . . . 150
18.3 Steps of Compression . . . 153
18.4 Sparsity and Noise . . . 154
18.5 Moving to Compressive Sampling . . . 156

19 Compressed Sensing 2: Dr Steve Brunton **161**
19.1 References . . . 161
19.2 Notes . . . 161
19.3 Conditions . . . 163

Chapter 1

Introduction to Digital Signal Processing

Notes:

- In DSP, all the processing is done in the discrete domain
- Advantage of DSP over the analog domain:

  Suppose we have a system which suffers from noise, and we want to eliminate this noise. We could use some analog filter (form capacitors and inductors) and filter this noise at the unwanted frequency.

  Now, some day it becomes hot, and our filter start to play due to the hot temperature, so we need to change the filter and the hardware.

  However in the digital domain, it's all about the software and the implementation, we could only change the filter coefficient and keep the same hardware.

Chapter 2

Sampling

2.1 References

This chapter is mainly taken from [1].

2.2 Introduction to Sampling and Reconstruction

Sampling is a common operation in signal processing, since our computers and hardware we use process ***discrete sample***. An example is shown in Figure 2.1.

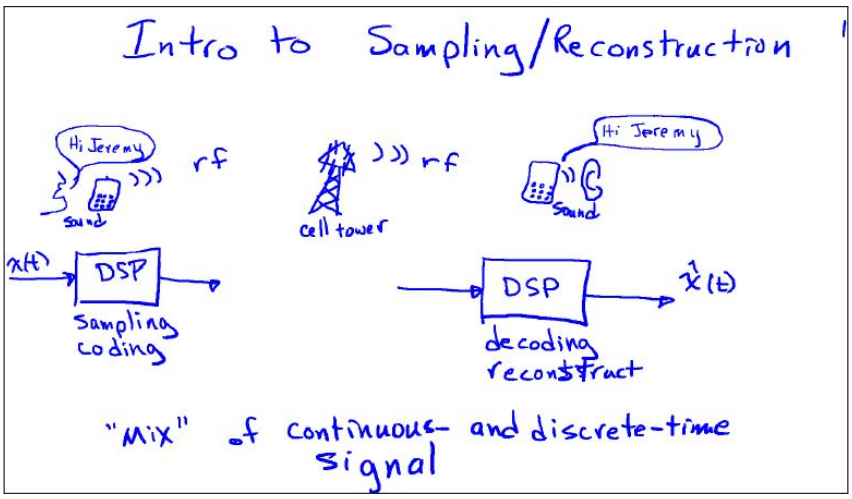


Figure 2.1: Example of Sampling: digital transmission

- We have some sound denoted by $x(t)$
- Inside the phone, there is some DSP chip which do the sampling and the coding process
- Then some hardware to modulate the signal to transform it into RF signal for transmission
- At the receiver, the signal is demodulated back to the base band status

- Then a decoding and reconstruction process is done to reconstruct the original signal $x(t)$.

The goal is that we hope that $\hat{x}(t) \approx x(t)$.

We will use the Fourier transform to analyze the effect of sampling and tackle the problem of reconstruction, and also we ask several question to build an efficient system which can do this reconstruction:

- How fast do we need to sample ?
- How to reconstruct ?

2.3 Sampling Continuous Time Signals

2.3.1 Impulse Sampling Analysis using Fourier Transform

To do the mathematical derivation later.

2.3.2 Intuition and Interpretation

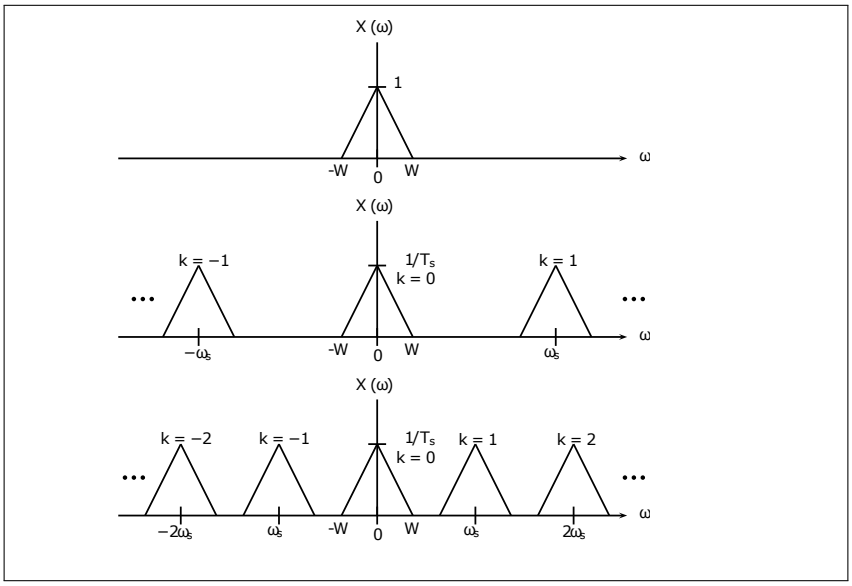


Figure 2.2: Sampling Higher then the Nyquite Rate

- When sampling higher then the Nyquist rate, we could still distinguish the original version of the spectrum, which is in the middle

Let's try now to decrease the sampling rate (ω_s decrease) as shown in Figure 2.3.

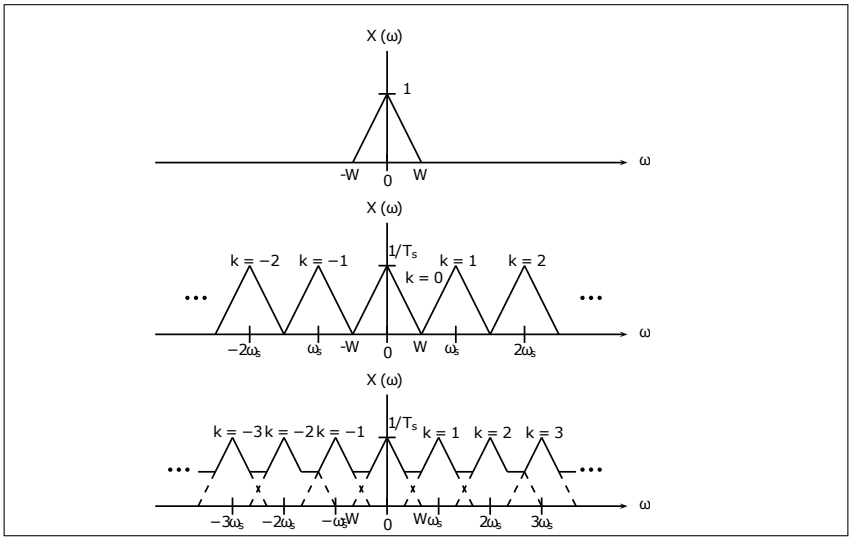


Figure 2.3: Decreasing the Nyquist Rate: Aliasing Result

- When decreasing the sampling rate ω_s, the shifted spectrum get close to each other
  - In the 2nd row of Figure 2.3, we are sampling exactly at the Nyquist rate
  - No gap between the original spectrum and the shifted version
- When sampling rate ω_s less then the ***Nyquist rate***, the shifted versions have collide with the original spectrum
  - The only thing we can see is the blue solid line
  - We could not retrieve the original spectrum as before now.

2.3.3 Example: Sampling a Cosine

Let's take the signal $x(t) = \cos(\pi t)$, and sample this signal at different sampling rate ω_s and study the effect for different values of ω_s.

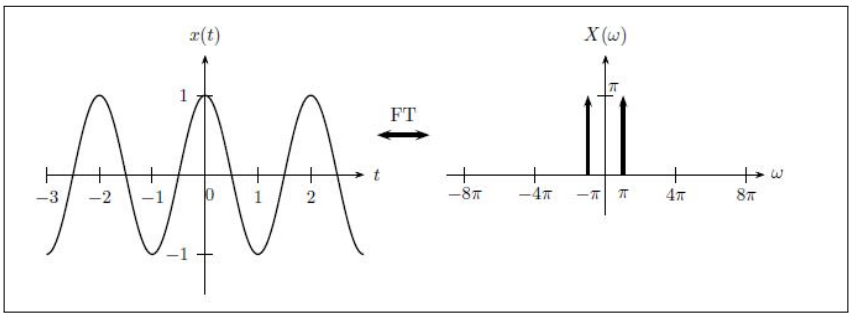


Figure 2.4: Example of Sampling a cos: period = 2 s

First case we take $\omega_s = 8\pi \rightarrow T_s = \frac{1}{4}$s. Case is shown in Figure 2.5.

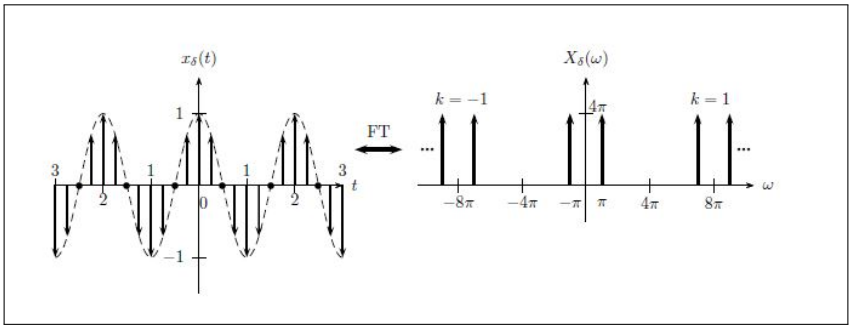


Figure 2.5: $\omega_s = 8\pi$; $T_s = \frac{1}{4}$ s

- The shifted spectrum are repeated at multiple of ω_s
- Don't forget that we have the scale factor $\frac{1}{T_s} = 4$

Case 2 shown in Figure 2.6: we decrease ω_s to become $2\pi \rightarrow T_s = 1$s. So we take less samples per second.

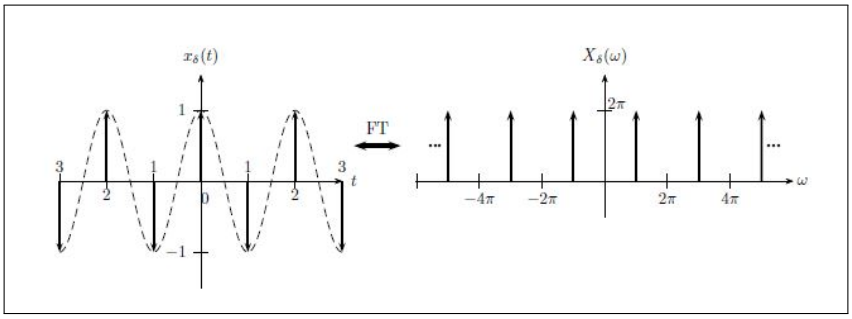


Figure 2.6: Exactly Sampling at Nyquist rate ; $\omega_s = 2\pi$; $T_s = 1$ s

- The case shown in Figure 2.6: we are sampling ***exactly at the Nyquist rate*** where $\omega_s = 2W$ and $W = \pi$.
- We are still able to see the original spectrum located at π and $-\pi$
- However notice that at the scaling factor: it is true that $\frac{1}{T_s} = 1$, but we have 2π
  - This is because when we shift the original spectrum at multiple of ω_s, the dirac shifted land on top of the original dirac , and added together

Last case shown in Figure 2.7 is we sample below Nyquist rate and aliasing effect occur also.

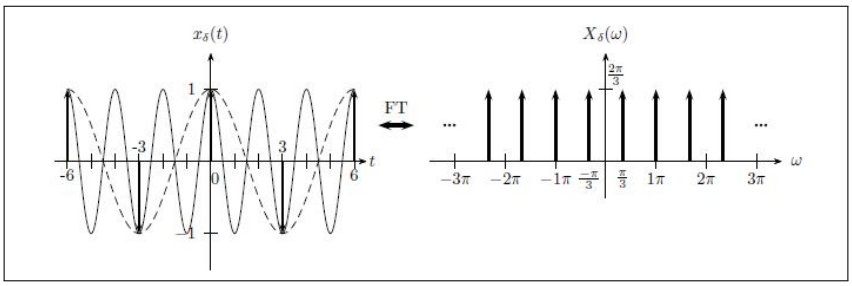


Figure 2.7: Sampling below Nyquist rate ; $\omega_s = \dfrac{2\pi}{3}$; $T_s = 3$ s

- Because we are sampling below the Nyquist rate, the reconstructed cosine appear to be a low frequency one
- This is can be deduced from the frequency spectrum: notice that when the spectrum is shifted at multiple of $\omega_s = \dfrac{2\pi}{3}$,
  - the dirac component located at $-\pi$ (this is the original one) is shifted to $\dfrac{-\pi}{3}$
  - Same for the dirac component located at $+\pi$ (this is also the original one) is shifted to $\dfrac{+\pi}{3}$
  - So it seems that the original spectrum is located at $\left[\dfrac{-\pi}{3}\,;\,\dfrac{+\pi}{3}\right]$

2.3.4 Predicting Aliasing frequency

We have seen in Figure 2.7 that when we sample below the Nyquist rate, aliasing occurred and the cos signal has transformed from high frequency $f = \dfrac{1}{2}$ Hz to $f_{\text{alias}} = \dfrac{1}{3}$ Hz. In this section we try to use some equations in order to predict in advance the aliasing frequency f_{alias}.

The equation are:

$$\begin{aligned} f_{\text{alias}} &\leq \frac{Fs}{2} \\ f_{\text{alias}} &= |f_{\text{original}} - m\, f_s| \quad \text{where } m = 0\,;\,1\,;\,2;\,\cdots \end{aligned} \tag{2.1}$$

An example is shown in Figure 2.8.

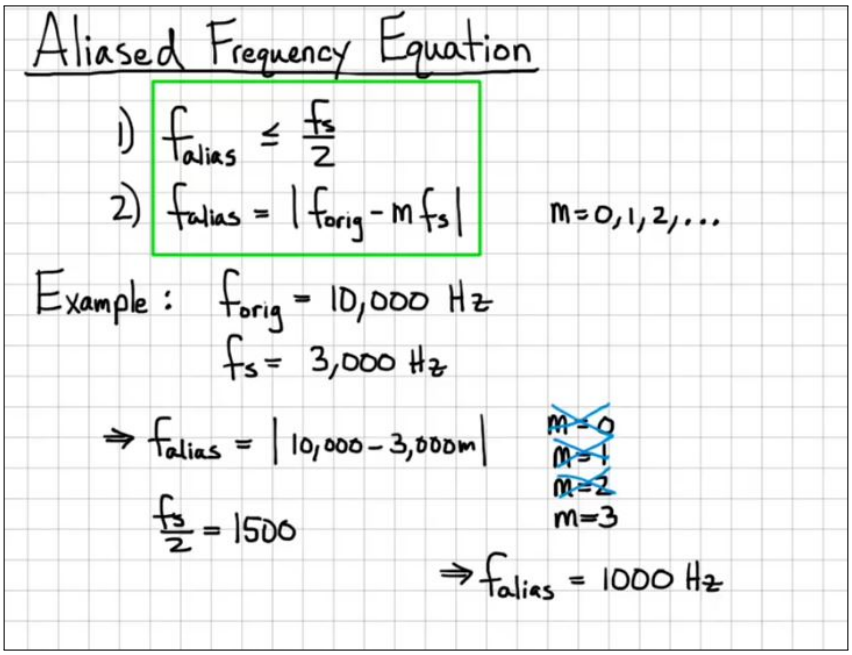


Figure 2.8: Predicting alias frequency

- We have some signal where $f_{\text{original}} = 10^4$ Hz
- We sample the signal with $f_s = 3000$ Hz
  - Clearly, aliasing will occur since we dind't respect the Nyquist condition
- We keep using 2.1:
  - We solve for $m = 0$, we obtain $f_{\text{alias}} = 10^4$. This f_{alias} doesn't satisfy the condition $f_{\text{alias}} \leq \frac{Fs}{2}$, so we keep solving for other value of m.
  - For $m = 3$, we get $f_{\text{alias}} = 10000$ Hz, and this frequency satisfy the condition $f_{\text{alias}} = 1000\,\text{Hz} \leq \frac{Fs}{2}$

So when we sample a signal of $f_{\text{original}} = 10^4$ Hz with $f_s = 3000$ Hz, we get an alias frequency of 1000 Hz.

2.4 Examples

Consider the analog signal: $x(t) = 4 + 2\cos\left(150\pi t + \frac{\pi}{3}\right) + 4\sin(350\pi t)$.

To be done later.

2.5 Other Practical Considerations

In this section we look to more sophisticated points related to sampling, like the fact that in ***real world signals***, they are always time limited, and the implication of this time limitation on sampling. Also we deals with several ***application to sampling*** like pulse modulation,$\cdots$.

2.5.1 Time Limited Signals

The time-bandwidth product also know as ***uncertainty principle*** is shown in equation (2.2)

$$\begin{aligned} T_d &= \left[\frac{\int_{-\infty}^{+\infty} t^2\,|x(t)|^2\,dt}{\int_{-\infty}^{+\infty} |x(t)|^2\,dt}\right]^{0.5} \\ B_w &= \left[\frac{\int_{-\infty}^{+\infty} \omega^2\,|X(\omega)|^2\,d\omega}{\int_{-\infty}^{+\infty} X|(\omega)|^2\,d\omega}\right]^{0.5} \\ T_d \times B_\omega &\geq 0.5 \end{aligned} \tag{2.2}$$

Equation (2.2) says that if we have a ***time limited signal*** , the ***frequency content*** of the signal will ***extent to infinity***, and hence this will cause a ***problem when doing sampling***. Sampling theorem tell us that we need to sample at the Nyquist rate F_s where F_s is at least 2 times higher than the maximum frequency of the signal. Since the maximum frequency is infinity, we can't sample at infinity, and ***this cause any frequency content above*** $\frac{F_s}{2}$ ***to fold with the original spectrum***.

This is illustrated in Figure 2.9.

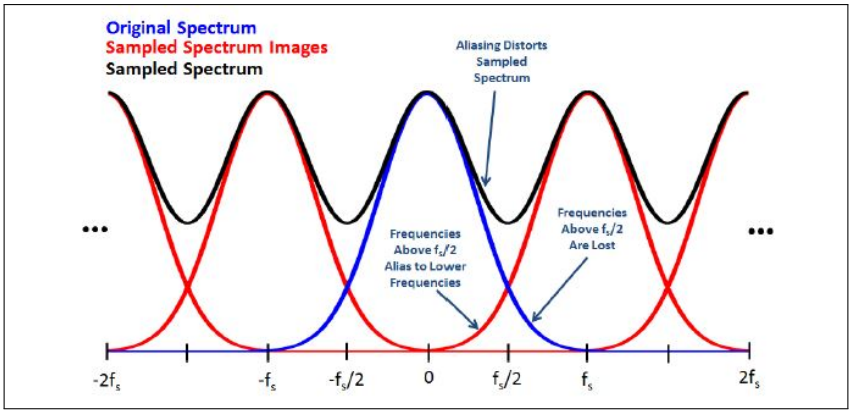


Figure 2.9: Signal Aliasing: frequency at $\frac{F_s}{2} + f_x$ shows at $\frac{F_s}{2} - f_x$

- We have the original theoretical spectrum in blue we wish to recover
- We know from sampling theorem that due to sampling, the spectrum will get replicated at multiple of F_s, and the resultant spectrum will be the summation of all the copies at every point, which give us the black spectrum.

Now when we wish to recover the spectrum (the blue one in Figure 2.9), we apply the ideal reconstruction by cutting above $\frac{F_s}{2}$ and below $-\frac{F_s}{2}$ as shown in Figure 2.10.

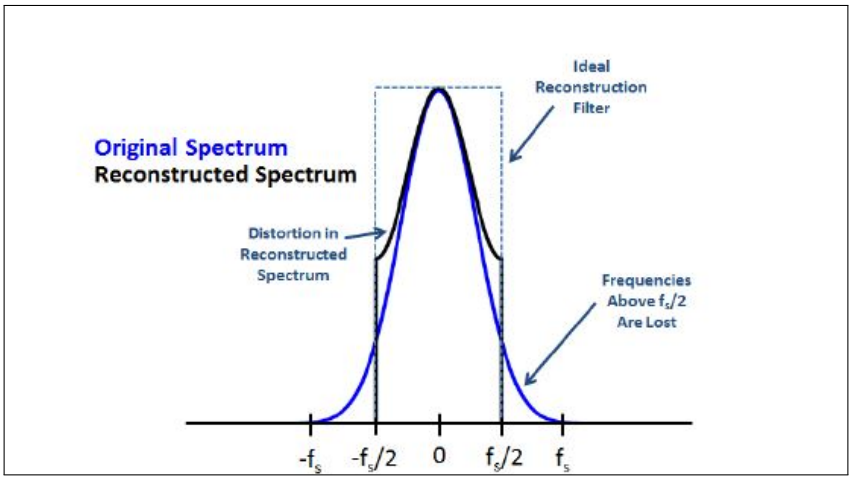


Figure 2.10: Reconstructed Signal is Distorted

However, instead of getting back to the blue (original one), we end up with the black one. The black spectrum contains ***2 type of distortion***:

1. Distortion due to folding (aliasing): the black amplitude values are the result from the summation of frequencies above and below $\pm\frac{F_s}{2}$, which fold back in the interval $\left[-\frac{F_s}{2}; +\frac{F_s}{2}\right]$

2. Distortion due to the loss of high frequency content: the original blue spectrum does indeed have high frequency content above $\frac{F_s}{2}$ due to the uncertainty principle.

2.5.2 Anti-Aliasing Filter

How can we fight the distortions introduced in Figure 2.10?

First let's talk about the aliasing distortion. We know that frequencies above $\frac{F_s}{2}$ will fold back to the original spectrum (as discussed in Figure 2.9), so what we can do is get off the frequencies above $\frac{F_s}{2}$ so they won't fold back when doing the reconstruction. In other words, we are limiting the signal to what it's called ***effective bandwidth*** when doing the cutting. Process is shown in Figure 2.11.

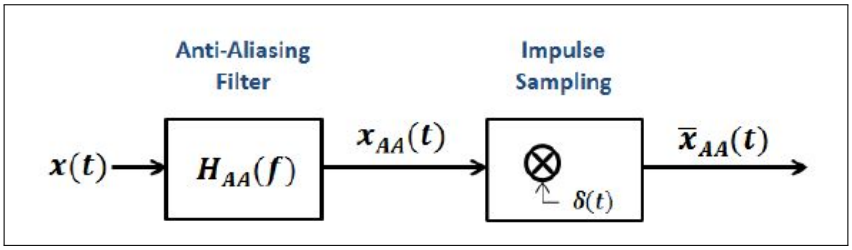


Figure 2.11: Anti Aliasing Filter

- We have some time limited signal $x(t)$
- We apply the anti-aliasing filter
- Then we do impulse sampling

Result of this process is shown in Figure 2.12.

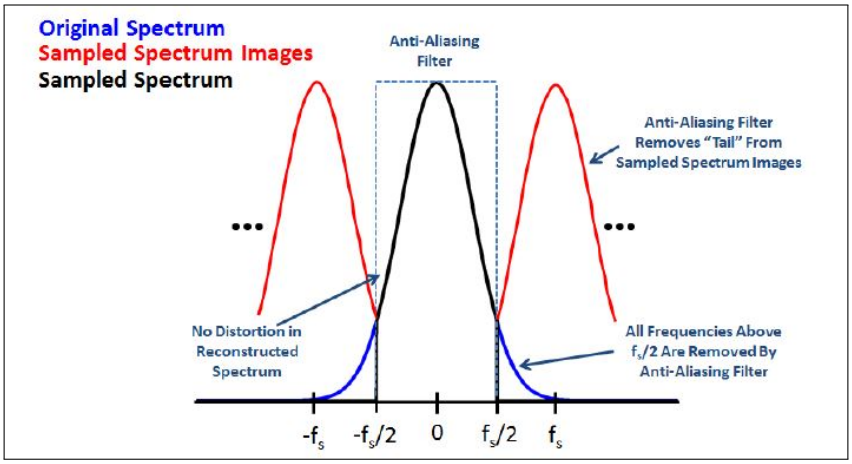


Figure 2.12: Anti Aliasing Filter: no folding frequencies

- If we compare Figure 2.12 to Figure 2.9, we see that the tails shown in Figure 2.9 of the replicated spectrum are gone now.

- However, we did do a compromise, by throwing away the high frequency content above $\frac{F_s}{2}$.
  - But this type of distortion can be controlled by increasing F_s and hence throwing less of these high frequency content.
  - So we need to do some engineering judgment about the choice of F_s and the notion of effective bandwidth

2.5.3 Practical Sampling with finite width pulse

So far, we have done reconstruction using ideal train of pulse, which have no width at all. However,these type of pulses don't exist in reality, and the pulses which do exist are with finite width. So can we do the same math and reconstruction using these finite width pulses as we did with the ideal train of pulses ?
The answer is yes we can and that's thanks to Fourier series as we will see later.
First we develop the theory of these computations, then we take some practical example.

Theoretical Development:

The steps of the theory are show in Figure 2.13 and Figure 2.14

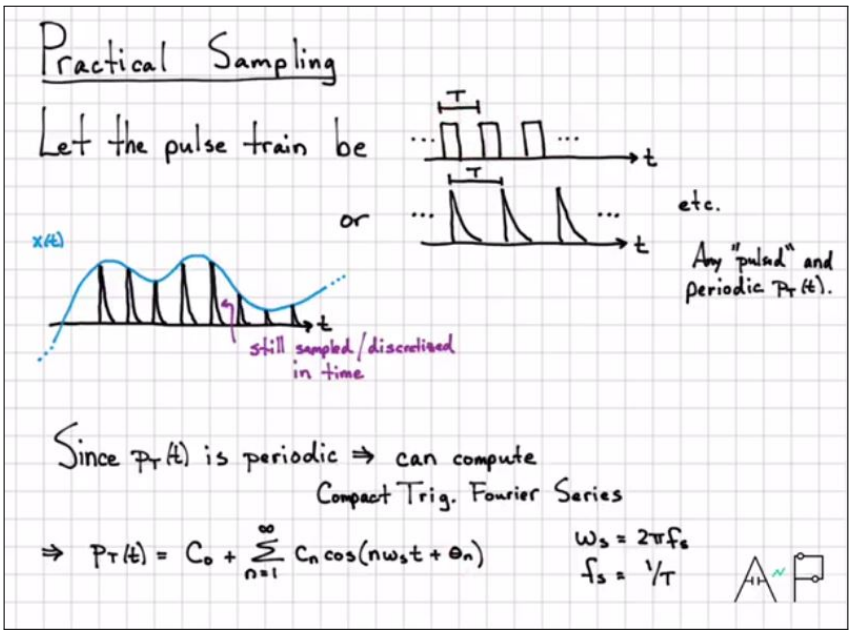


Figure 2.13: Practical Sampling using periodic finite width pulses

- We can use any form of ***periodic waveform*** (square, triangular) of period T_s, denoted by $p_{T_s}(t)$ where p stand for pulses
- We multiply $x(t)$ by these periodic pulses, and hence we localize the signal at some discrete time interval as shown in
- Since the resultant of the multiplication is periodic signal, we can use the Fourier series (any form of Fourier Series, here we use the compact form)

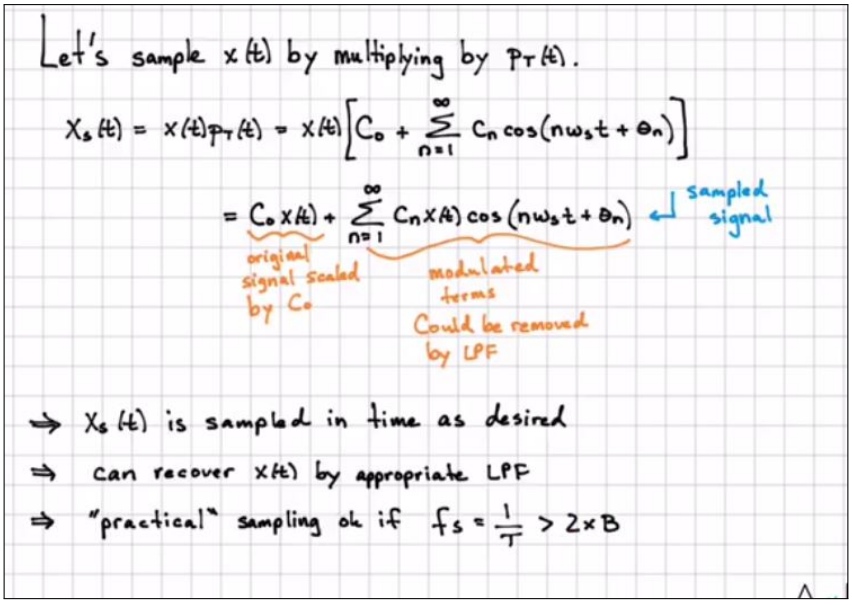


Figure 2.14: Practical Sampling using periodic finite width pulses: Fourier series

- Notice that the signal is composed from 2 terms:

  1. $C_0\, x(t)$ which is the signal scaled by a factor C_0
  2. The modulated terms $\sum_{n=1}^{+\infty} C_n\ x(t) \cos\left(n\,\omega_s\, t\ +\ \theta_n\right)$, which can be removed using low pass filter

Examples:

Given are shown in Figure 2.15.

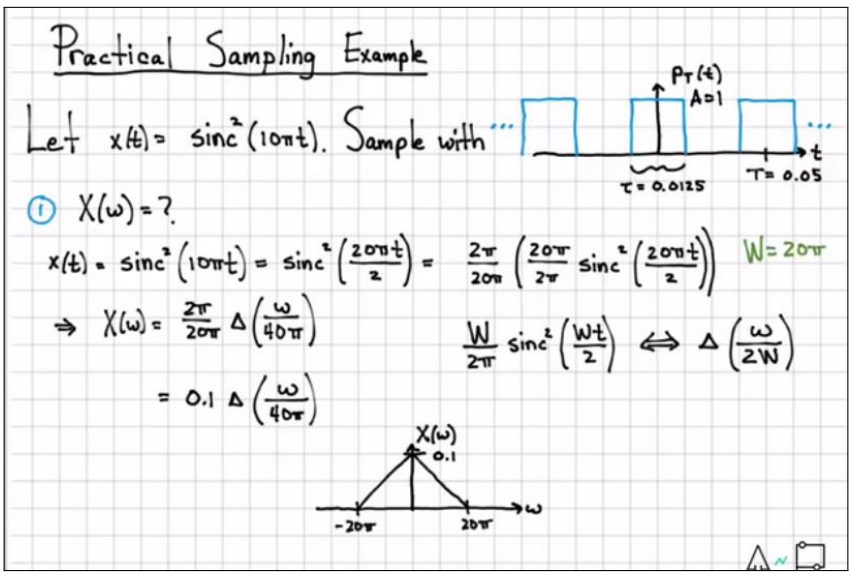


Figure 2.15: Practical Sampling Example

- The periodic waveform we used for sampling is a square wave where the sampling period (period of the square wave) is $T_s = 0.05$ s and pulse width $\tau = 0.0125\,\text{s} = \frac{T_s}{4}$
- $x(t) = \text{sinc}^2(10\pi t)$ has been reformulated so we can use a look up table to deduce $X(\omega)$ which is a triangular pulse of width 40π
- Computation and development of Fourier series shown in Figure 2.16, Figure 2.17 and Figure 2.18

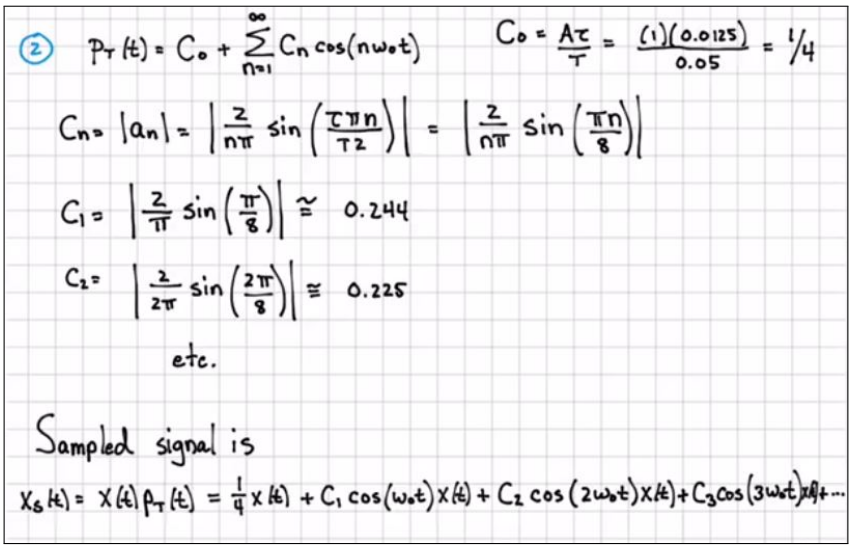


Figure 2.16: Practical Sampling Example: Computing Fourier Series

  - In Figure 2.16, first we develop the Fourier series of the square wave
  - As a 2nd step, the sampled signal will be the product between $x(t)$ and train of square wave pulses $p_{T_s}(t)$.

- Continue to expand $x_{\text{ZOH}}(t)$ (shown in Figure 2.17) where the cos is a dirac signal, and the multiplication in the time domain is a convolution in the frequency

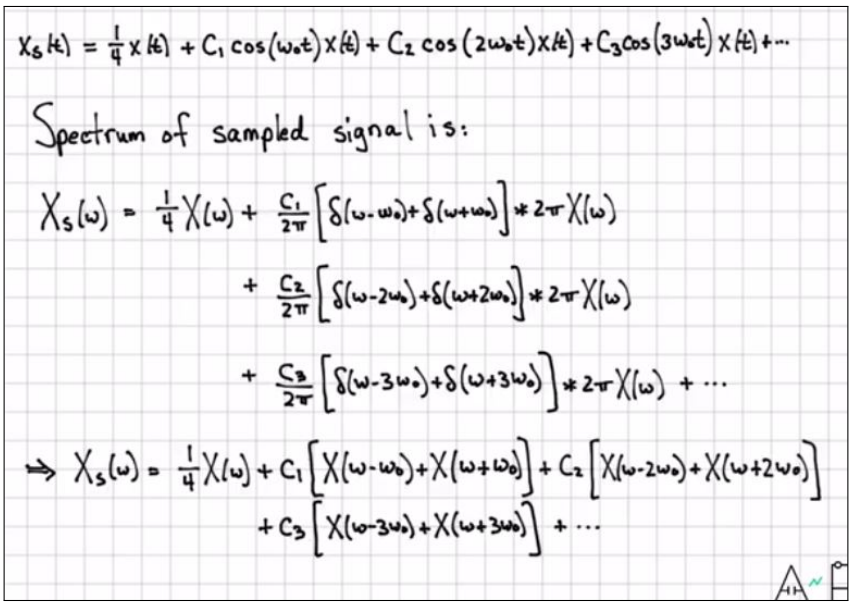


Figure 2.17: Practical Sampling Example: Computing Fourier Series

  - Then we use the convolution property of a signal with a dirac function

- Sketching the amplitude spectrum of $x_{\text{ZOH}}(t)$ (shown in Figure 2.18)

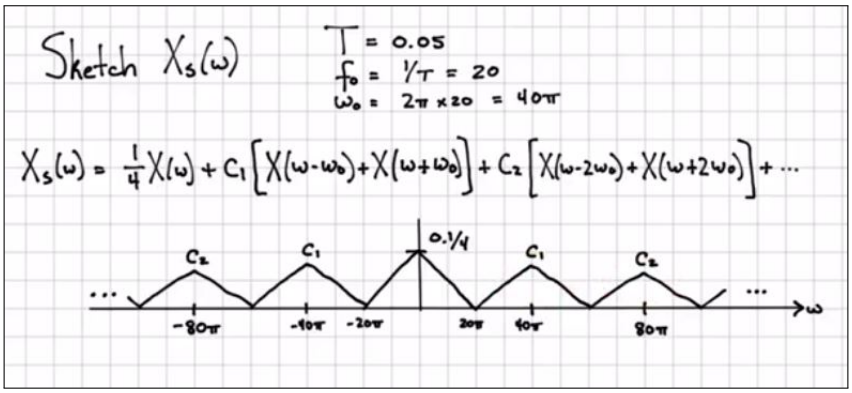


Figure 2.18: Practical Sampling Example: Amplitude Spectrum of $x_{\text{ZOH}}(t)$

2.6 Ideal Reconstruction

After we have seen the ideal sampling method, our goal now is to ***undo the effect of sampling*** and reconstruct back the continuous time signal.

Some people call this process ***reconstruction*** or ***interpolation***.

It should be noted that the ***results*** developed ***in this section*** are ***under the conditions that***:

1. The signal is in time is ***not time limited*** (so not truncation in time)
2. And its is ***bandlimited***

Practical sampling and reconstruction will be developed later in section 2.7.

2.6.1 Sampling theorem and the Uniqueness of Reconstruction

Suppose we have been given some set of discrete values as shown in Figure 2.19.

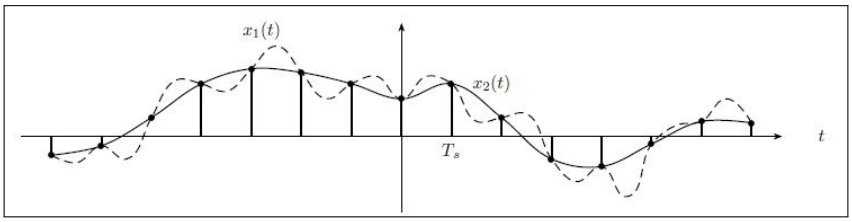


Figure 2.19: Which signal is the correct one ?

There are many ways to plot a signal using these discrete values: one can plot and obtain $x_1(t)$ and another can obtain $x_2(t)$, so how can we ensure which signal is the correct one?

We can do this using the frequency representation of the sampling studied in the previous sections and using the sampling theorem, which is stated formally in Figure 2.20.

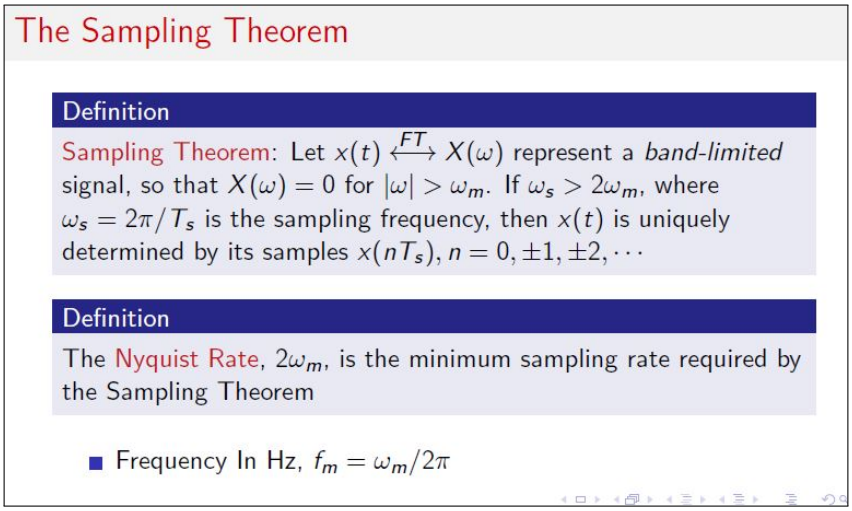


Figure 2.20: Sampling Theorem Definition

- Notice that sampling theorem is for ***band limited signal***
- The sampling theorem give us an ***upper bound*** that we should sample at least from $2\,\omega_m$ to avoid aliasing and reconstruct the signal back

2.6.2 Ideal (Sinc) Interpolation

Now we develop the mathematical model for reconstruction for impulse sampling case. Given that we have the impulse sampled spectrum as shown in Figure 2.21, how we can get back to the original spectrum ?

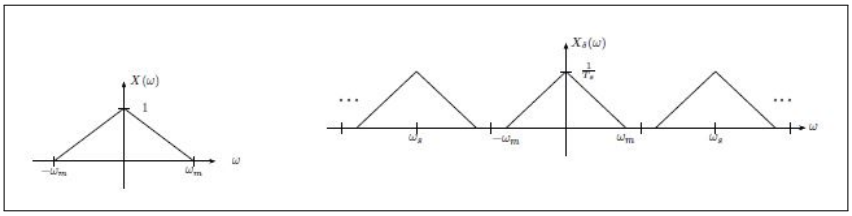


Figure 2.21: Ideal Reconstruction: Spectrum of Original Signal and Impulse Sampled Signal

The answer is that we need to get rid off of the replicate at multiple of ω_s, and then apply the proper gain for the filter because we have the scale factor $\frac{1}{T_s}$, and the original spectrum $X(\omega)$ has an max amplitude of 1 as shown in Figure 2.21.

The correspondent filter is shown in Figure 2.22.

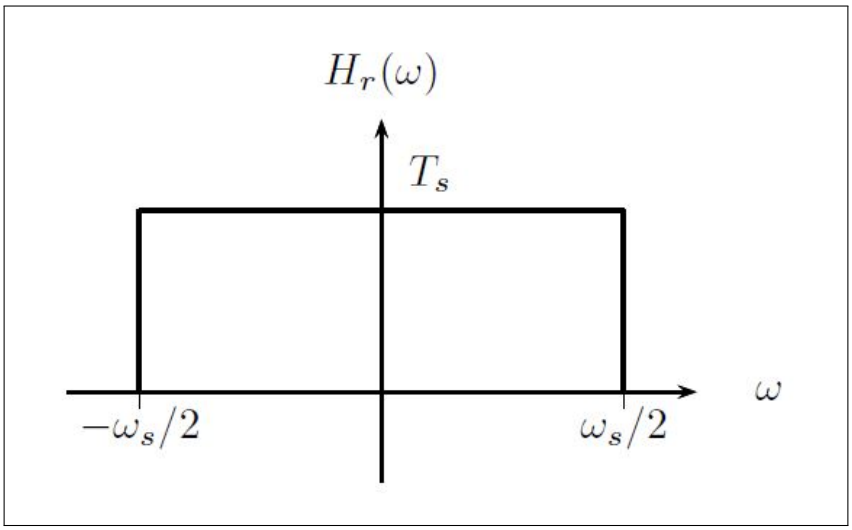


Figure 2.22: Frequency Response of Ideal Reconstruction Filter

Now we have done the reconstruction in the frequency domain, the computations (shown in Figure 2.23 and Figure 2.24) are also done in the time domain to understand what happens also, and how the reconstruction is done.

- We start from frequency domain formulation $X(\omega) = H_r(\omega) \times X_\delta(\omega)$
- Multiplication in the frequency domain is convolution in the time domain

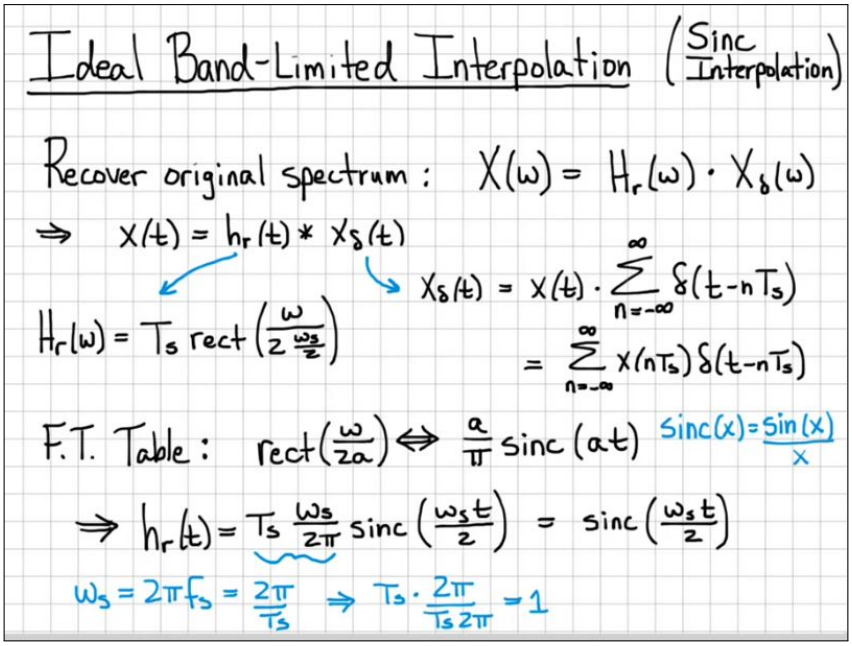


Figure 2.23: Computing $x(t)$ in the time domain starting from frequency domain formulation

- In $x_\delta(t)$, we can get $x(t)$ into the summation as $x(n\,T_s)$
- We use some look up table to see what is $h_r(t)$.
- Be aware to use the proper definition of $\text{sinc}(x)$

Final form of $x(t)$ is shown in Figure 2.24.

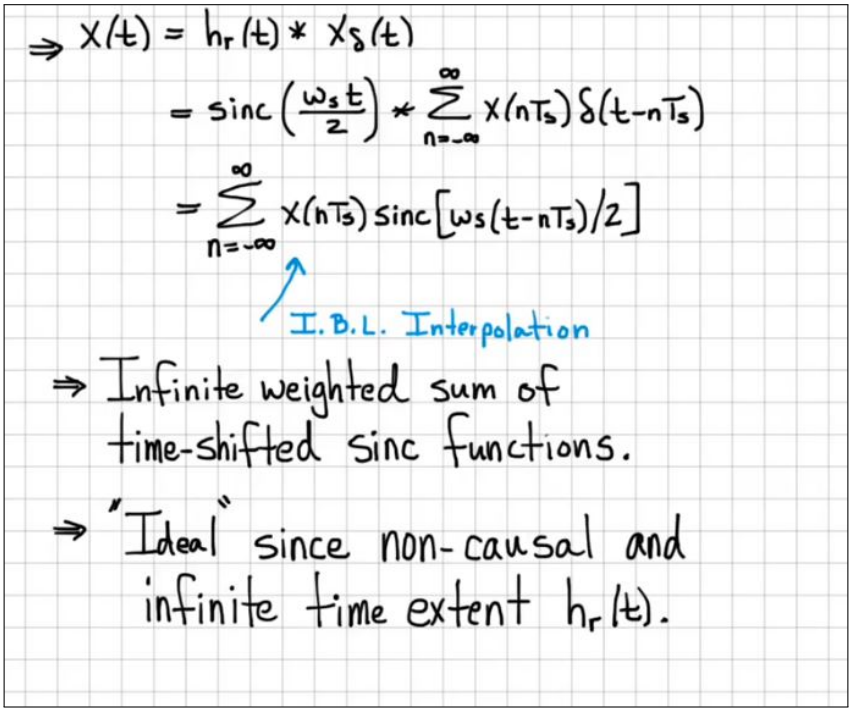


Figure 2.24: Expression of $x(t)$

- When we multiply some signal by impulse, it will be the signal shifted to the location of the impulse
  - So we get sinc into the summation and replace t by $t - nT_s$
- So $x(t)$ is the sum of shifted sinc, each weighted by a different amplitude $x(nT_s)$.

The term ideal is because the impulse response $h_r(t)$ is non causal (start before 0) and extent to infinity. A demonstration is shown in Figure 2.25.

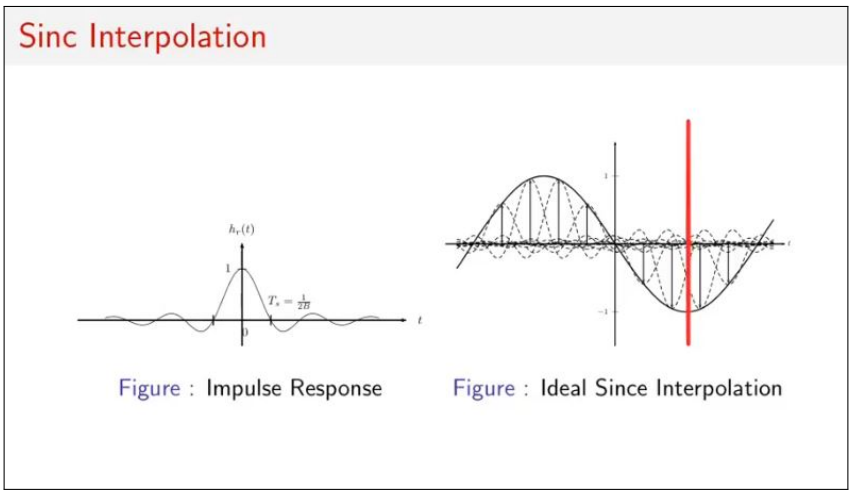


Figure 2.25: How the summation work

- We have all these infinite sinc, and at a given time we sum all the values to obtain some value of $x(t)$.

2.7 Practical Sampling and Reconstruction

Impulse sampling is one way we can get samples from the signal. Another way we can get samples is known as zero-order-hold (ZOH) as shown in Figure 2.26.

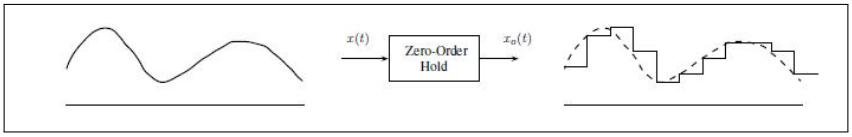


Figure 2.26: Sampling using ZOH

It is named zero-order-hold because this filter grab a value of the signal, and then hold this value till it grab the next value. So the ZOH filter returns a stair case like signal, where each value of the signal is held till the next sampling time occurred.

Mathematical modeling of the ZOH is shown in Figure 2.27.

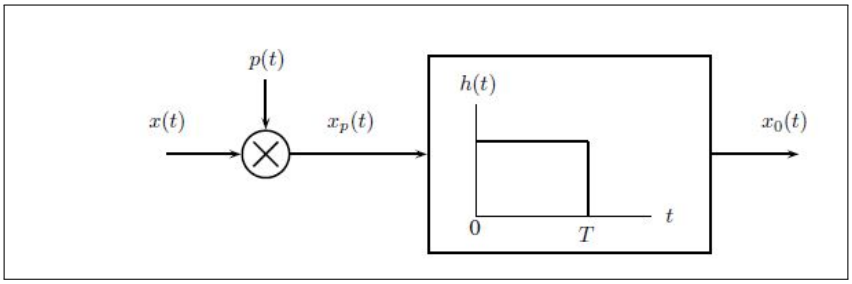


Figure 2.27: Mathematical modeling of the ZOH

We have our continuous time signal $x(t)$ multiplied by an infinite train of pulse $p(t)$, then we obtain like before $x_p(t)$ which is equal to $x_\delta(t)$, then we do some low pass filtering using a pulse of width T. Amplitude spectrum of ZOH is shown in Figure 2.28. Recall that when we work in the digital domain, the only thing we care about is between $\pm\omega_s$.

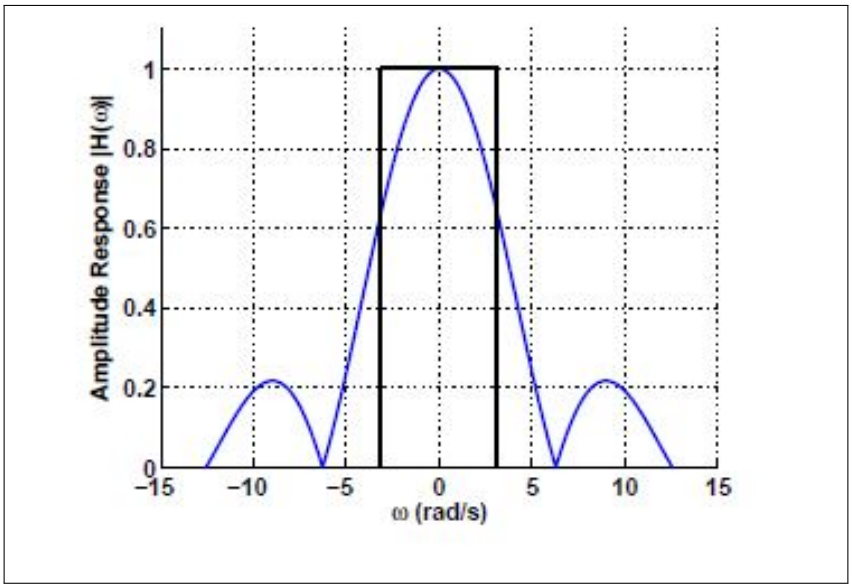


Figure 2.28: Amplitude Spectrum of ZOH: boxed region is between $\pm\omega_s$

Spectrum of the Signal Compared to Impulse sampling of Figure 2.2:

Spectrum of the signal $x(t)$ using ZOH is shown in Figure 2.29.

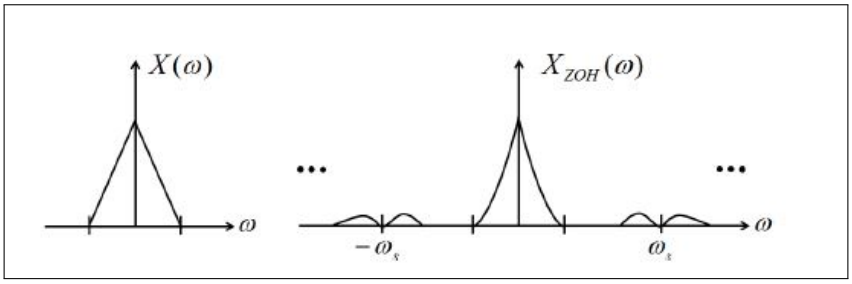


Figure 2.29: Output of ZOH in the frequency domain

- In impulse sampling (see Figure 2.2), the spectrum was the same (with the scale factor $\frac{1}{T_s}$), along with the replicate at multiple of ω_s.
- In ZOH, these replicate are attenuated due to the low pass nature of the ZOH, which is a good thing since we don't want these images.
- However, the original spectrum is getting distorted, and this distortion is controlled by the time sampling we choose to get our values.

2.7.1 Mathematical Formulation for ZOH

Ideal sampling followed by ideal bandlimited interpolation can't by done practically:

- Generating dirac pulses is very difficult
- In reconstruction, the value of $\hat{y}(t)$ at a value of t not multiple of T_s is obtained by infinite sum of weighted sinc function
- Ideal filters are also can't be made

ZOH is the practical way to grab a signal, which is done by taking a sample of the signal, and holding this value during the sampling interval.

Let's denote the rectangular pulse we are using by $h(t)$ denoted by:

$$h(t) = \begin{cases} 1 & 0 \leq t \leq T_s \\ 0 & \text{otherwise} \end{cases} \tag{2.3}$$

The x_{zoh} can be written as:

$$x_{\text{zoh}} = \sum_{n=-\infty}^{\infty} x(n) h\left(t - n T_s\right) \tag{2.4}$$

Recall that we can obtained a shifted version of a signal by convolving this signal with a dirac function so equation (2.4) can be further formulated:

$$\begin{aligned} x_{\text{zoh}} &= \sum_{n=-\infty}^{\infty} x(n)\, h\,(t - n\,T_s) \\ &= \sum_{n=-\infty}^{\infty} x(n) \times [\,\delta\,(t - n\,T_s) * h(t)\,] \end{aligned} \tag{2.5}$$

Manipulating the parenthesis of equation (2.5):

$$\begin{aligned} x_{\text{zoh}} &= \sum_{n=-\infty}^{\infty} x(n)\, h\,(t - n\,T_s) \\ &= \sum_{n=-\infty}^{\infty} x(n) \times [\,\delta\,(t - n\,T_s) * h(t)\,] \\ &= \sum_{n=-\infty}^{\infty} [\,x(n) \times \delta\,(t - n\,T_s)\,] * h(t) \\ &= x_\delta(t) * h(t) \end{aligned} \tag{2.6}$$

Taking the Fourier transform of (2.6) we get:

$$\begin{aligned} X_{\text{zoh}}(\omega_c) &= X_\delta(\omega_c) \times H(\omega_c) \\ &= \frac{1}{T_s} \sum_{n=-\infty}^{\infty} X_a\,(\omega_c - n\,\omega_s) \times 2\exp\left(-j\,\frac{\omega_c\,T_s}{2}\right) \times \text{sinc}\left(\frac{\omega_c\,T_s}{2}\right) \end{aligned} \tag{2.7}$$

Since the rectangular pulse of Figure 2.27 is not centered at 0, and it is shifted by $\frac{T_s}{2}$, we obtain a phase shift of $2\exp\left(-j\,\frac{\omega_c\,T_s}{2}\right)$ multiplying the sinc function.

2.7.2 Reconstruction for ZOH

Time delay by the factor of $\frac{T_s}{2}$ can't be corrected, we have to deal with it. However, the distortion in original spectrum and the replication can be fixed.

Key notes in designing anti-imaging filter:

- the key difficulty is in the transition band, where we need to suddenly pass to 0 in the stop band
- This can be easily achieved if we have a high transition band, which implies that $\omega_s >>$ bandwidth signal
- We later get back to this point when covering oversampling and undersampling, and how they can make the design of the anti-imaging filter less or more expensive.

2.8 Spectral Sampling

After we have been dealing with the sampling theorem, which tells what are the minimum number of samples we need from the time domain to construct a correct waveform, we move now to the dual form, which is ***sampling in the frequency domain*** and derive the number of samples in the spectrum required to construct the time domain signal. We are expecting similar results due to the time-frequency duality we now from Fourier transform lessons.
Explanation shown in Figure 2.30.

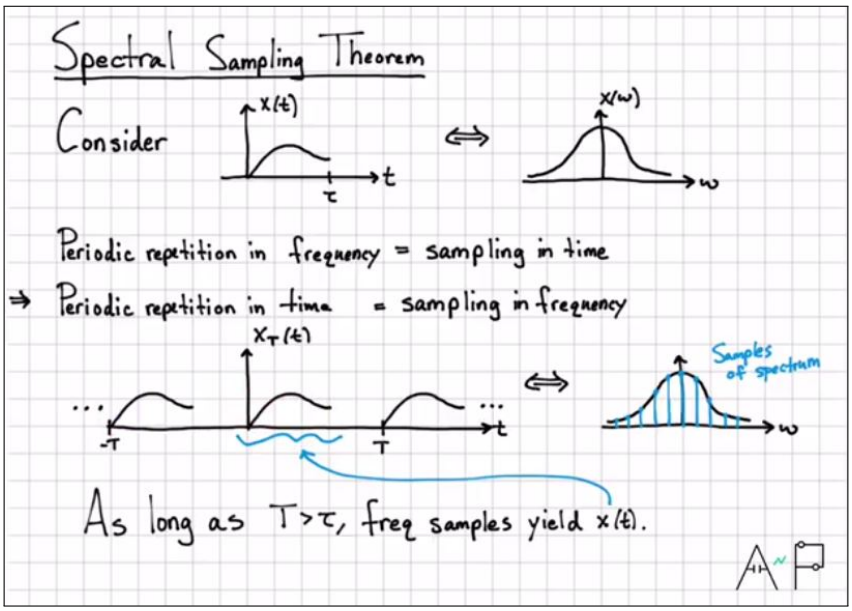


Figure 2.30: Replicating the time signal give us samples in the frequency domain

- We are assuming we have a time limited domain signal $x(t)$ of duration τ.
- We know from section 2.3 that sampling in time give us replication of the spectrum in the frequency domain
  - The dual form now is that replicating in time give us samples in the frequency domain
  - More generally, sampling in one domain give us replication in the 2nd domain

Now moving to deal with the mathematical development which relates the samples of $X(\omega)$ with $X(\omega)$ itself, shown in Figure 2.31.

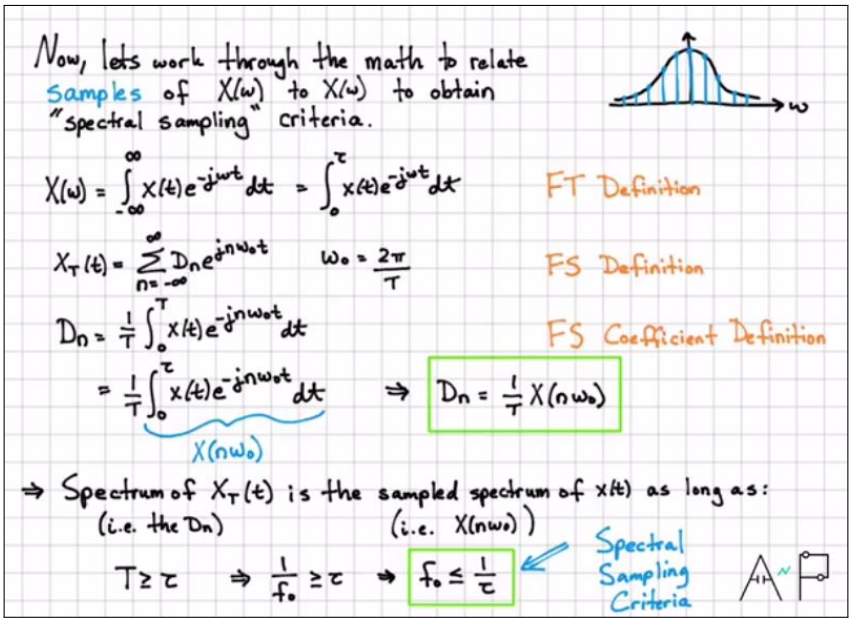


Figure 2.31: Mathematical formulation between samples of $X(\omega)$ and $X(\omega)$ itself

- We start by Fourier transform definition
- Since we are replicating in the time domain now, we can use Fourier series on $x_T(t)$
- Notice the relation between D_n and $X(\omega)$: they are the same except for the argument of the exponential:
  - In $X(\omega)$, we have ω
  - In D_n, we have $n\,\omega_0$

The ***spectral sampling theorem*** tells us: if we want to ***describe a continuous time-limited signal*** from its ***spectrum samples***, we need to make sure that the ***frequency spacing*** $\left(f_0 = \frac{1}{T} \right)$ is less than the inverse of the signal duration $\frac{1}{\tau}$

2.9 Applications

Sampling theorems let us replace a continuous time signal with a discrete set of numbers, so all we have to do now is to process these discrete numbers.

In this section we will look to some application of sampling, like how we can use the sample values to change the characteristic of a pulse.
A pulse can have different characteristics, like amplitude, width, position,$\cdots$. We start first by the following type of pulse modulation: PAM, PWM, PPM, and then treat PCM separately because it is the most sophisticated one among them.

2.9.1 PAM, PWM and PPM

Figure 2.32 shows different type of pulse modulation

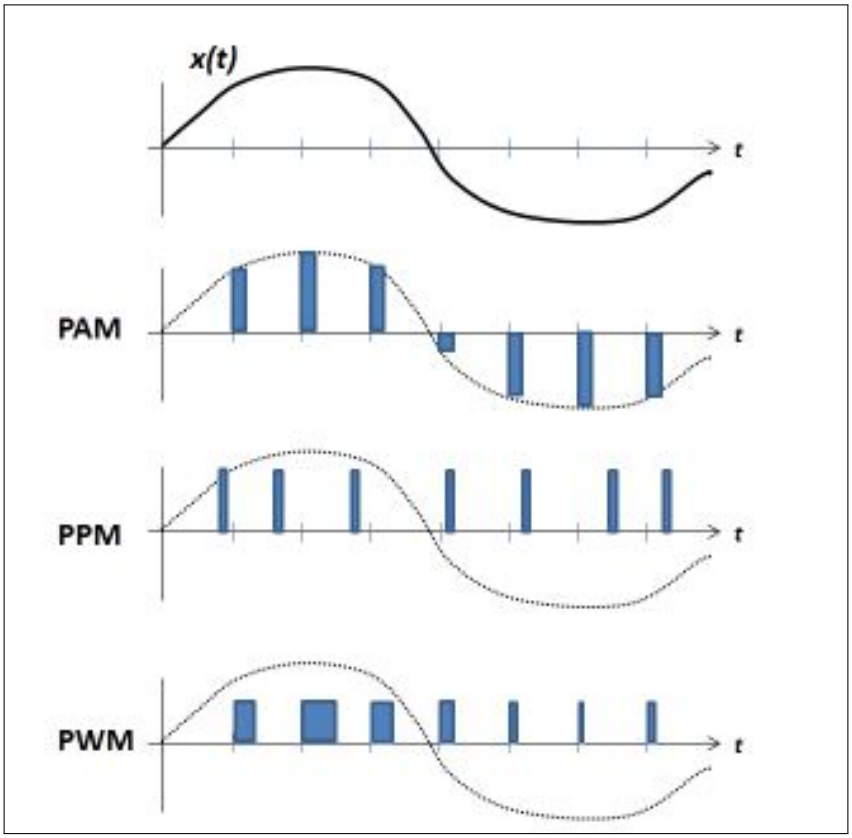


Figure 2.32: Pulse Modulation

- In PAM: the more the sample value is higher, the amplitude of the pulse is higher
  - The more the value is negative, same of the amplitude of the pulse
- In PPM, the width and amplitude of the pulses are all the same. However, the position of the pulse change
  - If the value is more positive, the pulse become more early before the sample value
  - If the value is negative, the pulse will be delayed and come after the sample value
  - So we can now about sample value depending if the pulse arrive more early (more positive) or more delayed (more negative)

2.9.2 Quantization and A/D Process

Before getting into PCM, we need to do learn some concepts whihc are prerequist to PCM. Let's start by some definitions:

- ***Digital Signal:*** discrete time signal and also discrete amplitude values

  - Another definition of digital signal is that can be any continuous time signal which represents a stream of bits, like in PCM

- ***Quantization:*** the process of rounding continuous valued amplitudes to a set of discrete amplitudes

  - Continuous-Time Signals Are Converted To Digital Signals By Sampling and Quantizing (A/D converters do this)

- L-ary signal: A digital signal that can take on L different values

  - Advantages of digital signal: resiliant to noise, use of repeaters, error correction scheme more easy on digital signal (as opposite to continuous time signal which can take any value),

Example of Quantization:

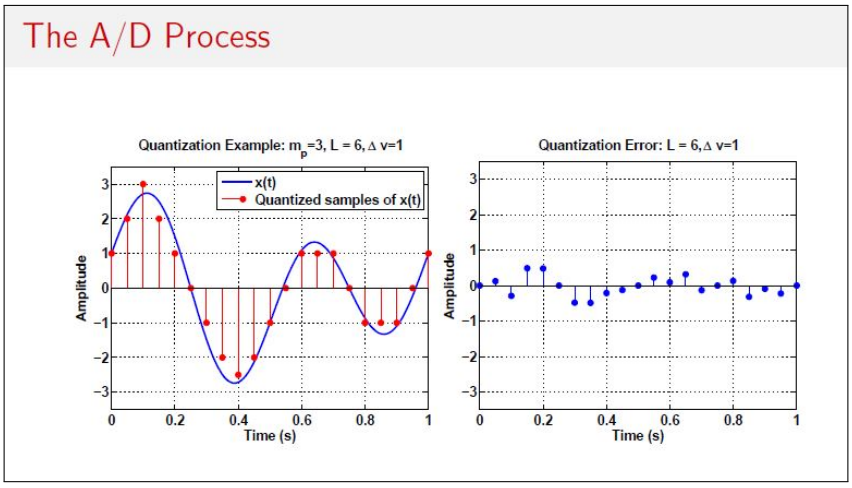


Figure 2.33: Quantization Example

- In A/D process, first we sample the signal then we round up the value to a defined level

- The parameters of the quantization (rounding) are:

  1. The number of levels L: here we have 6 levels
  2. The step size between each level δv
  3. The peak value of the signal m_p

- Now ***rounding introduce errors***, known here as ***quantization error***
  - If we want to decrease the quantization error, we need to increase the number of levels L as Figure 2.43
  - However we need to allocate more bits to represents all of these levels (trade off)

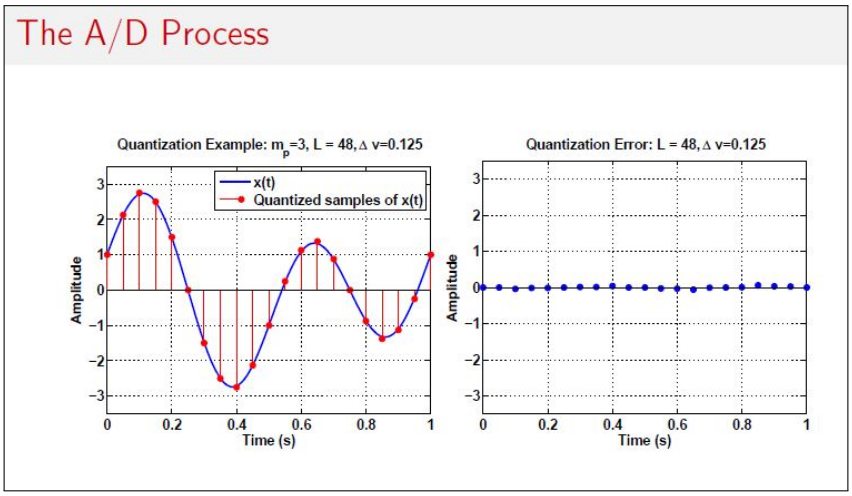


Figure 2.34: Increasing number of levels L to decrease quantization error

2.9.3 PCM

PCM is a modulation scheme where we represent a stream of bits (after sampling and quantizing) into a sequence of pulses. An example is shown in Figure 2.35

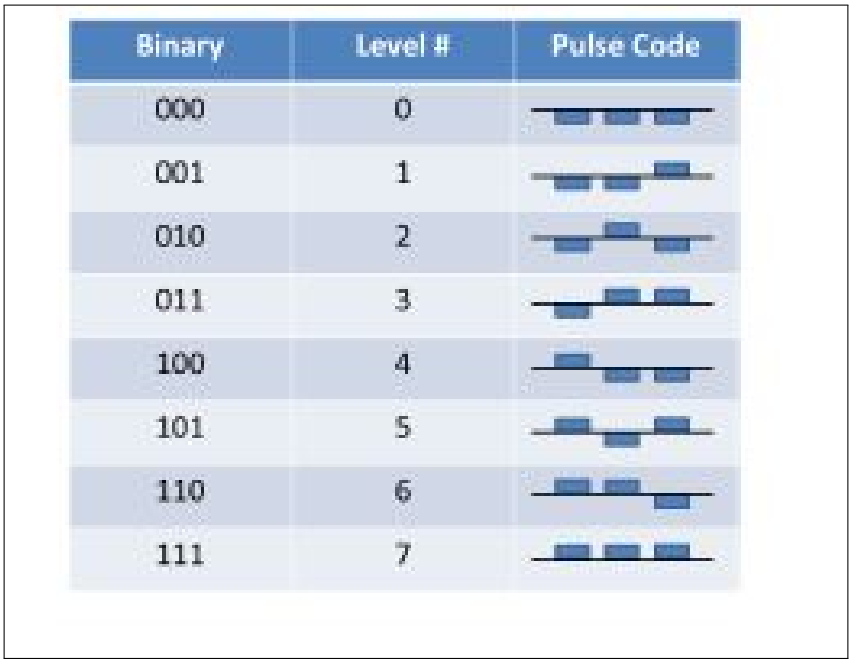


Figure 2.35: PCM modulation using $L = 8$ levels

- After finishing the A/D process, we do mapping to these discrete numbers into a stream of bits
- The the stream of bits is represented using pulses
  - 1 using an positive pulse and 0 using a negative pulse

2.10 Practical Digital Filtering and Oversampling

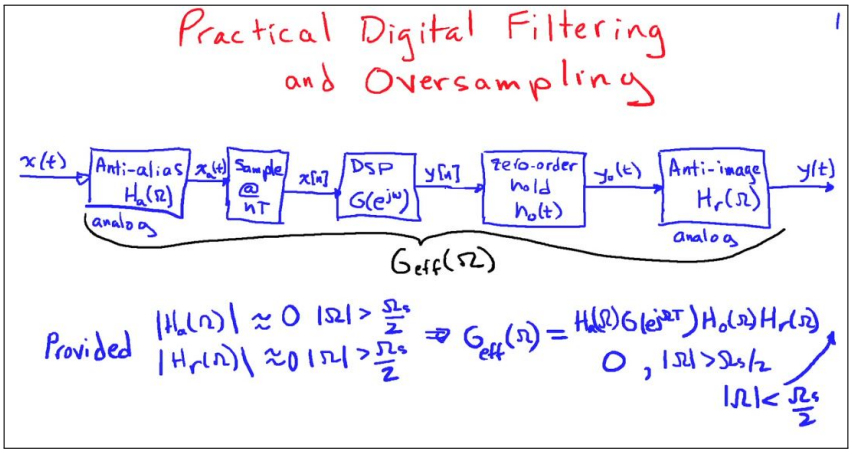


Figure 2.36: Overall Sampling and Reconstruction process

- The DSP system is in the discrete frequency domain ω_d

We can replace the whole system by a equivalent analog filter called $G_{\text{eq}}(\omega_c)$ where the transfer function is the multiplication of each transfer function block.

2.10.1 Building Analog filters

There are 2 filters presented in Figure 2.36 which are both ***analog***: the anti-aliasing filter $H_{\text{ant}}(\omega_c)$ which cut high frequency to ***prevent folding*** and anti-imaging filter $H_{\text{ant-img}}(\omega_c)$ to ***smooth the signal*** after the ZOH operation.

- For the anitliasing filter: if $\frac{\omega_s}{2}$ is close to the bandwidth of the signal ω_{signal}, this means that the ***transition band*** of the filter is ***very narrow*** and that's ***hard to implement in analog filters***.
- For the anti-imaging filter: in the pass-band of the filter, we need to implement the inverse of the sinc to compensate the distortion made by the main lobe of the ZOH filter.

  We need to make ω_s larger compared to the bandwidth of the signal so we don't have narrow band transition and the pass band

So in summary, we need a high sampling rate. But in implementation, this comes at the cost excessive memory storage, because if my sampling rate is high, I have more data to process in a 1s, which means we need more memory for the storage.

So there is a trade-off between a high sampling rate and the memory needed for storage. As a solution, we will change the sampling rate in the system of Figure 2.36:

- Decrease the sampling rate after the ADC process, right before the discrete processing
- Increase the sampling rate right before the ZOH

The new process including upsampling and downsampling is shown in Figure 2.37.

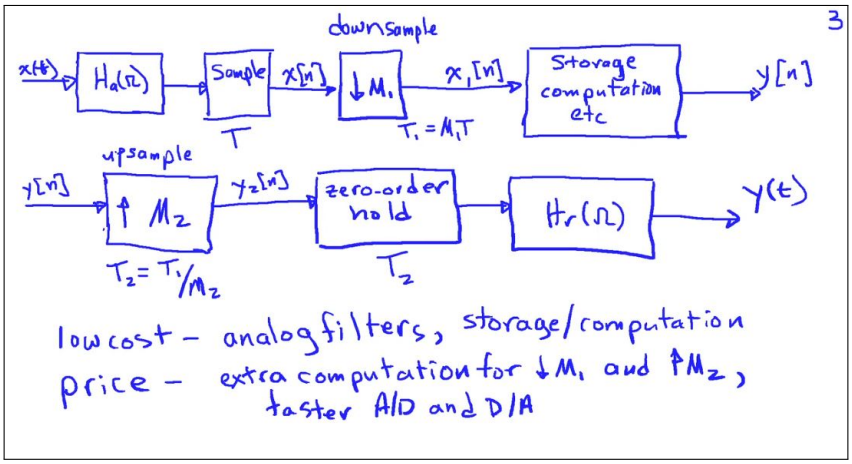


Figure 2.37: Upsampling and downsampling for lower cost analog filters

- There are many efficient numerical methods to implement the up and down sampling operations
- The price we pay in these discrete operations is worthy and make the design of the analog filters (anti-aliasing and anti-imaging) less expensive

2.10.2 Example of Oversampling

Given are shown in Figure 2.38.

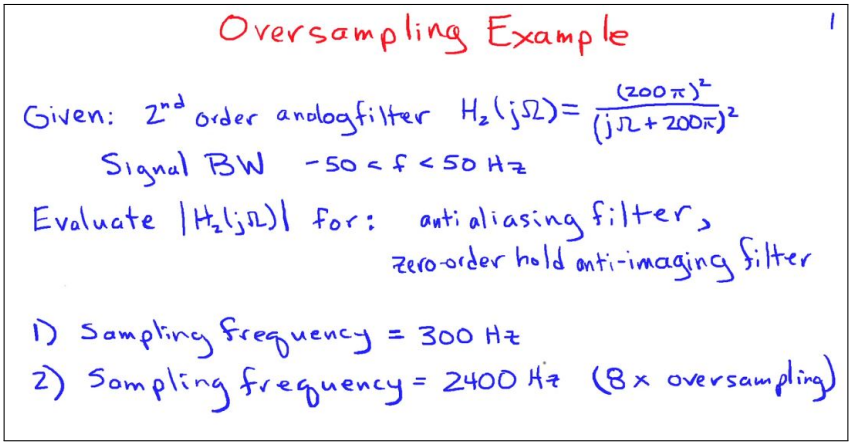


Figure 2.38: Given Example for oversampling

The goal of this exercise is to show how oversampling can serve the analog filters (anti-aliasing and anti-imaging).

- The filter is an analog 2nd order filter
- The signal bandwidth is 50 Hz.
- 2 sampling rate which both satisfy the Nyquist rate will be tested

Solution:

1. For the anti-aliasing filter:

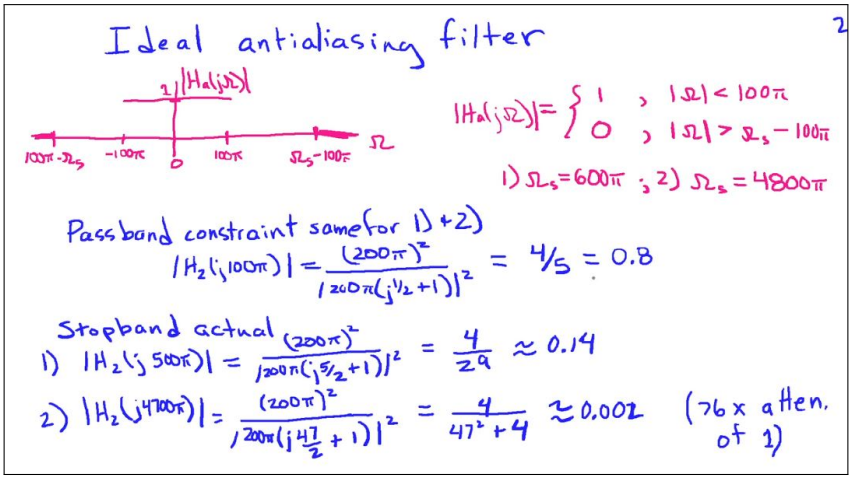


Figure 2.39: Anti-aliasing filter solution

- The ideal anti-aliasing filter has an amplitude response of 1 in the pass-band and 0 in the stop band.

- Then we plug the computation for both ω_s and see the results
  - in the pass-band, the filtering is independent from ω_s so for both sampling rate, we have $\mid H_{\text{ant-alias}}(\omega_c) \mid = 0.8$
  - In the stop band we have the difference because the stop-band depend on ω_s

2. For the anti-imaging filter:

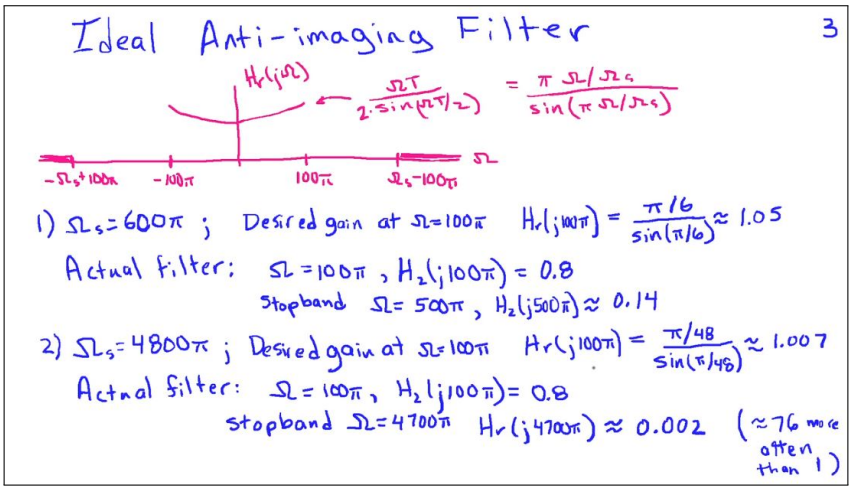


Figure 2.40: Anti-imaging filter solution

- The ideal anti-imaging filter has the inverse of the sinc function as a frequency response
- Also, the benefit is in the stop band, where oversampling is giving us more attenuation then the $F_s = 300$ Hz

2.10.3 Oversampling in D/A

Figure 2.41 shows the D/A process.

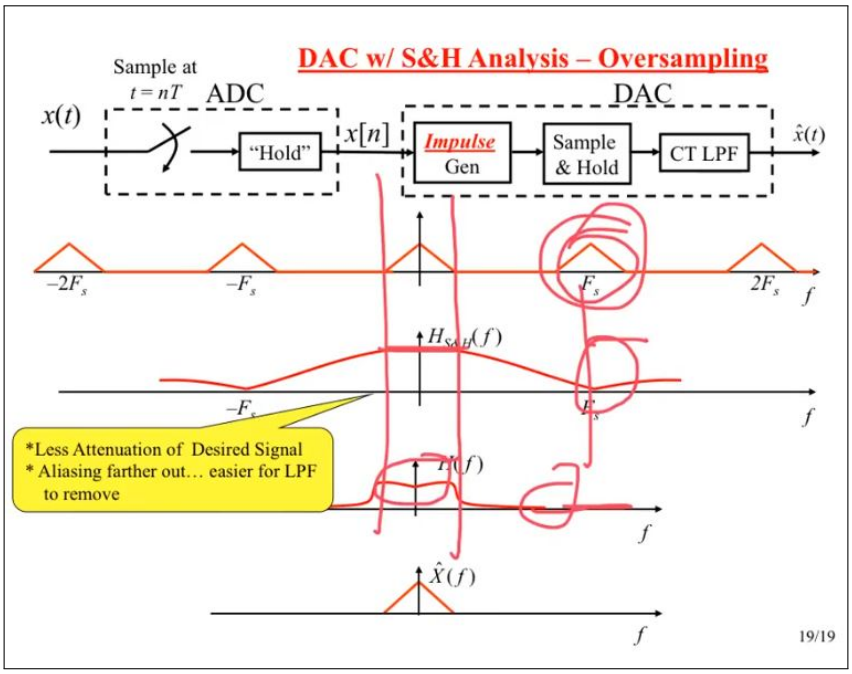


Figure 2.41: Over sampling Benefits

Oversampling have several benefits:

- Make the frequency response of the ZOH system more flat in the pass band
- Easier transition band design for the smoothing filter

2.11 Quantization and A/D Process

In this section, we study quantization and its error.
Reminder:

- ***Digital Signal:*** discrete time signal and also discrete amplitude values
  - Another definition of digital signal is that can be any continuous time signal which represents a stream of bits, like in PCM
- ***Quantization:*** the process of rounding continuous valued amplitudes to a set of discrete amplitudes
  - Continuous-Time Signals Are Converted To Digital Signals By Sampling and Quantizing (A/D converters do this)
- L-ary signal: A digital signal that can take on L different values
  - Advantages of digital signal: resiliant to noise, use of repeaters, error correction scheme more easy on digital signal (as opposite to continuous time signal which can take any value),

Example of Quantization:

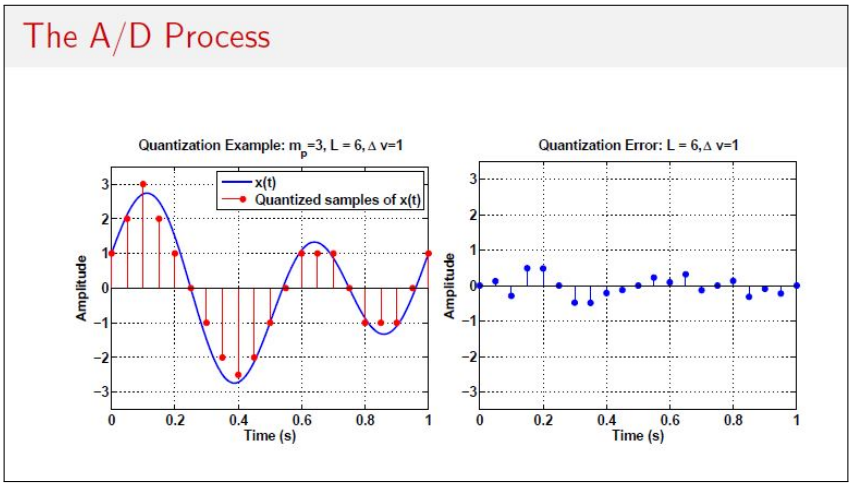


Figure 2.42: Quantization Example

- In A/D process, first we sample the signal then we round up the value to a defined level
- The parameters of the quantization (rounding) are:
  1. The number of levels L: here we have 6 levels
  2. The step size between each level δv
  3. The peak value of the signal m_p

- Now ***rounding introduce errors***, known here as ***quantization error***
  - If we want to decrease the quantization error, we need to increase the number of levels L as Figure 2.43
  - However we need to allocate more bits to represents all of these levels (trade off)

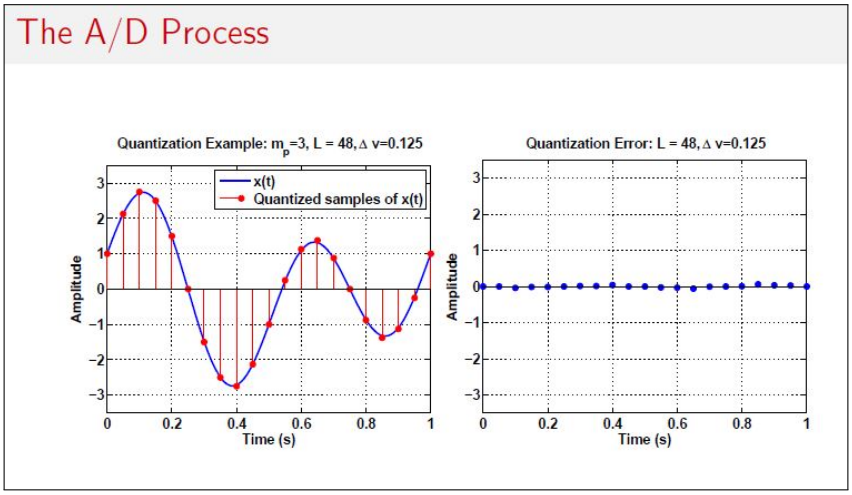


Figure 2.43: Increasing number of levels L to decrease quantization error

Qunatization Formula:

$$x_{\text{qunatized}} = \text{round}\left(\frac{x}{\text{step size}}\right) \times \text{step size} \tag{2.8}$$

2.11.1 Dynamic Range of an Ideal ADC

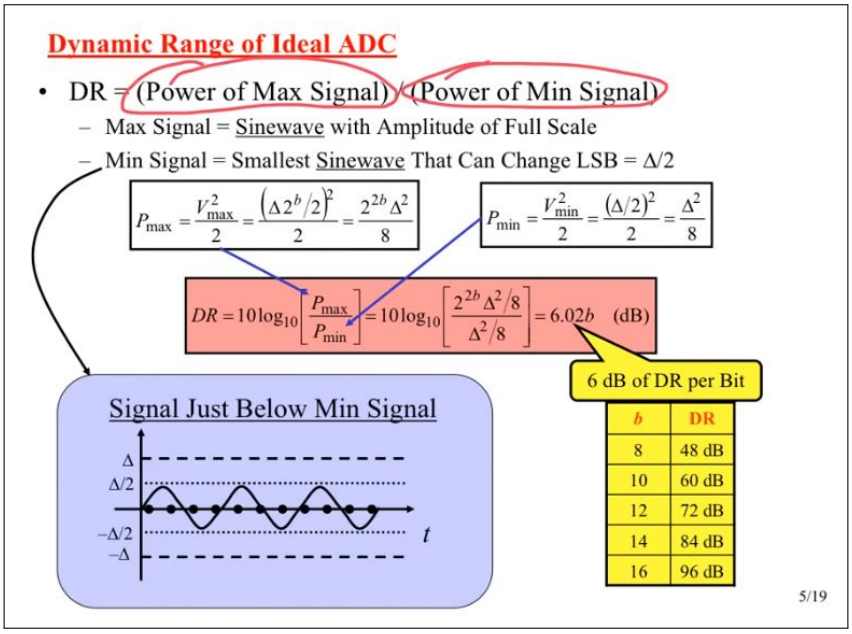


Figure 2.44: Dynamic Range of an Ideal ADC

2.11.2 Error Analysis of the Quantization Error

Since qunantization error is unpredictable, we use statistical analysis and probability laws to describe the error.

$$\begin{aligned} e_q[k] &= x[k] - x_q[k] \\ \frac{-\text{step}}{2} &\leq e_q[k] \leq \frac{+\text{step}}{2} \\ \text{step} &= \frac{\text{Range}}{\text{nb of Levels}} \\ &= \frac{\text{Range}}{2^{(\text{nb of bits} + 1)}} \end{aligned} \tag{2.9}$$

Error Analysis is shown in Figure 2.45.

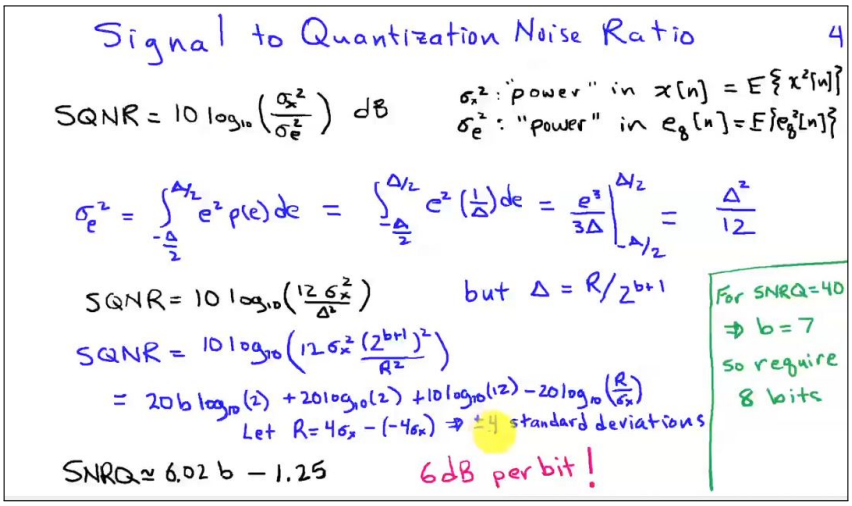


Figure 2.45: Qunatization Error Analysis

- $$\begin{aligned} \text{SNQR} &= 10\log_{10}\left(\frac{\sigma_x^2}{\sigma_e^2}\right) \\ &\approx 6.02 - 1.25 \text{ dB/bit} \end{aligned} \tag{2.10}$$

- The analysis done is under particular assumptions for this statistical analysis:

  1. $e_q[k]$ is a stationary random process with a uniform continuous distribution
  2. $e_q[k]$ values are uncorrelated
  3. $e_q[k]$ and $x[k]$ are independent

In Figure 2.46, we have an example of quantized and non-quantized sin.

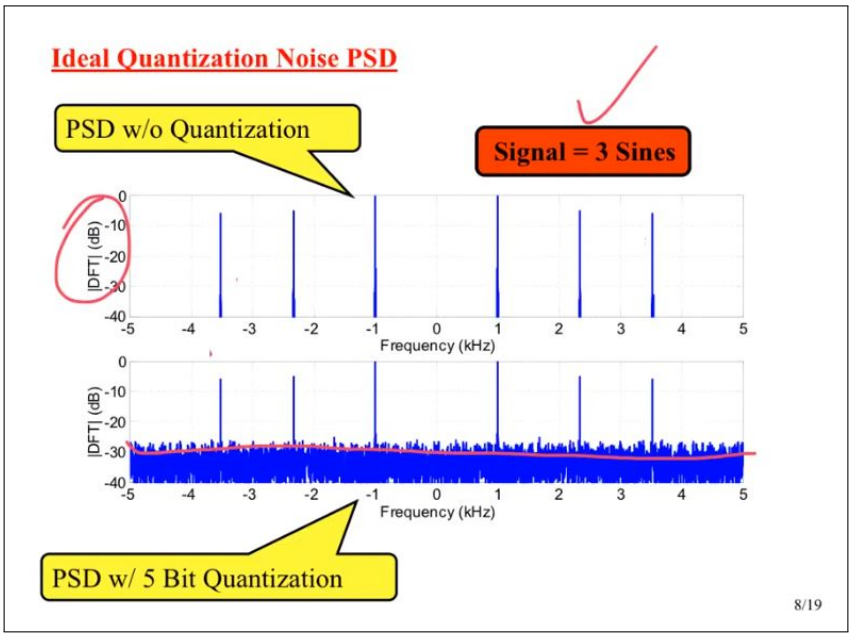


Figure 2.46: Qunatization Error Example

- We can see that the power sepctral density in the quantized part is flat over all the frequency ranges
- If we use fewer bit in the quantization process, the noise floor will get up

SNQR for Full Scale Sine wave:

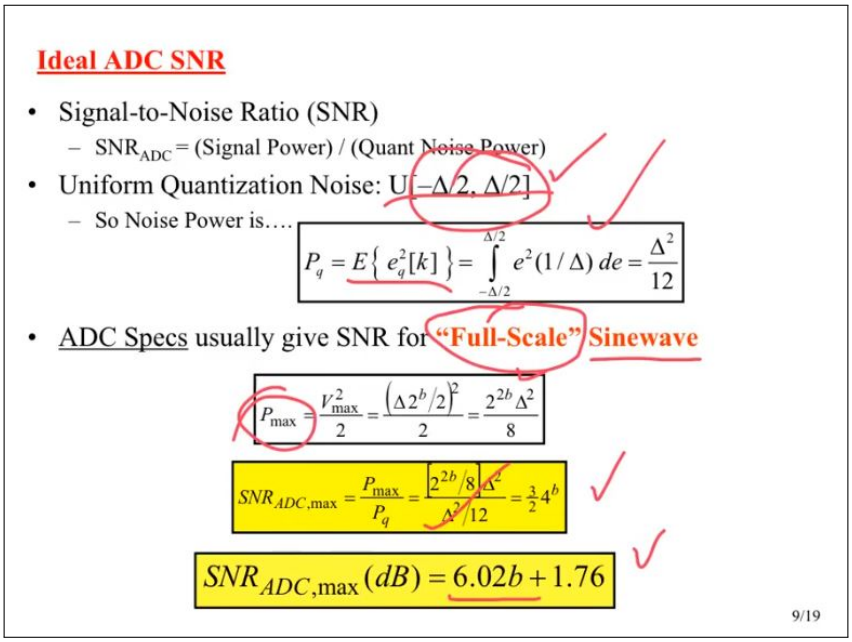


Figure 2.47: SNR for full sin wave

- The SNQR in Figure 2.47 is a little bit optimistic because we are assuming a full range for the sin wave

We can generalize more the SNQR as in Figure 2.48, where use the idea of peak factor to see the exact SNQR.

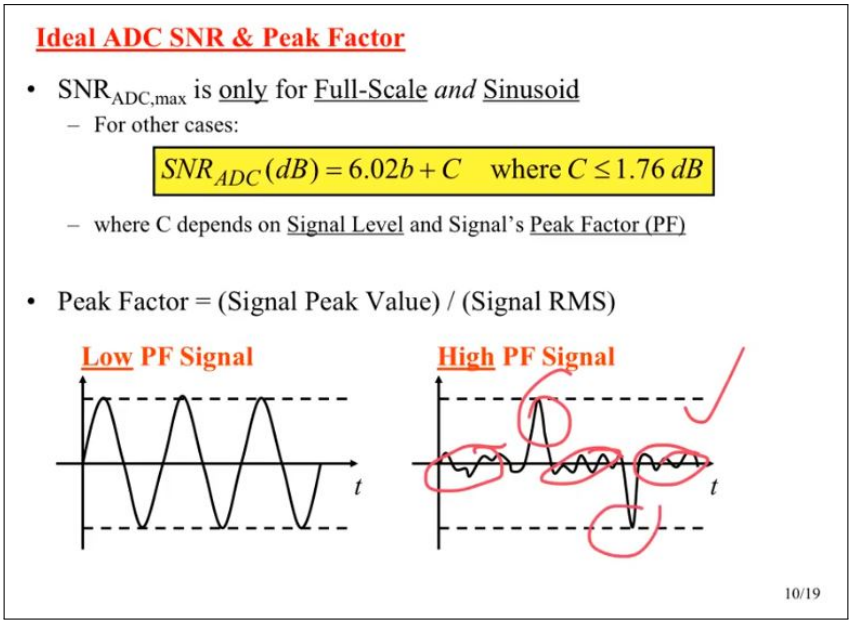


Figure 2.48: SNR and peak factor

Figure 2.49 shows the relation of SNQR and peak factor in dB.

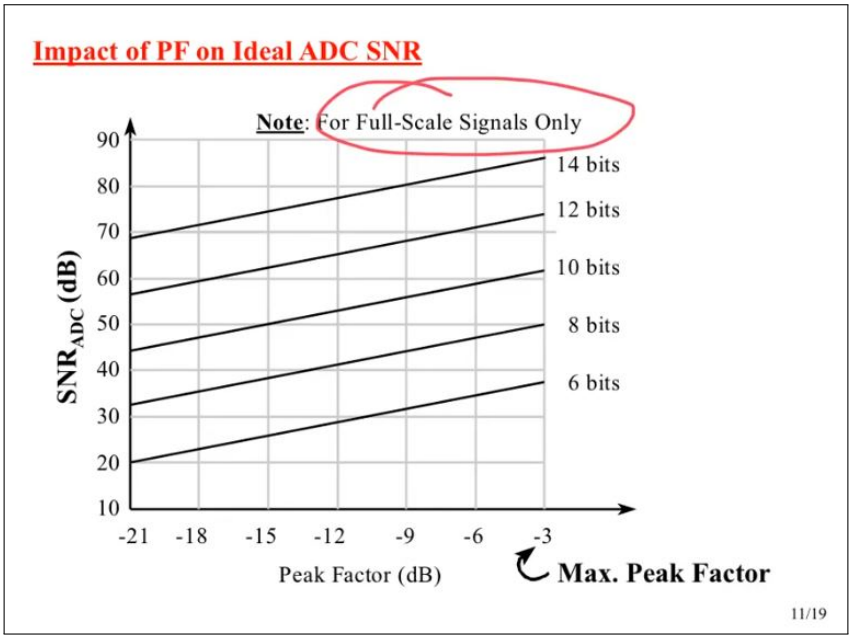


Figure 2.49: SNR and peak factor relation

- The more the signal is peaky for a fixed number of bits, the more SNQR is low

2.11.3 More Source of Errors

Figure 2.50 shows more sources of errors we can get

- Jitter: comes from clock circuitry we use to do the sampling
- These errors also shows decrease in the SNQR

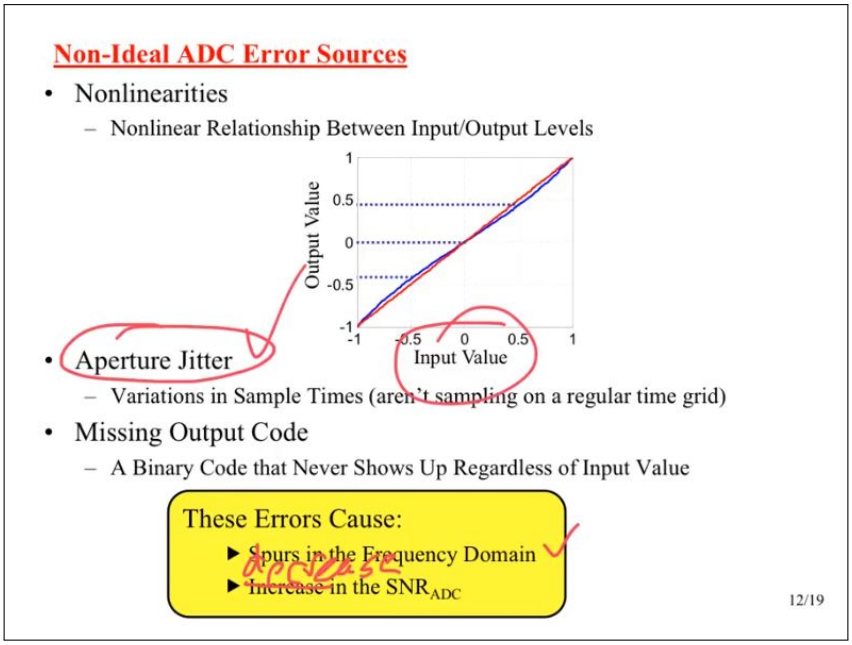


Figure 2.50: SNR and peak factor relation

Figure 2.51 shows how the spruse comes into the amplitude spectrum plots in addition to the noise floor.

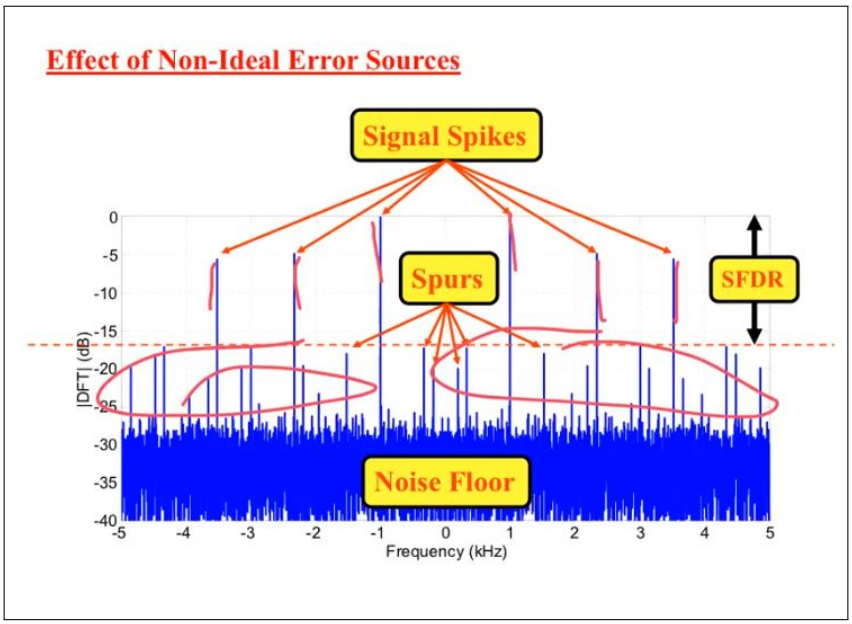


Figure 2.51: Spurse Errors in the Spectrum

- SFDR set the distance between the actual signals and the spurse

2.11.4 Specs for ADC

- ENOB: take into consideration the actual number of bits due SINAD (spurse errors)
  - It is the algebraic solution of SINAD $= 6\,\text{bits} + 1.76$

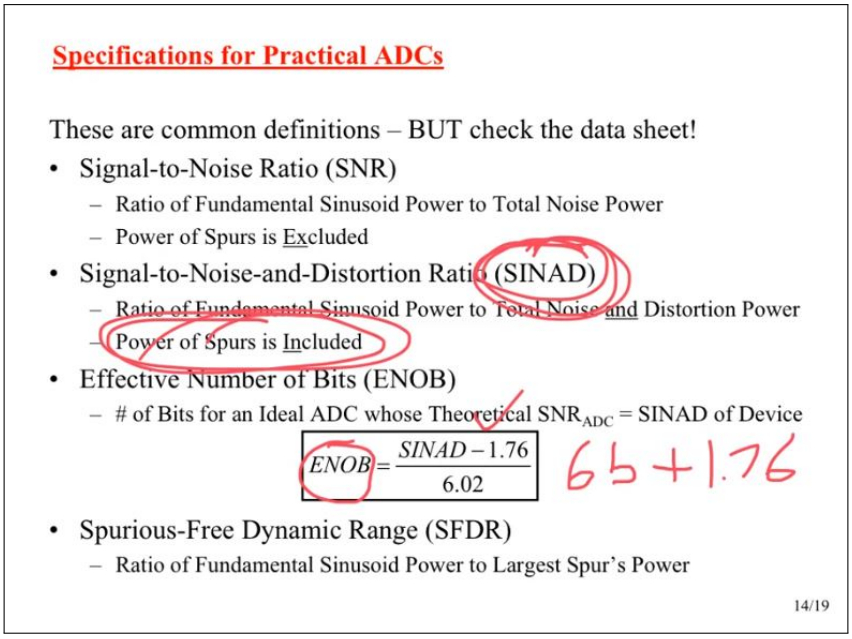


Figure 2.52: Most common specfication on ADC

– Due to hardware imperfections (spurse), in theory we could have 8 bits, but in reality it may comes down in to 6 bits

2.12 Oversampling and $\Sigma - \Delta$ Converter

We have seen that oversampling have several advantages. Now let's relate the oversampling to the SNQR.

- Since we are sampling much higher than the signal bandwidth, we can assume that the signal bandwidth is small compared to $\frac{F_s}{2}$, and the spectrum reside in a small region of the interval $\left[\frac{-F_s}{2}\,;\,\frac{+F_s}{2}\right]$
- So we have less noise in the signal spectrum $\rightarrow$ SNQR improve
- So also ENOB improves
  - However at the cost of a higher F_s
- But this is an inefficient trade-off: because we obtain 1 bit per 6 dB
  - Computation shown in Figure 2.53

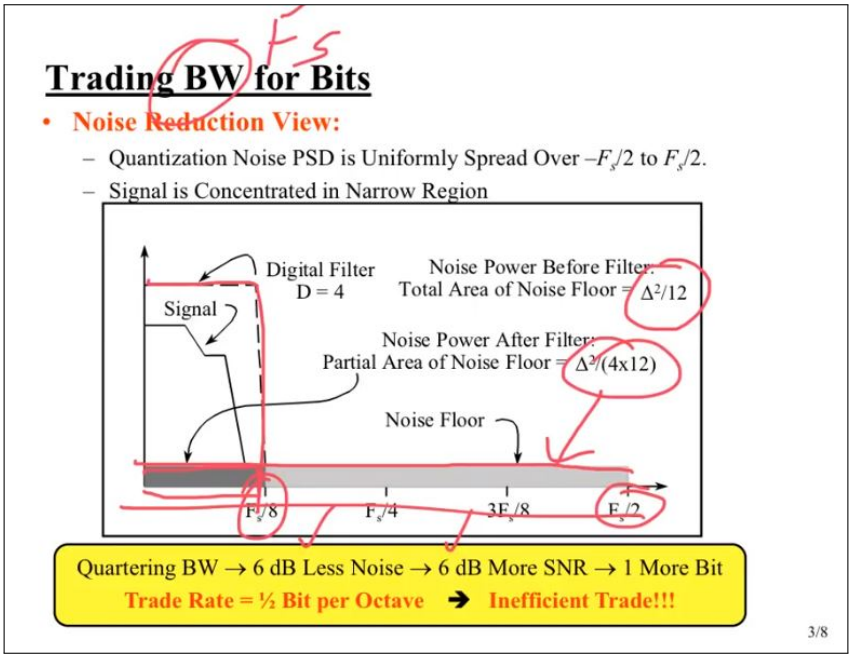


Figure 2.53: Oversampling and SNQR relation

  - The noise power is the area under the curve then we divide powers
- So engineers start to search for a better trade-off
- The idea of $\Sigma - \Delta$ converter: don't touch the signal but push the noise outside the signal spectrum

- Intuition and results are shown in Figure 2.54.

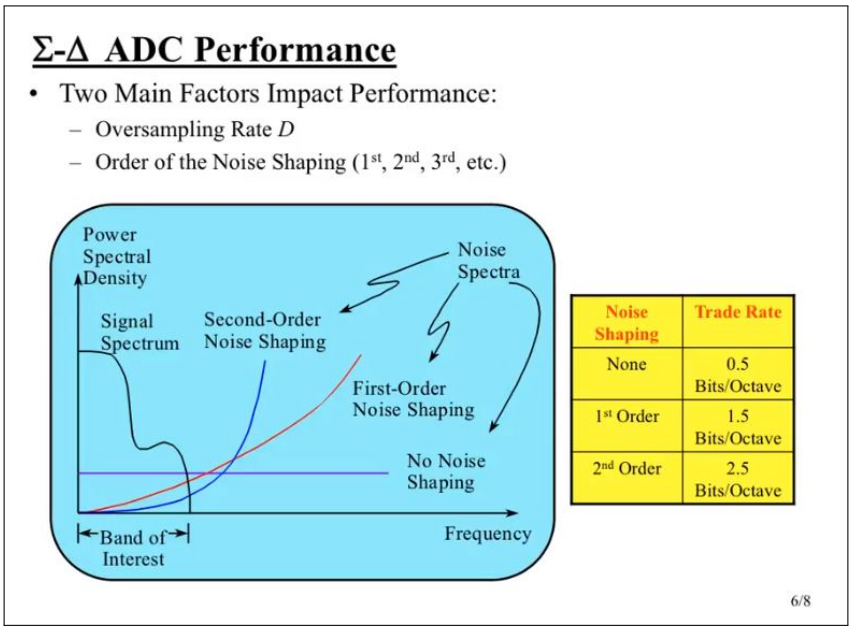


Figure 2.54: Noise shaping effect

– The more we increase the order of noise shaping, the area of the noise decrease and hence SNQR and ENOB increase for the ***band of interest***

Chapter 3

Bandpass Sampling

Now we move to what we call bandpass sampling or higher order sampling. Before getting directly into the process, we talk first about bandpass signals in general, and how to represent them.

- Bandpass signal are encountered in RF application (communication and radar)
- The amplitude spectrum is centered at a center frequency f_c
- When a receiver process a transmitted signal, they don't process it in the bandpass form, they turn into what we call ***equivalent low pass signal***, which is a ***generalization of the idea of phasor***.

There are 3 ways of sampling in bandpass cases:

1. Integer band positioning
2. Non Integer Band positioning
3. Quadrature sampling, which use IQ representation of a signal

3.1 Integer Band Positioning

Integer Ban Positioning

To write it later

3.2 Non-Integer Band Positioning

Non- Intege Band Positi ing

To write it later

3.3 Quadrature Sampling

Before, we review the concept of IQ representation.

3.3.1 Review of Phasor Notation

Phasor Idea:

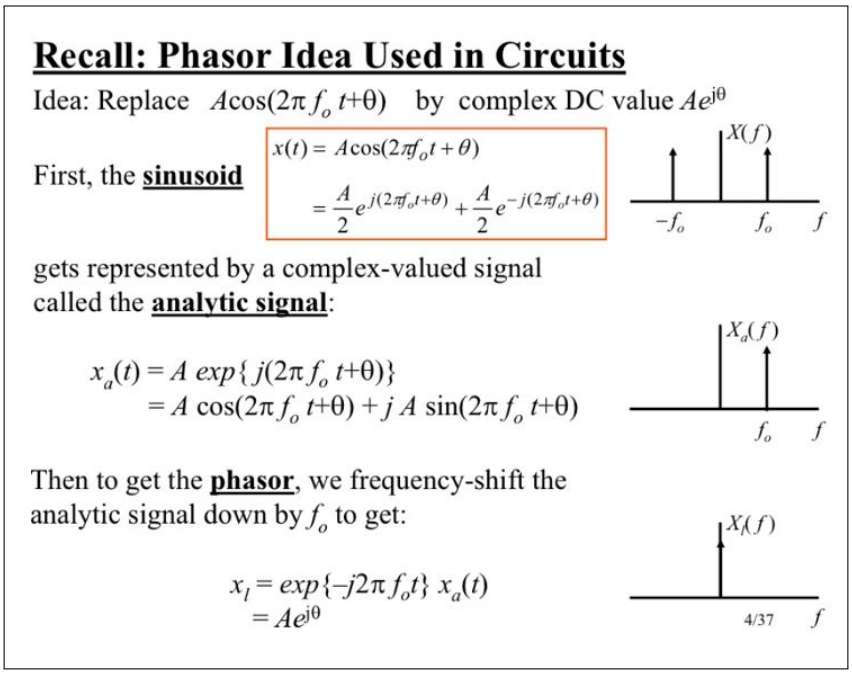


Figure 3.1: Equivalent Low pass signal to a cosine signal

- We have some bandpass signal $x(t) = A\cos(2\pi f_0 + \theta)$
- Using Euler formula, we obtain 2 dirac function at f_0 and $-f_0$
- Turn the signal into analytically form by suppressing the negative frequency component at $-f_0$
- Then we shift the component at $+f_0$ to dc by multiplying by $\exp(-j2\pi f_0 t)$
  - The obtained signal is called $x_{low-pass}$ which is ***complex value signal***

3.3.2 Equivalent Low Pass Signal

We can generalize this idea into a general bandpass signal, as shown in Figure 3.2.

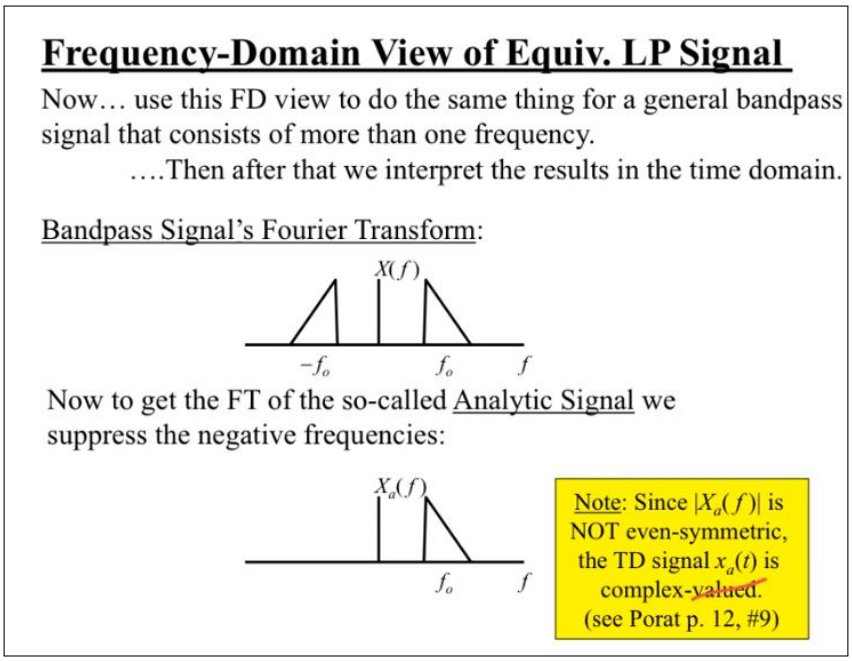


Figure 3.2: Equivalent Low pass signal to a general bandpass signal

- Be aware that since $X_{\text{analyticall}}(f_c)$ has no symmetry around the y-axis, this means that $x_{\text{analyticall}}(t)$ is ***complex signal***

So we start by a real value bandpass signal, and turn it into a complex value low pass signal.

A system point of view for implementing the suppression of the negative frequency part is shown in Figure 3.3, where $H(f_c)$ is known as Hilbert transform.

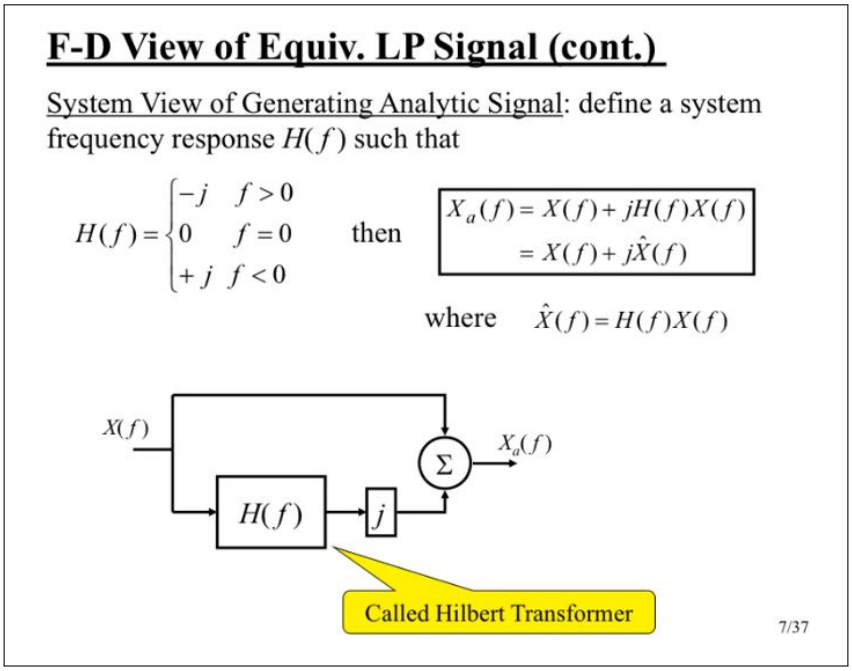


Figure 3.3: System point of view to implement the suppression of negative frequency spectrum

- We apply Hilbert transform to $X_{\text{band-pass}}(f_c)$, then add the output with the original signal $X_{\text{band-pass}}(f_c)$

After we obtain $X_{\text{analyticall}}(f_c)$ which is a complex signal, we shift down the frequency spectrum and hence we obtain $X_{\text{low-pass}}(f_c)$ as shown in Figure 3.4.

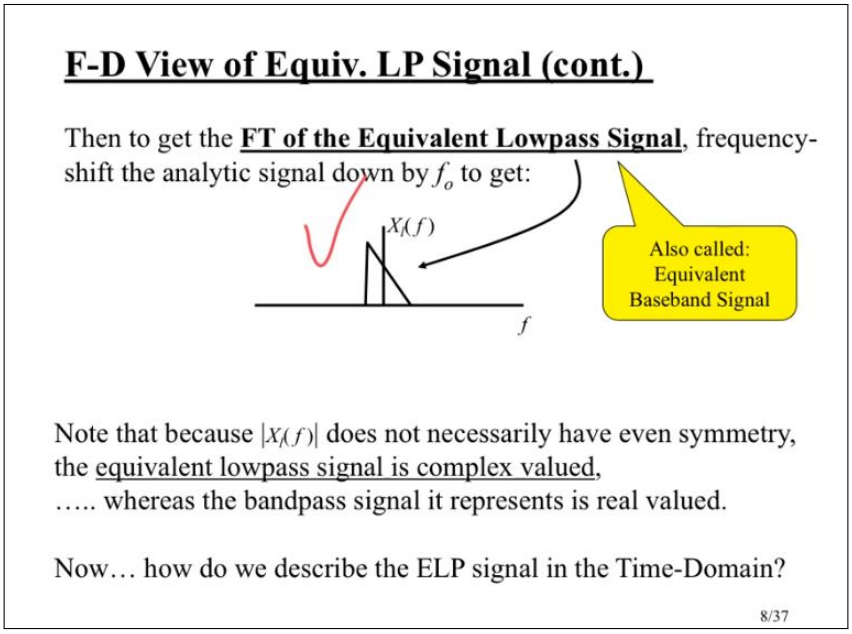


Figure 3.4: Shifting down the frequency spectrum of $X_{\text{analyticall}}(f_c)$

3.3.3 Recap

So as a recap of the process in terms of formulas:

1. We begin from a general band-pass signal $x_{\text{band-pass}}(t)$
2. Obtain its analytic form:

$$\begin{aligned} X_{\text{analytic}}(f_c) &= X_{\text{band-pass}}(f_c) + j\, X_{\text{Hilber}}(f_c) \\ x_{\text{analytic}}(t) &= x_{\text{band-pass}}(t) + j\, x_{\text{Hilber}}(t) \end{aligned} \tag{3.1}$$

3. Down frequency shifting to obtain $x_{\text{Eq-Baseband}}(t) \in \mathbb{C}$:

$$\begin{aligned} x_{\text{Eq-Baseband}}(t) &= \exp\left(-j2\pi f_{\text{center}}\, t\right) \times x_{\text{analytic}}(t) \\ &= \exp\left(-j2\pi f_{\text{center}}\, t\right) \times \left[x_{\text{band-pass}}(t) + j\, x_{\text{Hilber}}(t)\right] \end{aligned} \tag{3.2}$$

4. Since $x_{\text{Eq-Baseband}}(t)$ is a ***complex signal***, we can ***express it in a rectangular form*** known also as in-phase and quadrature form

$$\begin{aligned} x_{\text{Eq-Baseband}}(t) &= \exp\left(-j2\pi f_{\text{center}}\, t\right) \times x_{\text{analytic}}(t) \\ &= \exp\left(-j2\pi f_{\text{center}}\, t\right) \times \left[x_{\text{band-pass}}(t) + j\, x_{\text{Hilber}}(t)\right] \\ &= x_{\text{in-phase}}(t) + j\, x_{\text{quadrature}}(t) \end{aligned} \tag{3.3}$$

5. The purpose know is to find the relation between $x_{\text{band-pass}}(t)$ and in-phase and quadrature of $x_{\text{Eq-Baseband}}(t)$.

$$\begin{aligned} x_{\text{band-pass}}(t) &= x_{\text{in-phase}}(t)\cos\left(2\pi\, f_{\text{center}}\, t\right) + x_{\text{quadrature}}(t)\sin\left(2\pi\, f_{\text{center}}\, t\right) \end{aligned}$$

Similarly but less important

$$\begin{aligned} x_{\text{Hilber}}(t) &= x_{\text{in-phase}}(t)\sin\left(2\pi\, f_{\text{center}}\, t\right) + x_{\text{quadrature}}(t)\cos\left(2\pi\, f_{\text{center}}\, t\right) \end{aligned} \tag{3.4}$$

6. Final form using polar form between bandpass signal and equivalent baseband signal

$$x_{\text{band-pass}}(t) = \text{Amp}(t) \times \cos\left(2\pi\, f_{\text{center}}\, t + \text{phase}(t)\right) \tag{3.5}$$

where
$$\begin{cases} x_{\text{Eq-Baseband}}(t) = \text{Amp}(t) \times \exp\left[j \times \text{phase}(t)\right] \\ \text{Amp}(t) = \sqrt{x^2_{\text{in-phase}}(t) + x^2_{\text{quadrature}}(t)} \\ \text{phase}(t) = \tan^{-1}\left[\dfrac{x_{\text{in-phase}}(t)}{x_{\text{quadrature}}(t)}\right] \end{cases}$$

- The equivalent baseband signal $x_{\text{Eq-Baseband}}(t)$ has the same envelope and phase of the original bandpass signal $x_{\text{band-pass}}(t)$
- The conditions for equation (3.5) is that the amplitude and the phase vary slowly compared to carrier signal.

  If not, equation (3.5) won't be bandpass signal in necessarily.

3.4 Generation of I and Q components

As a recap, we have:

$$x_{\text{band-pass}}(t) = \text{Amp}(t) \times \cos\left(2\pi\, f_{\text{center}}\, t + \text{phase}(t)\right) \tag{3.6}$$

In communication and radar signal processing, we usually process the I and Q components of the bandpass signal where:

$$\begin{aligned} x_{\text{in-phase}}(t) &= \text{Amp}(t) \times \cos\left[\text{phase}(t)\right] \\ x_{\text{quadrature}}(t) &= \text{Amp}(t) \times \sin\left[\text{phase}(t)\right] \end{aligned} \tag{3.7}$$

To implement these signals of equation (3.7), we begin first from manipulating equation using trigonometric identities as shown in Figure 3.5, then building these signals using analog devices as shown in Figure 3.6.

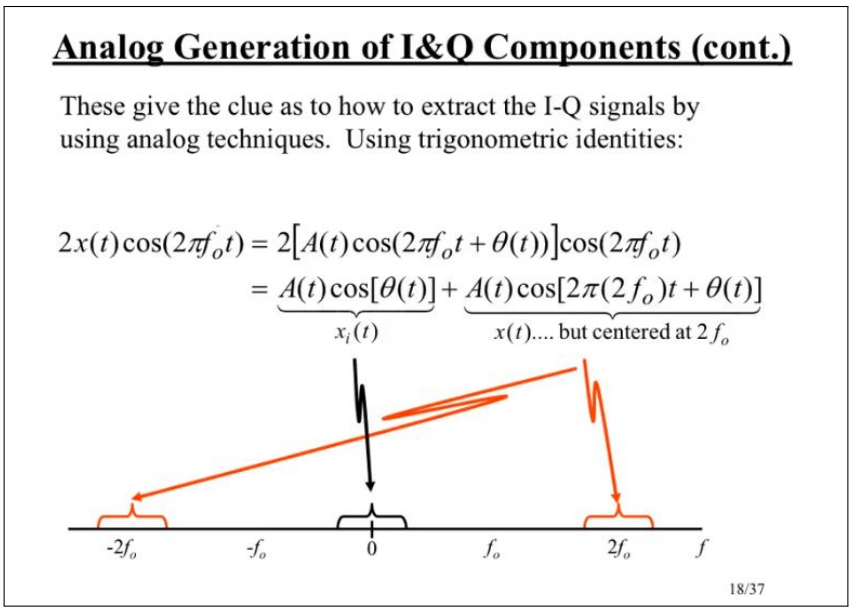


Figure 3.5: Inphase term generation

- We use the trigonometric identities of the argument of a cos
- Notice that we obtained the in-phase term, but with a high frequency component centered at $2\,f_{\text{ceneter}}$, so we need to use a low pass filter

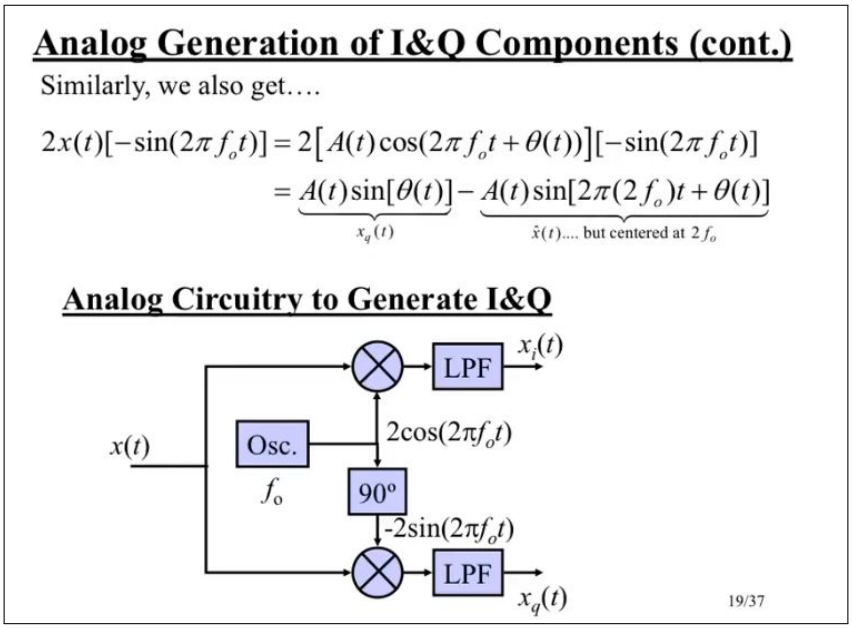


Figure 3.6: IQ generation and implementation using analog devices

- The multiplication is a mixer and the phase shift is implemented using a phase shifter
- The cos signal is generated using an oscillator

Also, the in-phase and quadarature components can be generated digitally, as shown in, using some DSP processor.

- Instead of a mixer, we can do multiplication numerically
- Instead of an oscillator, we can do generate the cos numerically also

3.5 Sampling Dilemma using I and Q Components

Suppose we have generate the I and Q components and we want to process them in a system for some purpose. Usually, we put some ADC after the I and Q to do the processing in discrete time.

According to Shannon theorem, we need to sample twice as fast the highest frequency component, which is B according to , if we take the $X_{\text{Eq-Baseband}}(f_c)$ amplitude spectrum.

However, the original band pass signal $X_{\text{Bandpas}}(f_c)$ has a bandwidth of B, so according to Shannon, we need to sample at $2\,B$.

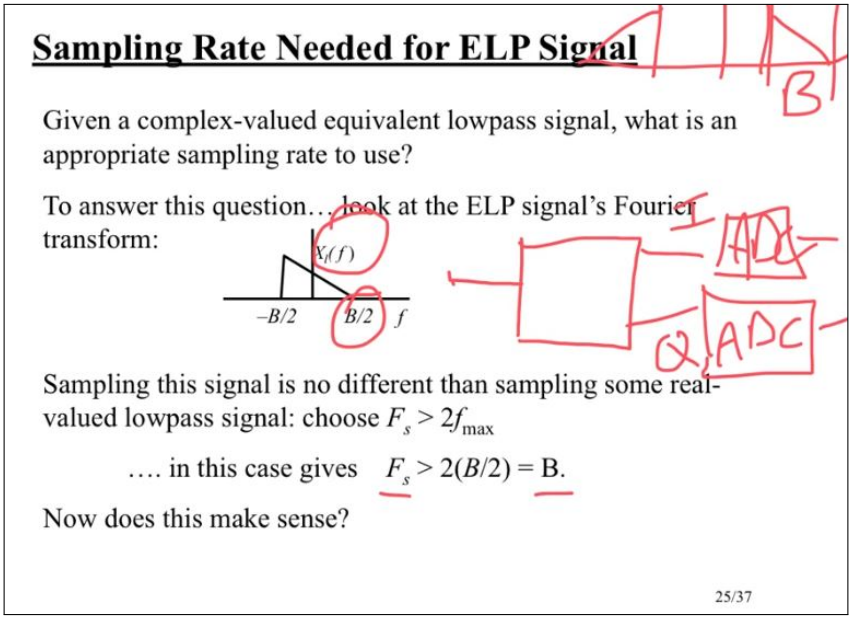


Figure 3.7: IQ sampling

But we need not to forget that $X_{\text{Eq-Baseband}}(f_c)$ is composed from 2 signals, and each signal is sampled at a rate of B, which make sense.

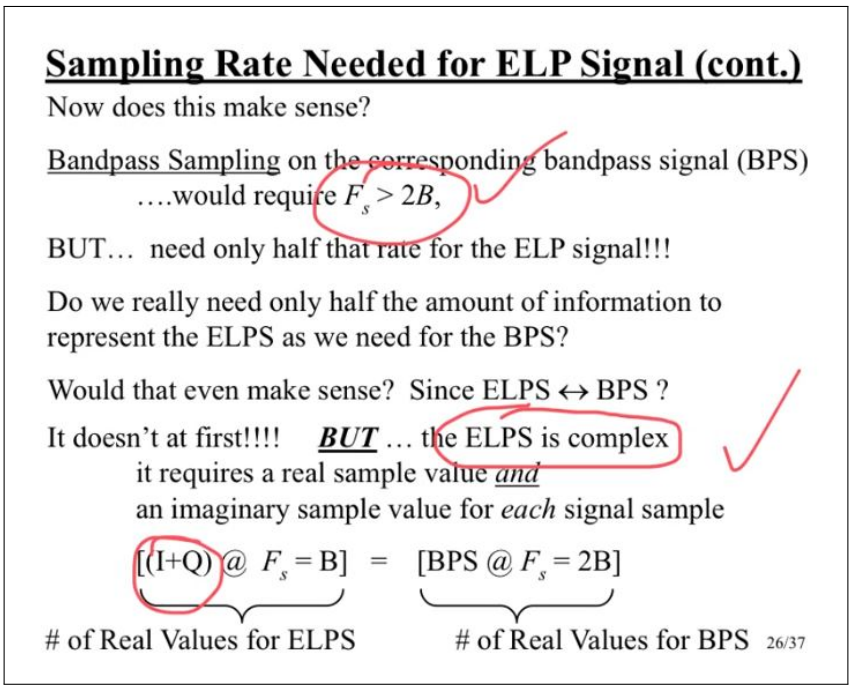


Figure 3.8: IQ sampling: each component is sampled at a rate of B

3.5.1 Integer band positioning

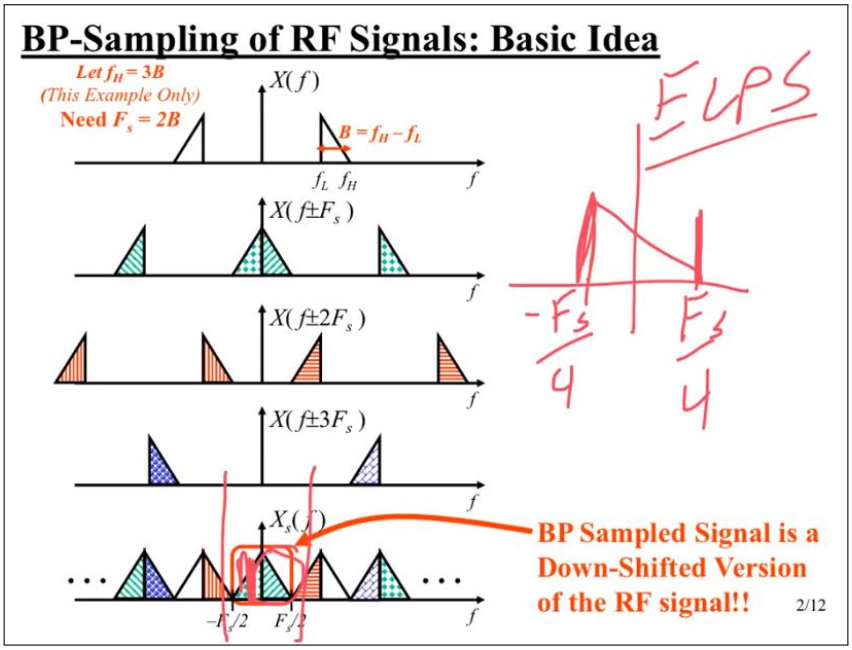


Figure 3.9: Bandpass sampling example 1

- We have a bandpass signal of bandwidth B and $f_{\text{high}} = 3B$
- The theory tells us that we need to sample at twice the maximum frequency in the signal, so $F_s = 6B$
- However, in the example of Figure 3.9, we sample at $F_s = 2B$
- When we sample $F_s = 2B$, the theory tells to add the replicate of the spectrum at multiple of F_s
  - Each portion of the spectrum (positive and negative) will be shifted at multiple of F_s
- At the end we obtain $X_{\text{sampled}}(f_c)$
  - To reconstruct the bandpass signal, we low pass the signal then shifted to obtain the equivalent base band signal

General Case:

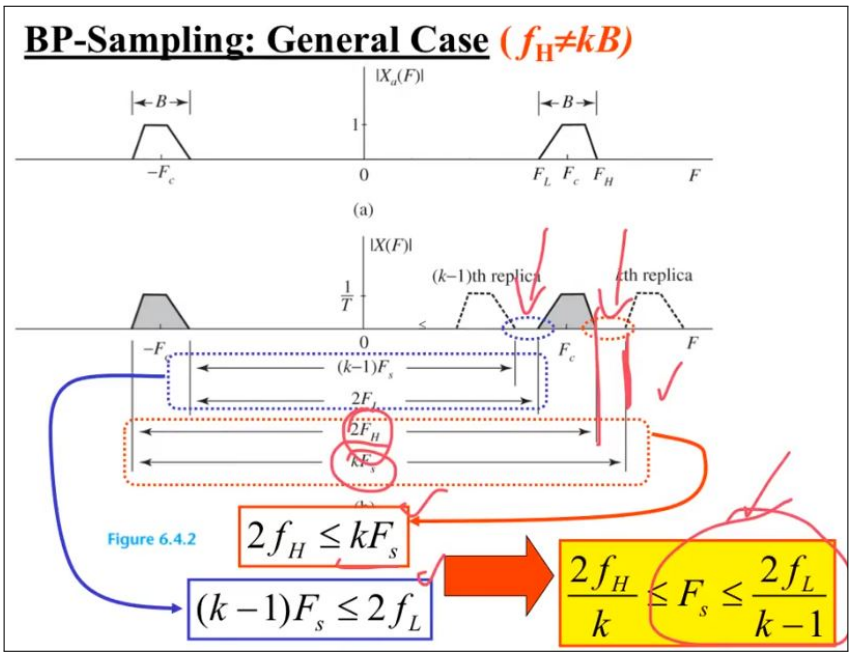


Figure 3.10: Bandpass sampling non integer case

- We want that the images don't overlap with the original spectrum of the bandpass signal
- The 2 inequality contstraints comes from f_{low} and f_{high}
- The larger k we have, the lower F_s we can choose.
  - However, we don't have an idea about k yet

- Now we formulate a bound on k shown in Figure 3.11.

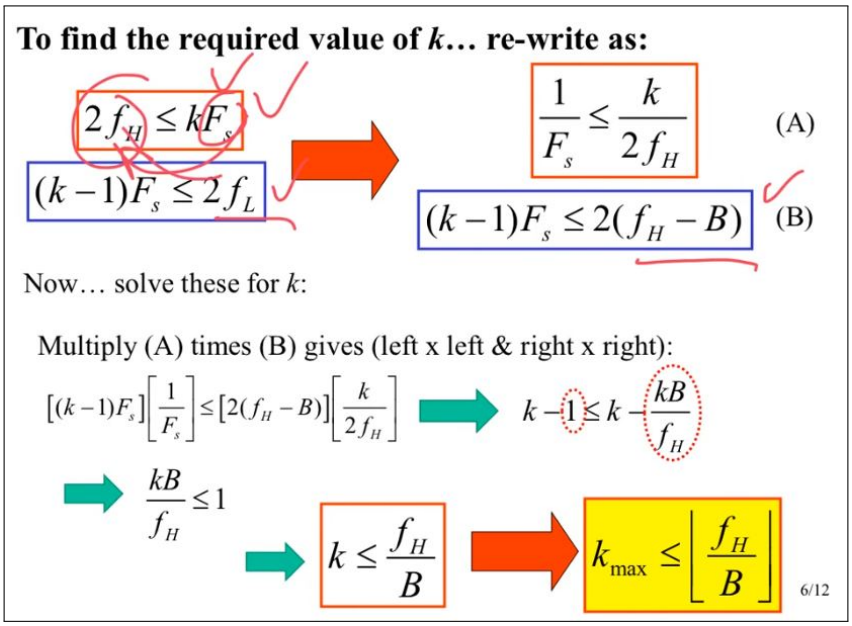


Figure 3.11: Bandpass sampling non integer case: bounding k

- As summary:

$$\begin{aligned} \frac{2\,f_{\text{high}}}{k} \;&\le\; F_s \;\le\; \frac{2\,(\,f_{\text{high}} \;-\; \text{Bandwidth}\,)}{k \;-\; 1} \\ k \;&\le\; \frac{2\,f_{\text{high}}}{\text{Bandwidth}} \end{aligned} \tag{3.8}$$

Plotting equations (3.8) give us plot shown in Figure 3.12.

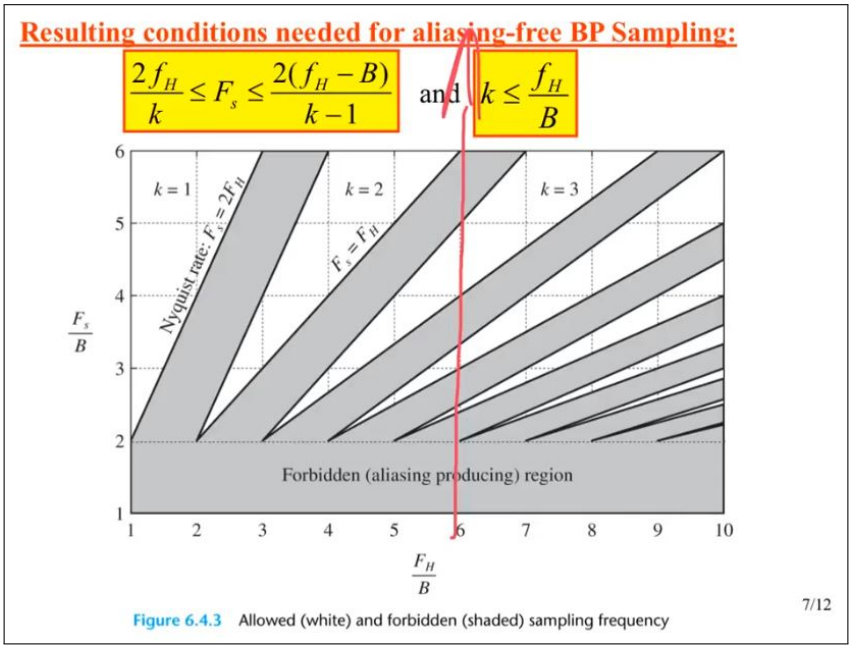


Figure 3.12: Bandpass sampling plot: F_s function of f_{high}

- The plots are normalized over the bandwidth B, which makes plotting the equation more easy
- $k = 1$: we are operating on the standard Nyquiste rate
- if we have $\frac{f_{\text{high}}}{\text{bandwidth}} = 6$, we can operate at multiple k: 1,2, 3 and 4 and even 5 (the higher k the lower is F_s)

– However, we have to pay attention at $k = 5$

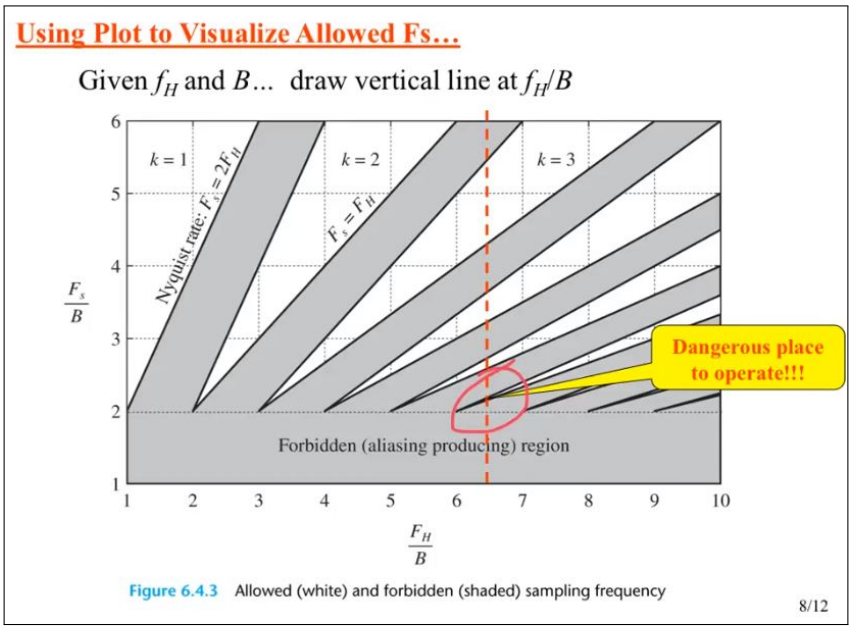


Figure 3.13: Bandpass sampling plot: dangerous area we need to pay attention from

– From this dangerous place, comes the idea of guard band shown in Figure 3.14

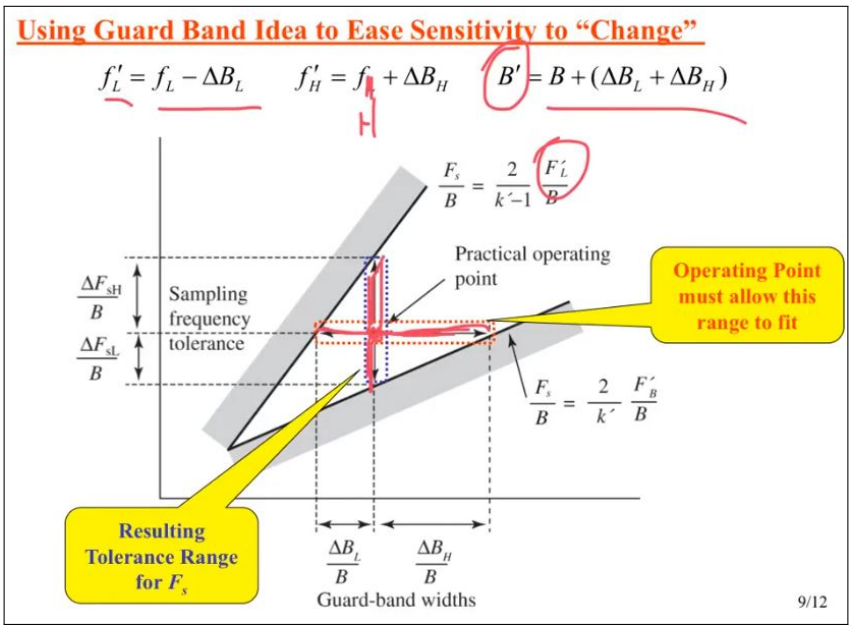


Figure 3.14: Bandpass sampling plot:putting some guard bands

Part I

DSP Time Domain Tools

Chapter 4

Convolution

4.1 Introduction

Conv Intro

Conv Intro: To write it down later.

4.2 Definition

The definition of convolution between 2 signals x and h is:

$$\begin{aligned} y(t) &= \int_{-\infty}^{+\infty} x(\tau)\, h\,(\,t\,-\tau\,)\, d\tau \\ y[k] &= x[k] * h[k] \\ &= \sum_{m=-\infty}^{+\infty} x[m]\, h[k\,-\,m] \end{aligned} \tag{4.1}$$

Notes for Equation 4.1:

- $h\,(\,t\,-\tau\,)$: there is an inversion with respect to time axis (the $-$), then a delay by t
  - The integration is with respect to τ, so τ is the time axis
  - The inversion with respect to time, that is $h(-\tau)$, we flip $h(\tau) \rightarrow$ we have a symmetry (mirror) with respect to y axis
- So the shifting will be with respect to t or m in discrete time case: it can be < 0 or > 0
  - To evaluate the $\int$ or $\sum$ in Equation 4.1, we need to take all shifting cases

4.3 Properties

Width Property:

$h_1[k]$ has a length of m_{h_1} and $h_2[k]$ has a length of m_{h_2}, the resultant signal will have a length of $m_{h_1} + m_{h_2} - 1$.

Start and Stop time:

- Start time of $y[k]$ = start of x + start of h
  - Where the start of x and start of h is from the non zero values
- Stop time of $y[k]$ = stop of x + stop of h
  - Where the stop t of x and stop of h is from the non zero values

4.4 Example in Continuous Time

4.4.1 Example 1

Given is shown in Figure 4.1

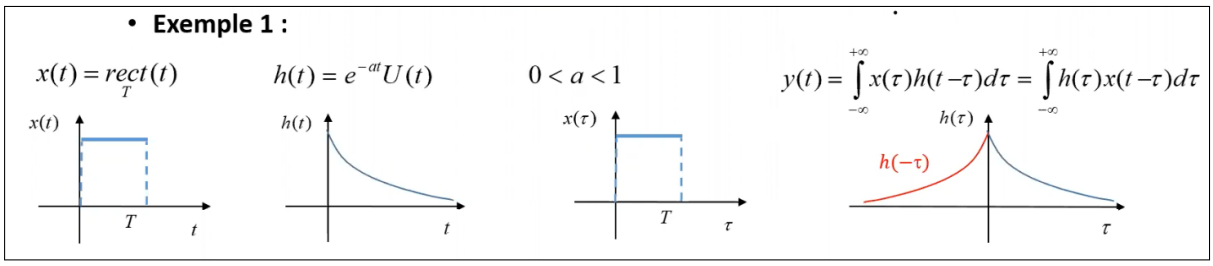


Figure 4.1: Example 1 given

Before solving, some comments regarding the example:

- The signal $x(t)$ is time shifted with respect to T, it's not like the standard rect function which starts from $\frac{-T}{2} \rightarrow \frac{T}{2}$
- Remember that now the new time axis is τ and not t.
  - t will act as some constant which can have different cases

Now to solve this example, we need to study different time shift cases for $h(\tau)$, but after inversion, that is shifting for $h(-\tau)$. The shifting cases are illustrated in Figure 4.2.

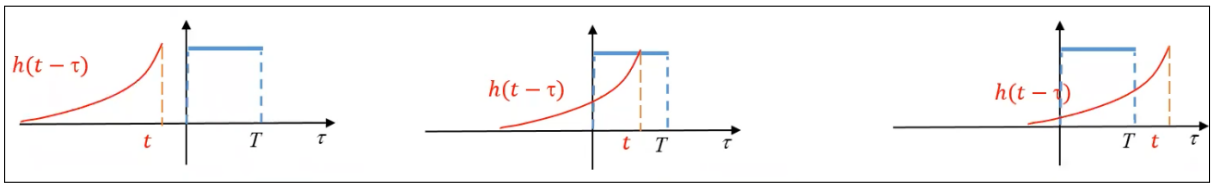


Figure 4.2: Example 1 time shifting cases

- Tor $t < 0$: there is no common area between x and h, so $y(t) = 0$ for $t \leq 0$
- For $t > 0$: the signal x finishes at a duration T, but h can continue beyond T, so in case of > 0, we distinguish 2 cases:
  1. $0 < t < T$: the boundaries of common surface is from $\tau = 0 \rightarrow \tau = t$. Using Equation 4.1, we have:

$$y(t) = \int_{-\infty}^{+\infty} x(\tau)\, h\,(\,t\, -\tau\,)\, d\tau$$

$$= \int_{0}^{t} 1 \times e^{-a\,(t-\tau)}\, d\tau$$

Since the integration is with respect to τ, we can all the non τ variables (t and a) act as constant and we can get them out of the integral

$$
\begin{aligned}
y(t) &= e^{-a\,t} \int_0^t e^{a\,\tau} d\tau \\
&= e^{-a\,t} \left[\frac{e^{a\tau}}{a} \right]_0^t \\
&= \frac{e^{-a\,t}}{a} \left(e^{a\,t} - 1 \right) \\
&= \frac{1}{a} \times \left[e^{-a\,t} \times \left(e^{at} - 1 \right) \right] \\
&= \frac{1}{a} \left(1 - e^{-a\,t} \right)
\end{aligned}
$$

2. $t > T$: in this case the common area from $\tau = 0 \rightarrow \tau = T$
   Solving we get:

$$
\begin{aligned}
y(t) &= \int_{-\infty}^{+\infty} x(\tau)\, h\,(\,t - \tau\,)\, d\tau \\
&= \int_0^T 1 \times e^{-a\,(t-\tau)}\, d\tau \\
&= e^{-a\,t} \left[\frac{e^{a\tau}}{a} \right]_0^T \\
&= \frac{e^{-a\,t}}{a} \left(e^{a\,T} - 1 \right)
\end{aligned}
$$

   Rearranging the constant terms we get:
   $y(t) = \left(\dfrac{e^{a\,T} - 1}{a} \right) \times e^{-a\,t}$ for $t > T$

As a recap, the solution is:

$$
y(t) = \begin{cases} 0 \quad \text{for } t < 0 \\ \dfrac{1}{a} \left(1 - e^{-a\,t} \right) \quad \text{for } 0 < t < T \\ \left(\dfrac{e^{a\,T} - 1}{a} \right) \times e^{-a\,t} \text{ for } t > T \end{cases} \tag{4.2}
$$

4.4.2 Example 2

Given: compute the convolution between $x(t)$ and $h(t)$ given in Equation 4.3

$$\begin{aligned} x(t) &= \text{some signal} \quad t \in]-\infty, -\infty[\\ h(t) &= \frac{1}{T}\,\text{rect(t)} \quad t \in \left[\frac{-T}{2}, \frac{T}{2}\right] \end{aligned} \tag{4.3}$$

Solution:

The signal $x(t)$ is assumed to be some general signal, and the only information we have is the that $x(t)$ exist for all value of t.
As for $h(t)$, it is centered at $t = 0$, and has a duration T from $\frac{-T}{2} \rightarrow \frac{T}{2}$.

The 1st step is always as seen in 4.4.1 is the time inversion of $h(\tau)$ with respect to the y axis. Since we have a rect(t) signal, the inversion will result in the same signal since the rect signal is already symmetric with respect to the y axis.

After we have $h(-\tau)$, we start shifting it with different values of t. In other words, $h(-\tau)$ is centered at 0, now it will be centered at t, and the t can be < 0 or > 0, as shown in Figure 4.3.

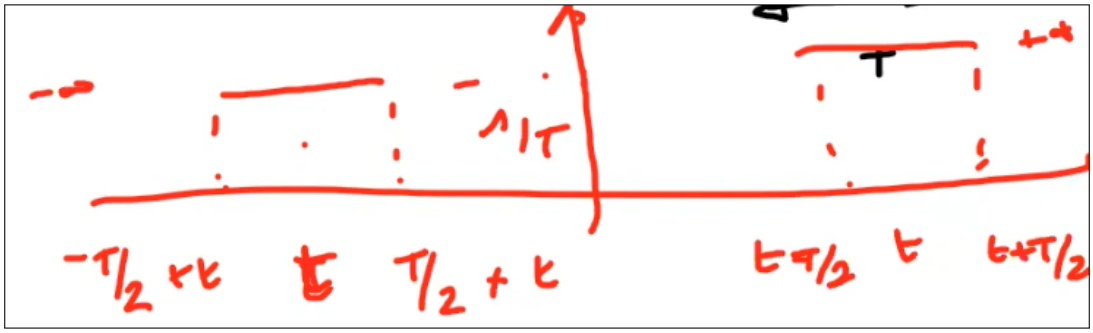


Figure 4.3: Shifting rect signal with different t values

Applying Equation 4.1 for different values of t give the following cases:

$\underline{t < 0}$:

$$\begin{aligned} y(t) &= \int_{-\infty}^{+\infty} x(\tau)\, h\,(\,t\, -\tau\,)\, d\tau \\ &= \int_{-\frac{T}{2}+t}^{\frac{T}{2}+t} x(\tau)\, \times \frac{1}{T}\, \times\, 1\, d\tau \\ &= \frac{1}{T} \int_{-\frac{T}{2}+t}^{\frac{T}{2}+t} x(\tau)\, d\tau \end{aligned}$$

$\underline{t > 0}$:

For $t > 0$, we have the same computation since the integral bounds will be the same.

The expression $\frac{1}{T} \int_{-\frac{T}{2}+t}^{\frac{T}{2}+t} x(\tau)\, d\tau$ is similar to taking the average of a signal $x(t)$. Recall that the average of a signal $x(t) = \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} x(t)\, dt$.

So we can say that $\frac{1}{T} \int_{-\frac{T}{2}+t}^{\frac{T}{2}+t} x(\tau)\, d\tau$ is taking the average but with different values of t.

That's why $\frac{1}{T}\,\text{rect(t)}$ is called an average filter, since the result of the filtering is taking the average of signal $x(t)$ for all values of t.

Note: see simulation in `MATLAB`.

4.5 Examples in Discrete Time

4.5.1 Finite Short Signals

To do later.

4.5.2 Long or Infinite Signal Approach

In the example of subsection 4.5.1, the signals were having few values for some indexes, so the definition of equation (4.1) were applicable and we can compute $c[k]$ for each k index. But what if we have some long signals, or some infinite duration signals ? we need a more applicable approach.

The method is called ***sliding tapes approach***, where it consist to shift $h[k]$ along the index axis, and compute the $c[k]$ for each time we do the shifting of $h[k]$. The sliding tapes approach can be summarized by doing the following steps:

1. Plot $f[k]$ and $h[k - m]$.
2. Shift $h[k - m]$ to far to the left (large k).
3. Multiply and Sum Signals (do the dot product).
4. Shift k until new intermediate signal results.
5. Repeat 3 and 4 until done.

Example 1:

Let's compute the convolution between:

- $x[k] = \alpha^k \, u[k]$, where $0 \leq \alpha \leq 1$, so its a damping exponential.
- $h[k] = u[k]$.

First, let's recall the definition: $x[k] * h[k] = \sum_{m=-\infty}^{+\infty} x[m] \, h[k - m]$.

We need to able to sketch $x[m]$ and $h[k-m]$. The sketch is shown in Figure 4.4.

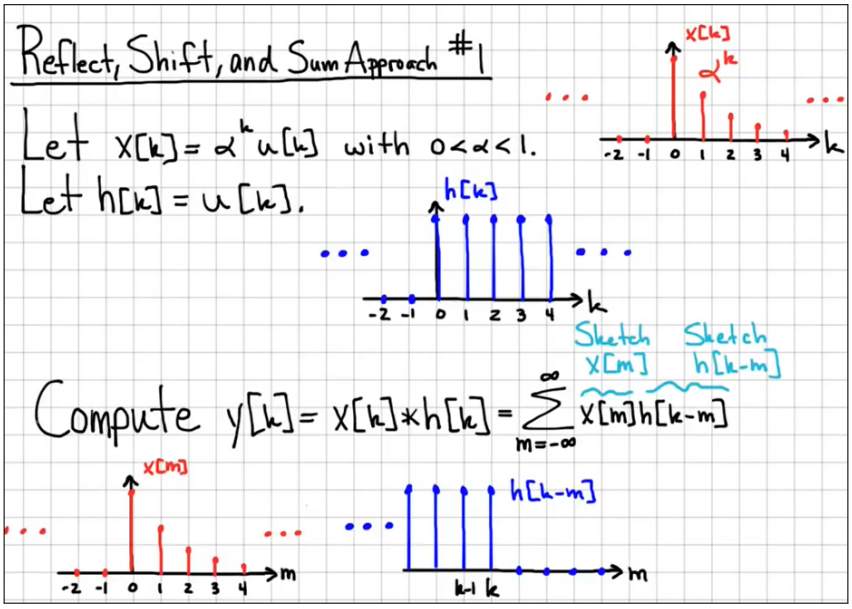


Figure 4.4: Sketching $x[m]$ and $h[k-m]$

- $x[m]$ is quite easy because is the same as $x[k]$
- $h[k-m]$ is the tricky one: it is a time reversed then shifted signal
  - We first do time reversing, so the $h[-m]$ part (reflection with respect to the y axis.)
  - Then we shift to the left

The difference between $h[k-m]$ and $h[k]$ is that:

- $h[k]$ will turn on at $k=0$ and still on until some k value
- $h[k-m]$ will be turned off at k, so it is the opposite of $h[k]$.

It is very important when plotting $h[k-m]$ to let k be a variable, notice that in Figure 4.4 we didn't specify the origin of $h[k-m]$, we let it, because for each value of k, we will have a different $h[k-m]$.

We will study the cases for 2 cases: $k < 0$, where we take as example $k = -2$, and $k \geq 0$, where $k = 2$. These cases are shown in Figure 4.5.

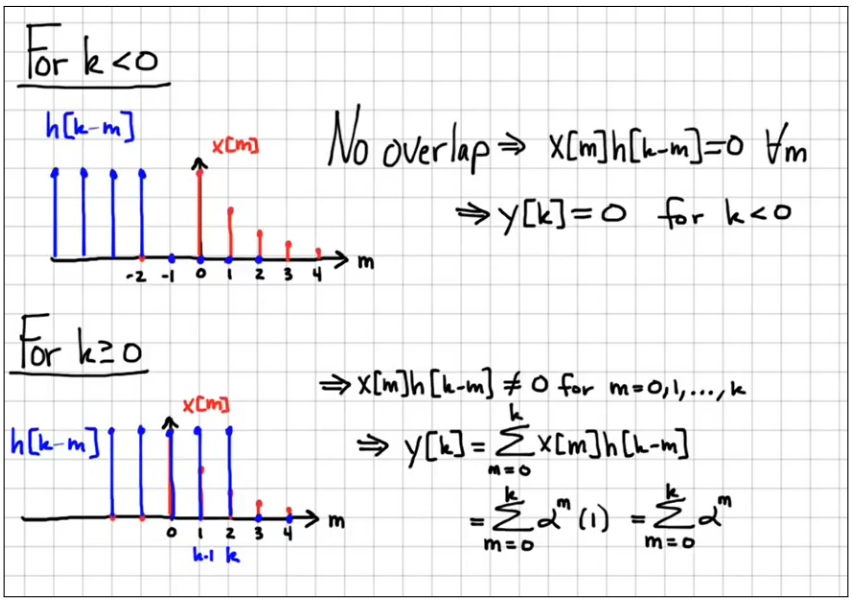


Figure 4.5: $h[k - m]$ for $k < 0$ and $k \geq 0$

- For any $k < 0$, there will be no overlapping between $h[k - m]$ and $x[m]$, so the result of the convolution will be 0 for $k < 0$.
- For $k \geq 0$, there is overlapping between the 2 signals, and this overlapping stop till k, so the upper limit of the summation will be till $m = k$ instead of $m = +\infty$.
  - And sine $x[m] = 0$ for $m < 0$, so the lower limit of the summation will start from $m = 0$ instead of $m = -\infty$.

  For $k \geq 0$, $h[k - m] = 1$.

The series for $k \geq 0$ can be solved using the following formula:

$$\sum_{n=0}^{N-1} \alpha^n = \begin{cases} N & \text{for } \alpha = 1 \\ \dfrac{1 - \alpha^N}{1 - \alpha} & \text{any complex } \alpha \neq 1 \end{cases} \tag{4.4}$$

The computation are shown in Figure 4.6, along with final answer of the convolution $y[k]$.

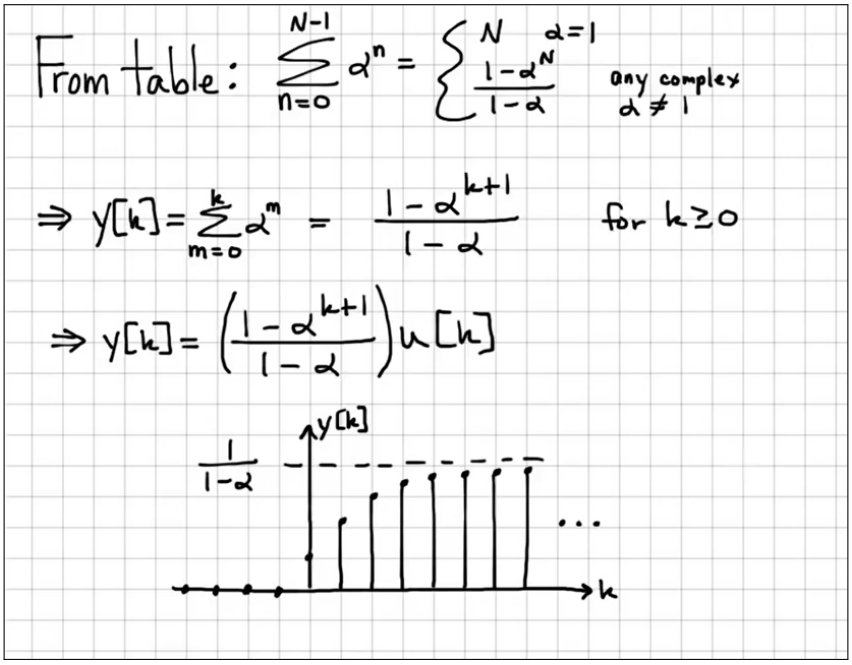


Figure 4.6: $y[k]$

Notice that when computing $y[k]$, we add $+1$ to k, because the formula of equation (4.4) add plus 1 to the upper limit ($N - 1$). Also the signal $y[k]$ converge to the limit $\dfrac{1}{1-\alpha}$ as $k \to +\infty$.

Example 2:

Example 2 is shown in Figure 4.7.

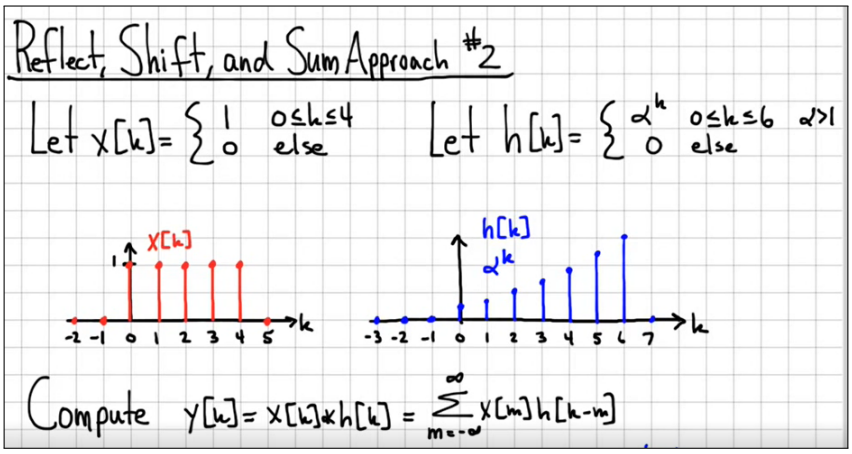


Figure 4.7: Question for Example 2

First step is to plot $x[m]$ and $h[k-m]$. The plot is shown in Figure 4.8.



Figure 4.8: Sketching $x[m]$ and $h[k-m]$

Again, $h[k-m]$ is time reversed first and then shifted to the left. We let k to be a variable, and we will study different cases for k. Notice that $h[k-m]$ now start at index $m = k-6$ and stop for index $m = k$.

The approach to compute the solution is the same as in example 1, but here we have more cases to be considered for k. All the cases are shown below from Figure 4.9 → Figure 4.11.

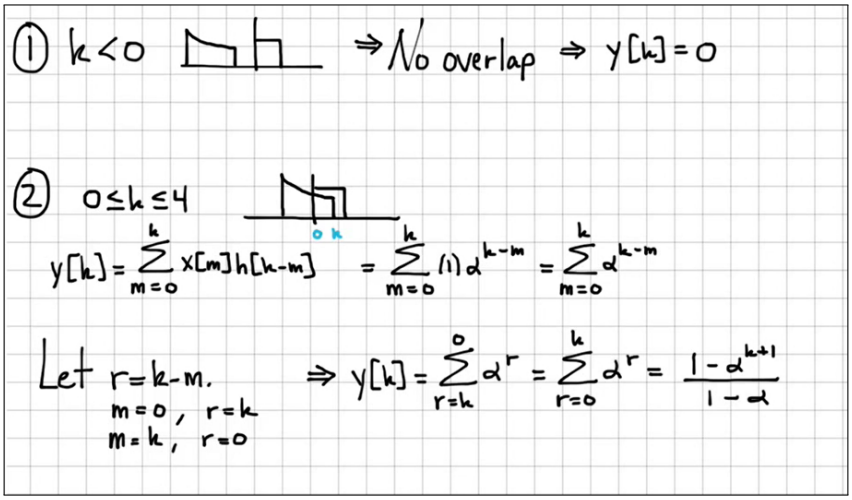


Figure 4.9: $k < 0$ and $0 \leq k \leq 4$

In Figure 4.9, we have made a change of variable $r = k - m$ so we can use equation (4.4), and also flip the order of series from $\sum_{r=k}^{0} \rightarrow \sum_{r=0}^{k}$.

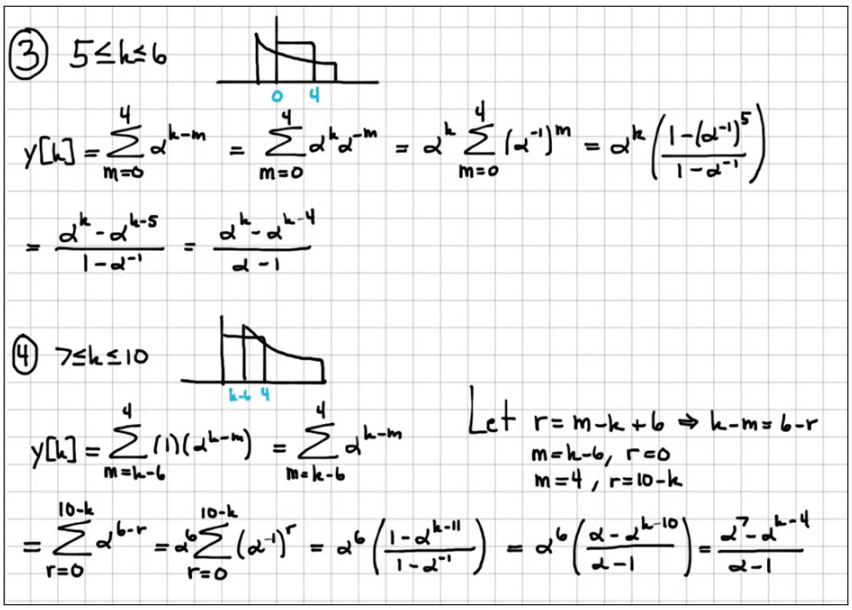


Figure 4.10: $5 \leq k \leq 6$ and $7 \leq k \leq 10$

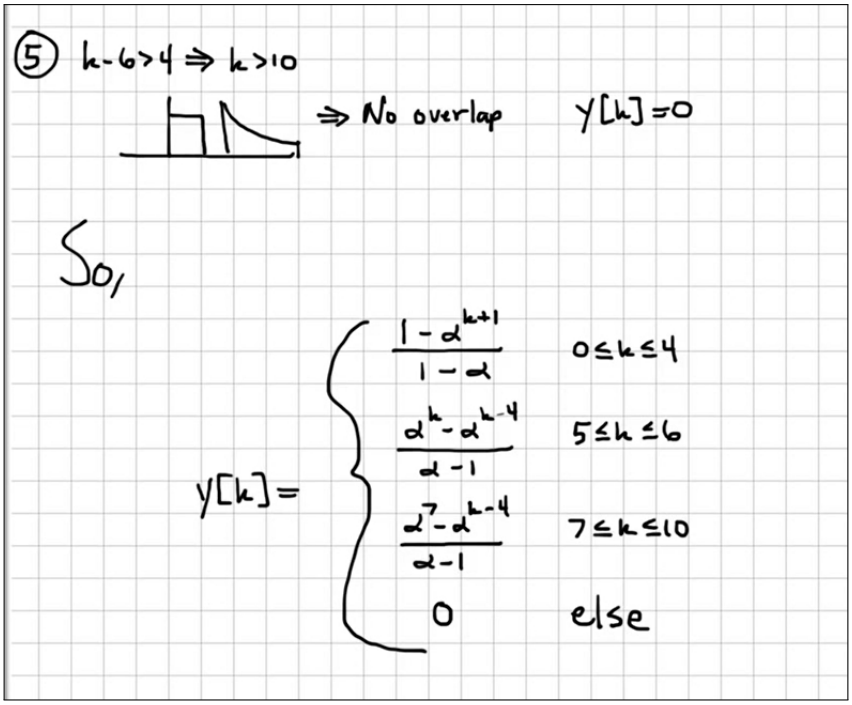


Figure 4.11: $k - 6 > 4$ and summing the cases

Chapter 5

Correlation

5.1 Definition

First we start by the definition.

$$\begin{aligned} R_{xy}(\tau) &= \int_{-\infty}^{\infty} x(t)\, y^{\star}(t-\tau) \\ R_{xy}[k] &= \sum_{n=-\infty}^{\infty} x[n]\, y^{\star}[n-k] \end{aligned} \tag{5.1}$$

Some points:

- We note that with respect to a convolution, the 2nd signal $y(t)$ is only delayed in time by τ, and there is no time flipping operation (in convolution, we have the 2).
- For a periodic signal with a period N, the $R_{xy}[k]$ will be also periodic.
- Length of signal:
  - Crosscorrolation: length $x[n] = N$, length $y[n] = M$, we have the $R_{xy}[k]$ length equal to $N + M - 1$
  - For autocorollation: for a length $N \rightarrow 2\,N - 1$

Chapter 6

Discrete Time Fourier Transform

6.1 Relation between DTFT and impulse sampled CTFT

$$\begin{aligned} \omega_c\,(\text{rad/sec}) &= \omega_d \times F_s \\ &= \frac{\omega_d}{T_s} \end{aligned} \tag{6.1}$$

Chapter 7

The Discrete Fourier Transform

7.1 Introduction

The discrete Fourier transform is a ***numerical technique*** which computes the DTFT (discrete time Fourier transform) and CTFT (continuous time Fourier transform). In other words, there is ***no transformation from time to frequency domain***.

7.2 Formula and Direct Results

Goal: Compute Numerically DTFT and CTFT. We start by DTFT.

Note: To see the derivation, review the slides.

7.2.1 Discrete time case: DFT of DTFT

Let $f[k]$ a discrete signal with index k, of length N_0.

$$\begin{aligned} \mathrm{DFT}\{f[k]\} &= F_r \\ &= F(r\,\Omega_0) \\ &= \sum_{k=0}^{N_0-1} f[k] \exp\left(-jr\Omega_0\, k\right) \end{aligned} \tag{7.1}$$

where $\begin{cases} r = 0, 1, 2, \rightarrow N_0 - 1 \\ \Omega_0 = \dfrac{2\,\pi}{N_0} = \text{freqeuncy step at which we sample the DTFT spectrum} \end{cases}$

Inverse DFT case:

$$f[k] = \frac{1}{N_0} \sum_{r=0}^{N_0-1} F_r \exp\left(-jr\Omega_0\, k\right) \tag{7.2}$$

7.2.2 Continuous time case: DFT of CTFT

- Let $f(t)$ be a continuous time signal of duration τ.
- Sample the signal $f(t)$ over a period $T_0 \leq \tau$, where T_s is the sampling time.
  - The ***number of sample*** we get is $N_0 = \dfrac{T_0}{T_s} = T_0 \times F_s$
- Formula:

$$\begin{aligned} \mathrm{DFT}\{f[t]\} &= F_r \\ &= F(r\,\omega_0) \\ &= \sum_{k=0}^{N_0-1} f_k \exp\left(-jr\Omega_0\, k\right) \end{aligned} \tag{7.3}$$

where $\begin{cases} \omega_0 = 2\,\pi\,F_0\,\mathrm{rad/s}\,(\text{from spectral sampling theorem}) \\ \Omega_0 = \omega_0\,T_s = \dfrac{2\,\pi}{N_0} = \text{sampling interval at wihich we sample the spectrum of the CTFT} \\ f_k = T_s\,f(k\,T_s) \end{cases}$

7.3 Examples

7.3.1 Continuous time case: Infinite Time

The signal is: $x(t) = \exp(-0.5\,t)\,u(t) \leftrightarrow X(\omega) = \dfrac{1}{j\,\omega + 0.5}$.

The Fourier transform can be found by any look up table.

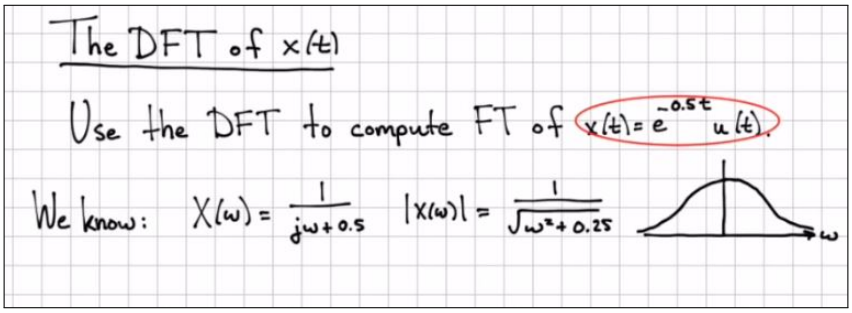


Figure 7.1: Given: Infinite time and infinite frequency signal

- To compute the DFT, we need to sample in the time domain (f_k) , and take samples in the frequency domain (Ω_0).
- However $x(t)$ in infinite in time and hence infinite in frequency, and there is no max frequency to apply the sampling period to.
  - Need to do some engineering judgment about this.
- We can put the following criteria: what is the value of ω at which the frequency content is bellow 2 % ?

- So we capture 98 % of the frequency content of the signal
- This value of $\omega = 50\text{rad/s} \leftrightarrow B = \frac{25}{\pi}$ Hz as shown in Figure 7.2

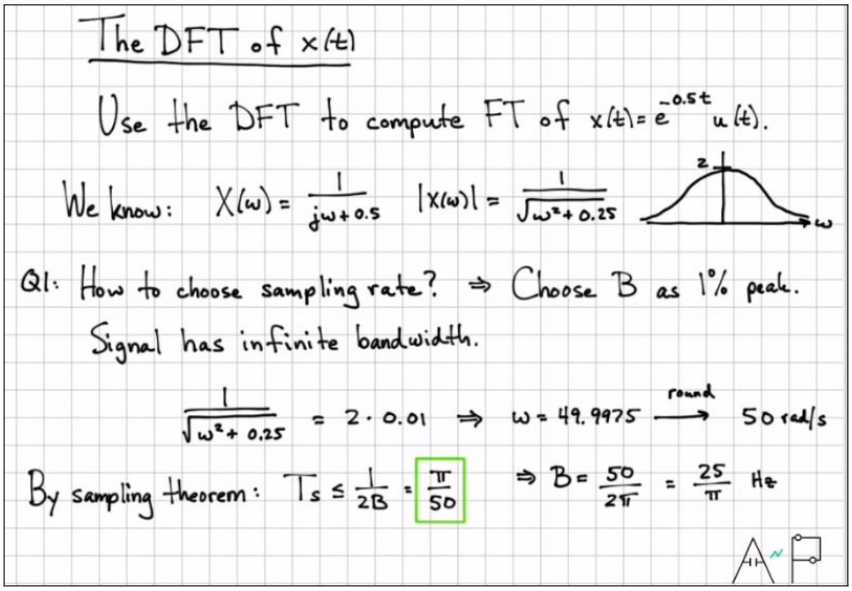


Figure 7.2: Solving for the sampling period T_s

- The we solve for the T_s and we get that we need to sample at a rate $\frac{\pi}{50}$ seconds (this the minimum, we could sample faster, we see the effect of this later)

- After we solve for $T_s = \frac{\pi}{50}$ s, there is another question to be asked: for ***how long do we need to sample***?
  - This signal is going forever, we can't just keep sampling forever
- So we need to solve for T_0

- We know that the signal is a decaying exponential, we can see for some value of T_0 where the amplitude value of $x(t)$ are very low, and by this we are not throwing some high value amplitude (information)
- Computation shown in Figure 7.3.

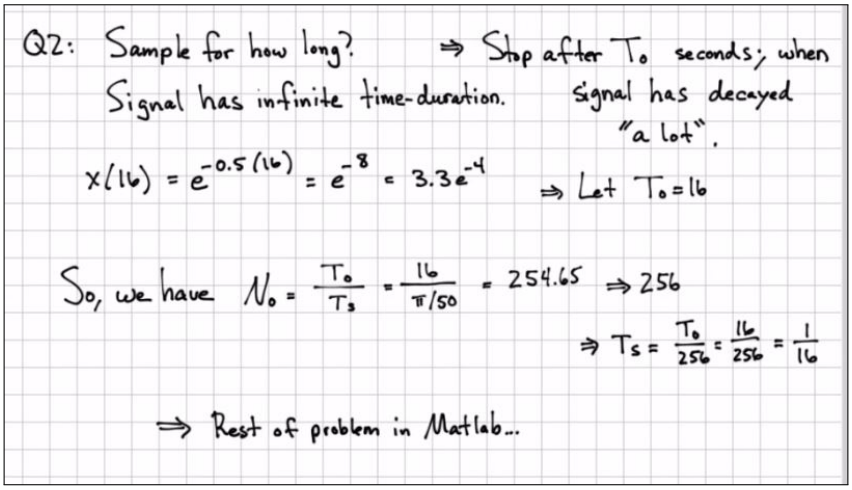


Figure 7.3: Solving for the sampling period T_s

  - We try for example for $T_0 = 16$s, we get $x(16) = 3.3\,e^{-}4$, very small number
  - If we choose $T_0 = 16$, the number of sample we get in this period using a sampling rate $T_s = \frac{\pi}{50}$ is: $N_0 = \frac{T_0}{T_s} = 254.65$
  - However, the number of samples we need to grab need to be integer, so let's say we need to grab 256 samples
  - We resolve again for T_s using $N_0 = 256$ samples, and we get $T_s = \frac{1}{16}$ seconds

The rest of the problem is done via coding.
As a summary we have:

- First question we ask since the signal $x(t)$ isn't band limited in the frequency domain: how do we choose the sampling rate T_s
  - $T_s = \frac{1}{16}$ seconds
- 2nd question: for how long we keep sampling ?
  - $T_0 = 16$ s, which give $N_0 = 256$ samples/second such that $T_s = \frac{1}{16}$ s

Some Tuning and Analysis:

2 factors influence the results of the DFT values: the sampling period T_s and signal duration T_0, which specify for how long we keep doing sampling. Let's try to change each factor while fixing the other.

Changing signal duration T_0:

The results of the computation are shown in Figure 7.4.

- The base line is the first row where $F_s = 16$ sample/ s and $T_0 = 16$s, as in the theoretical part
- First we truncate (throw away) the signal allot, which means T_0 decrease (decrease to 2 s in our choice)
  - Since $\Omega_0 = \dfrac{2\,\pi}{N_0}$ and $N_0 = \dfrac{T_0}{T_s}$, so we get less number of coefficient for DFT computation
  - And notice that the values of the DFT in the 2nd plot are below the blue curve
- When we increase the signal duration (truncate less) things tend to be better.

As a conclusion, even if the sampling rate F_s is good enough but we truncate the signal allot in the time domain, the spectral representation will not be good and we loss frequency content of the signal.

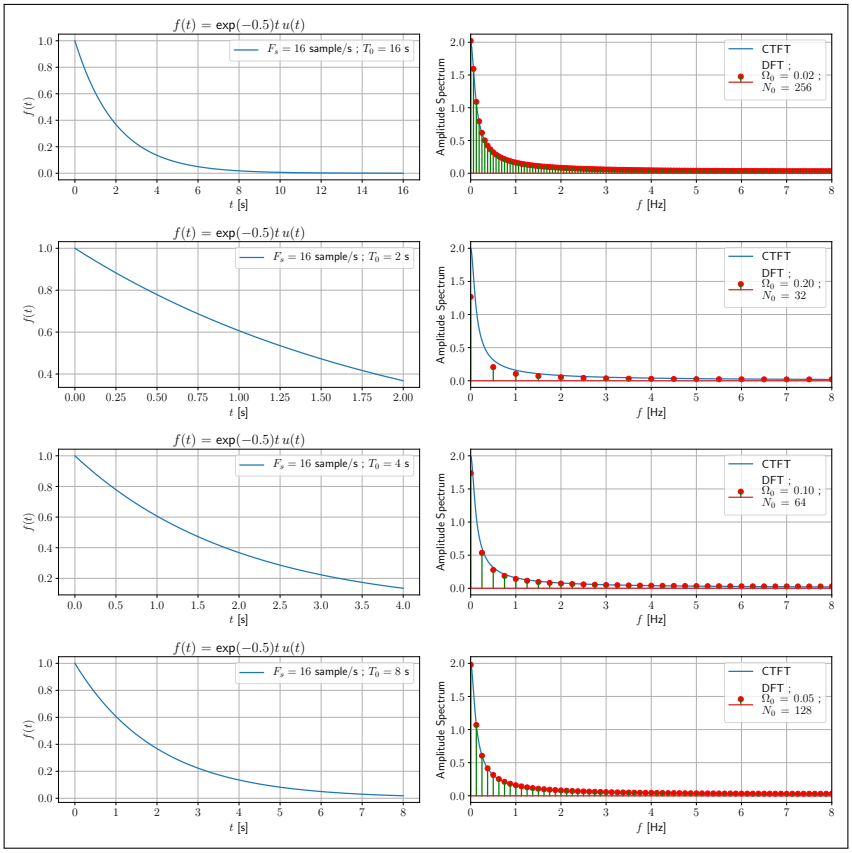


Figure 7.4: DFT results for example of Figure 7.1: Signal Duration T_0 and its effect on DFT

Changing the sampling rate F_s:

Results are shown in Figure 7.5.

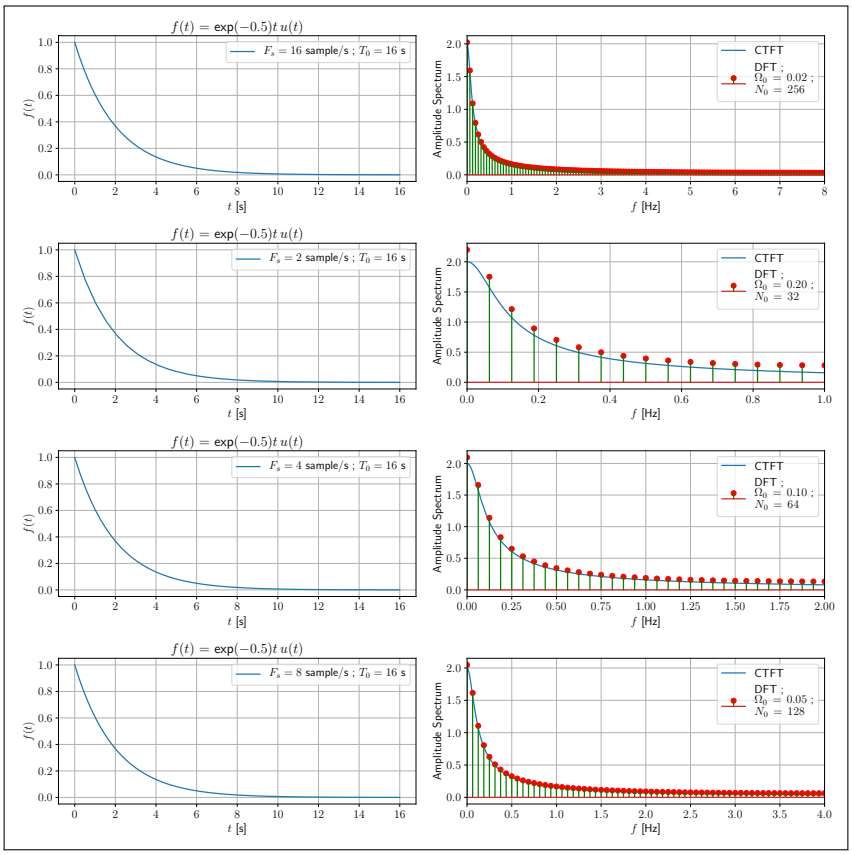


Figure 7.5: DFT results for example of Figure 7.1: Sampling rate F_s and its effect on DFT

- As usual the first plot is the base line: $F_s = 16$ sample/s and $T_0 = 16$s.
- Notice that for example when $F_s = 2$ sample/s (8 times less than the true one), the DFT values are above the true curve of the CTFT.

7.3.2 Discrete Time: Finite Length Signal

The problem is shown in Figure 7.6.

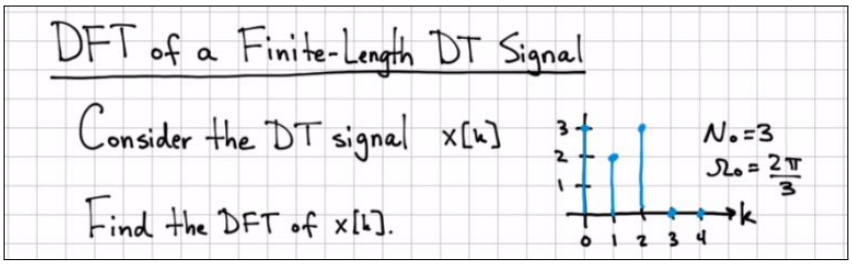


Figure 7.6: Discrete time signal: finite length

- It has 3 non-zero values at indices 0,1 and 2, so $N_0 = 3$
- Since $N_0 = 3$, so the DFT components for the DTFT will be also 3: $r : 0 \rightarrow 2$
- The ***sampling interval*** at which we compute the DFT coefficient values is $\Omega_0 = \frac{2\pi}{N_0} = \frac{2\pi}{3}$
  - So we compute the value at $0\,\Omega_0\,, 1\,\Omega_0$ and $2\,\Omega_0$

Since the values of the DFT are only 3, we can do the computation by hand easily using equation (7.1). The computations are shown in Figure 7.7

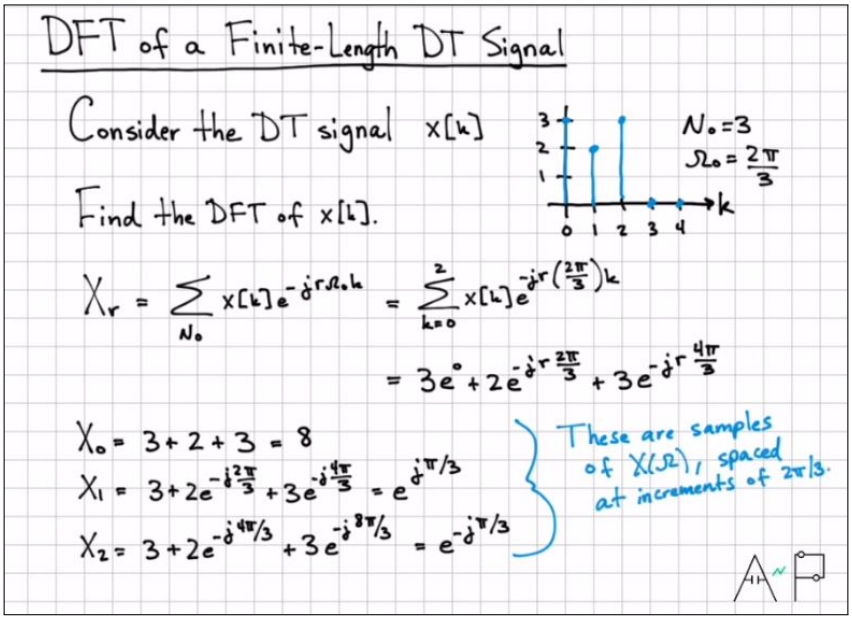


Figure 7.7: Computing DFT values of the DTFT using equation (7.1)

- First we compute a formula as a function of r for all values of time index k
- Then we plug each value of r to obtain $X_0\,,\ X_1$ and X_2

Now to compare the result of the DFT values, we compute also the expression of DTFT, detailed in Figure 7.8.

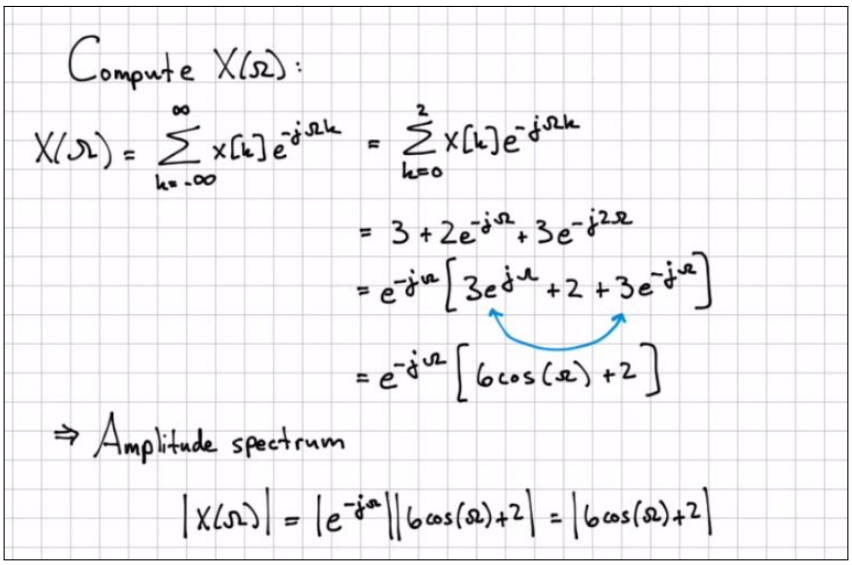


Figure 7.8: Computing DTFT expression for comparison with DFT values

Results and Analysis:

Results are shown in Figure 7.9.

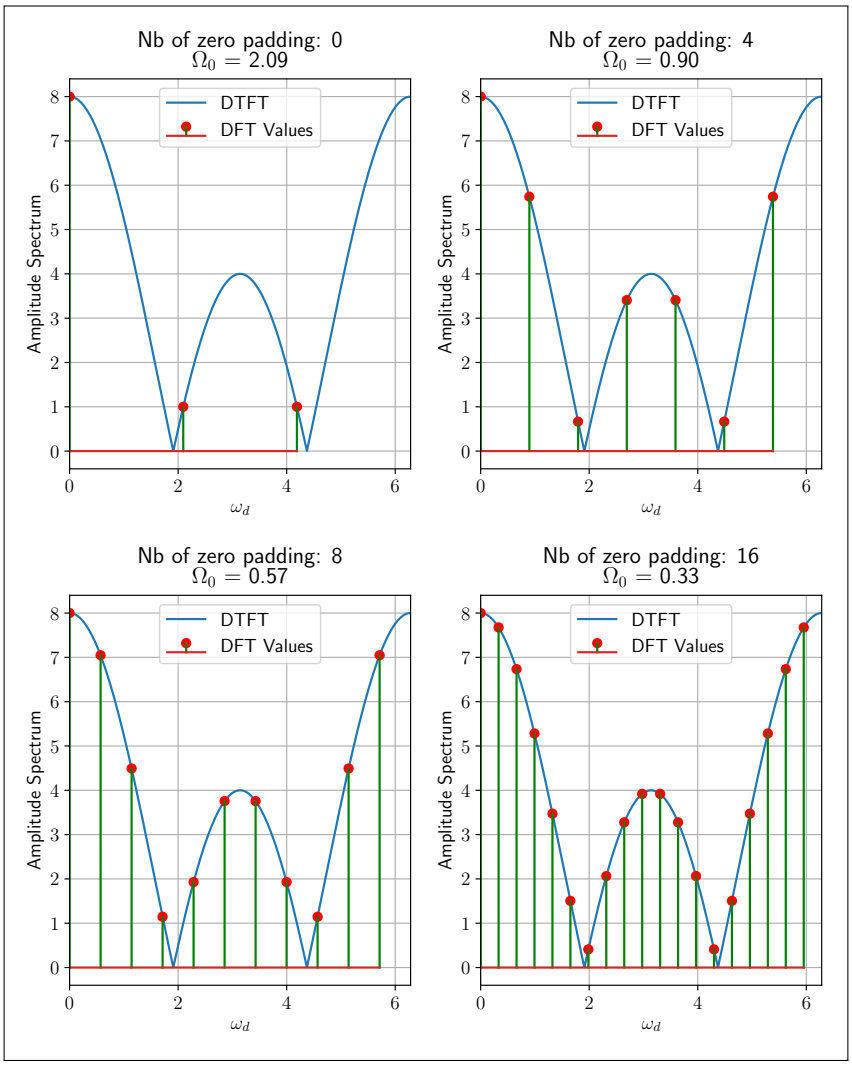


Figure 7.9: Results of Example of Figure 7.6
$0 \leq \omega_d \leq 2\,\pi$

- The result are also made with respect to ***zero padding*** effect
  - We pad (add) zeros values to the signal in the time (discrete) domain
  - This won't affect the signal, only the number of samples will increase
- Recall that the sampling interval $\Omega_0 = \dfrac{2\,\pi}{N_0}$ where N_0 is the length of the signal
- If we add more samples N_0, Ω_0 will decrease and hence we take more samples of the spectrum of the DTFT.

7.3.3 Discrete Time: Infinite Length Signal

Now we move with infinite length discrete time signal. As in 7.3.1, since we are dealing with infinite length sequence, we need to truncate the signal in the time (discrete) domain and make the index sampling k between 0 and $N_0 - 1$. Given are shown in Figure 7.10

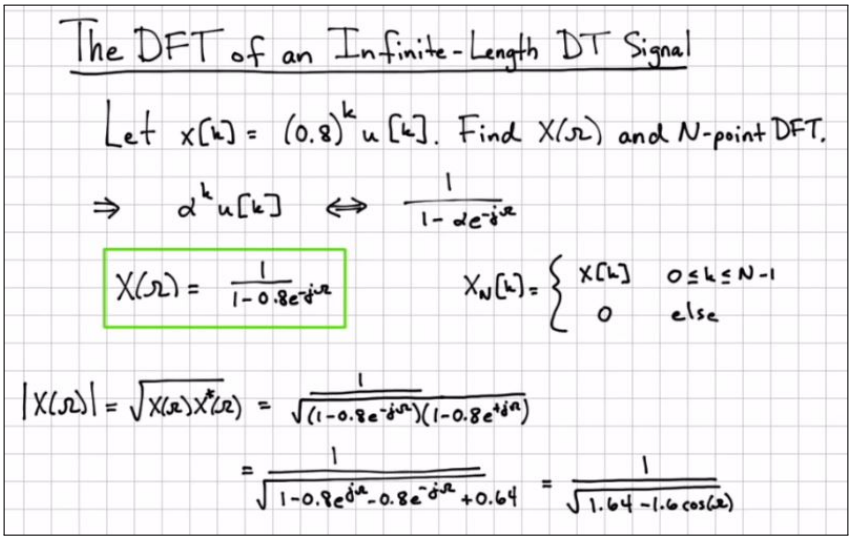


Figure 7.10: DFT for infinite length discrete time signal

Results are shown in Figure 7.11.

- k is the sampling index in the discrete time domain
- The base $a = 0.8$ in all cases
- p is the number of samples we get:
  - The more p we get, the ***less we truncate the signal in the discrete time domain***, and hence the DTFT using the loop formula and the DFT values will get more accurate to analytical DTFT formula

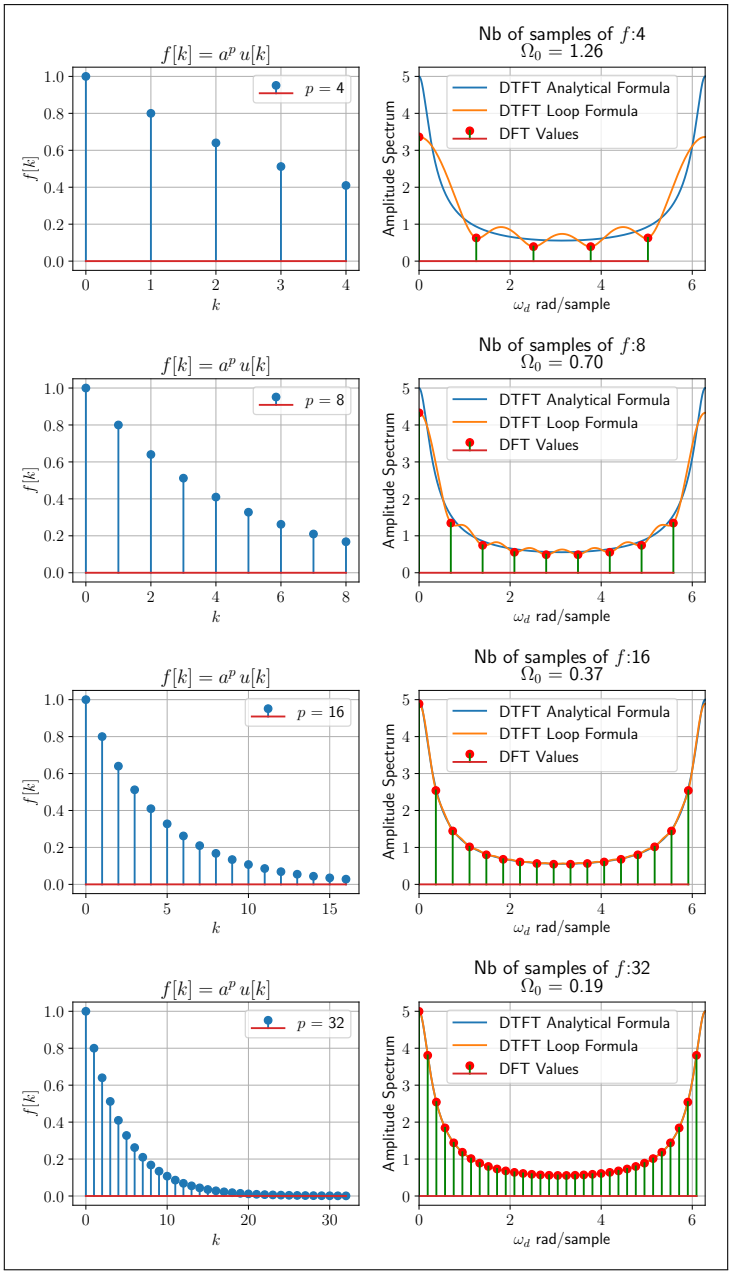


Figure 7.11: Result of example of Figure 7.10

7.4 Resolution and Zero Padding

After taking all possible cases of DFT computations for continuous and discrete time signals, we do some analysis of frequency resolution using 2 sin function.

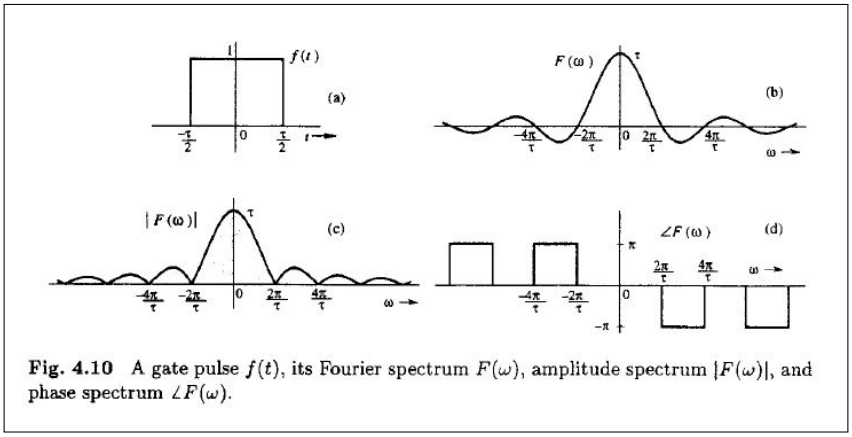


Fig. 4.10 A gate pulse $f(t)$, its Fourier spectrum $F(\omega)$, amplitude spectrum $|F(\omega)|$, and phase spectrum $\angle F(\omega)$.

Figure 7.12: rect Fourier Transform

- Recall that $\sin(\omega_0\, t) \leftrightarrow j\,\pi\,[\,\delta(\omega_c - \omega_0) + \delta(\omega_c + \omega_0)\,]$
- When we compute the Fourier transform of $\sin(\omega_0)\, t$, the signal in theory has domain $-\infty < t < +\infty$, but in reality we need to truncate the signal (or cut it), so we multiply it by a rect (pulse)
  - Recall that $\text{rect}\left(\frac{t}{\tau}\right) \leftrightarrow \text{sinc}\left(\frac{\omega_c\,\tau}{2}\right)$ shown in Figure 7.12.

Important Note about Data truncation:

- Let $f(t)$ some signal we want to study and $w(t)$ some window function we use where $\begin{cases} f(t) \leftrightarrow F(\omega_c) \\ w(t) \leftrightarrow W(\omega_c) \end{cases}$
- Truncation is done by multiplying the function by some window. Mathematically:

$$f_w(t) = f(t) \times w(t) \leftrightarrow F_w(\omega_c) = F(\omega_c) * W(\omega_c) \tag{7.4}$$

  - $f_w(t)$ denote the window signal resulting from windowing the original signal $f(t)$
  - Multiplication in the time domain result in convolution in the frequency domain
  - By the convolution width property, the width of the resulting signal will be the sum of the width of the 2 signal
  - So as an intuition, the bandwidth of $F(\omega_c)$ will increase by the amount of the bandwidth of window function we use $W(\omega_c)$

- Solving equation (7.4):

$$\begin{aligned} F_w(\omega_c) &= F(\omega_c) * W(\omega_c) \\ &= (j\pi\,[\delta(\omega_c - \omega_0) + \delta(\omega_c + \omega_0)]) * \text{sinc}\left(\frac{\omega_c \tau}{2}\right) \\ &= \text{sinc}\left(\frac{\tau \times (\omega_c - \omega_0)}{2}\right) + \text{sinc}\left(\frac{\tau \times (\omega_c + \omega_0)}{2}\right) \end{aligned} \tag{7.5}$$

  - We have the use the property of convolution with dirac function: when convolving a function with a dirac function, the function will be shifted to the dirac location.

- Equation (7.5) is illustrated in Figure 7.13

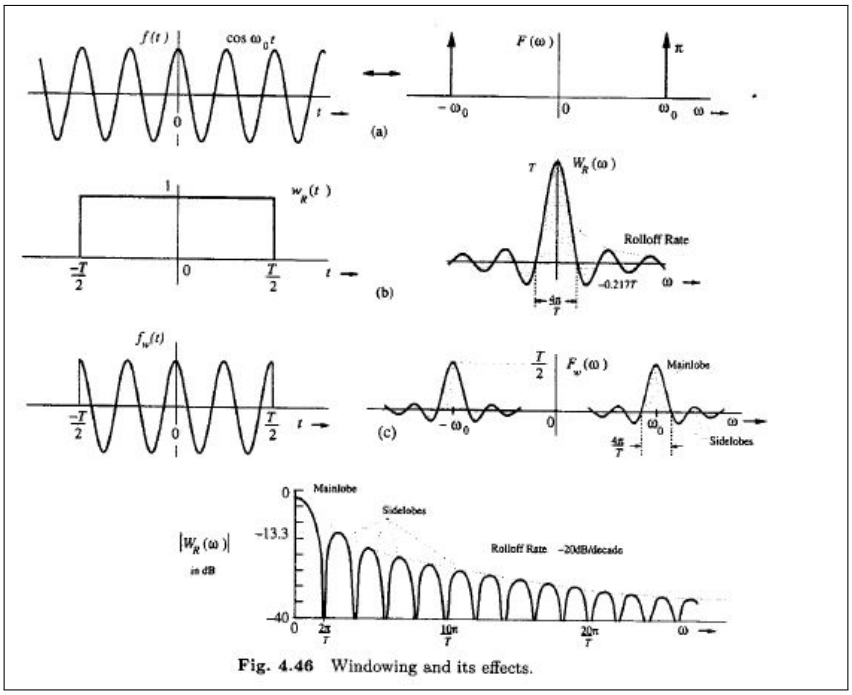


Figure 7.13: Window effect: ***spectral spreading*** and ***leakage***

  - The wider the window we use, the less we truncate
    - Since the bandwidth of the signal is inversely proportion

7.4.1 1 Sinusoidal Analysis

Consider the signal $f(t) = 2\sin(2\pi 500t)$. where $F_s = 8000$ sample/s.
The study again will be by changing 2 factors: number of samples we take in the time domain (so window length) and zero padding.

Zero Padding Effect:

We fix in this study the number of samples in the time domain (window length) and see the effect of zero padding.

Rule: zero padding doesn't effect the frequency resolution. In our case, the main lobe width will be the same whatever zero padding factor we use.
The only thing zero padding give is more sampling of the frequency spectrum, so we can get more point when plotting.
Results are shown in Figure 7.14.

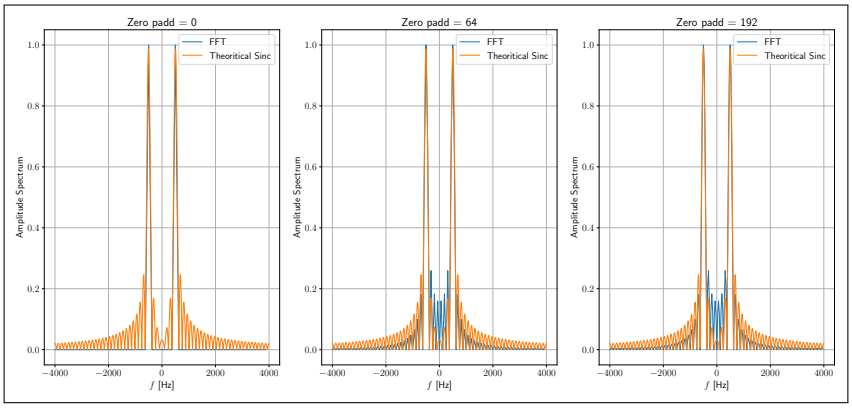


Figure 7.14: Zero Padding Effect: More samples in the frequency domain

Chapter 8

The Discrete Fourier Transform (Enhanced)

Reference: This way of introduction via the DTFT is from [2].

8.1 Introduction

- The DFT is a tool to approximate CTFT and DTFT
- First in [2] he talk about approximating DTFT
- Motivation for DFT:
  - In analysing data, we always have samples and not continous time signals
  - Even if we want to plug in $x[n]$ in the formula of the DTFT, we need also to evaluate on infinite grid over $[pi + \pi]$, and in computer we can't do this $\rightarrow$ this rise the need of approximation and evaluating DTFT over finite grid in $[pi + \pi]$
- Be aware when using IDFT, the formula is for N points, and it doesn't include zero padding option
  - In other words, if use $M > N$ points for DFT, the IDFT use only the original N points
- After finishing up with the DTFT, we will see the relation between the DTFT, CTFT and DFT
- FFT is an algorithm to implement the DFT in an efficient manner
  - Although people say *the* FFT, but in reality there is many version of the FFT

8.2 Finite Duration Case

- In this section, the focus were on the understanding of the DFT for the DTFT of some finite duration signal $x[n]$

  - How to take the frequency step
  - How to compute the DFT Formula
  - The idea of zero-padding and how it enhance the only the shape of the DFT spectrum

8.3 Infinite Duration Case

Now we move to the infinite duration case.

- Slide 11:

  - When doing this example, Prof Mark started with the case of a finite duration signal, then armed with this kind of understanding, we will tackle infinite duration case
  - Now some person wonder: practical signal are always finite in duration, and no signal can just keep going forevever.

    Well, the infinity here may come from different cases:

    1. Perhaps we are analysing some signal, which is very long and we don't have enough memory to store all (the finite) value of $x[n]$
    2. Maybe we are doing some radar processing, and we can't just keep waiting so long, and we need to just capture the signal and quickly analyze it (electronic warfare case)

- The infinite duration case can be modeled as windowing operation for the infinite duration signal

  - In frequency domain, it will be a convolution
  - This will induce smearing in the DTFT spectrum of the $x[n]$

- So an ***important point:*** the DFT will help us see only the smeared spectrum of $x[n]$ as shown in Figure 8.1

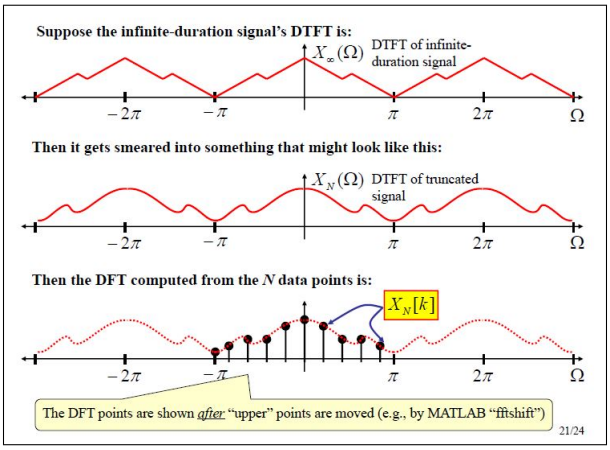


Figure 8.1: The DFT give us the smeared spectrum only

  - The 2nd is the smeared spectrum due to windowing
  - It is important to understand that the DFT will give us only the 2nd row of Figure 8.1
  - In other words, truncation or windowing (source of error) cause a smeared spectrum (type of error)
  - Another source of error is due to aliasing (this is the 2nd type of error we will explore its source later this later)

8.4 DFT for Spectral Analysis

Now we move on for the practical use case of the DFT.
Say we have some signal we are interesting in analyzing its spectral content:

- It could be a radar signal available only for a ***short time***
- It could be some RF signal captured by an antenna, downcevert it and shifting it using a mixer then pass it through an ADC (or the ADC is near the antenna, and the ADC is in the bandpass mode)

Design criteria: the real issue in DFT is when we reduce the samples we have for whatever the reason (signal available for only small time,$\cdots$) the resolution of the spectral component decrease.
So the big challenge is to do spectral analysis with high resolution with small amount of data.

8.4.1 Modeling the Truncation Problem

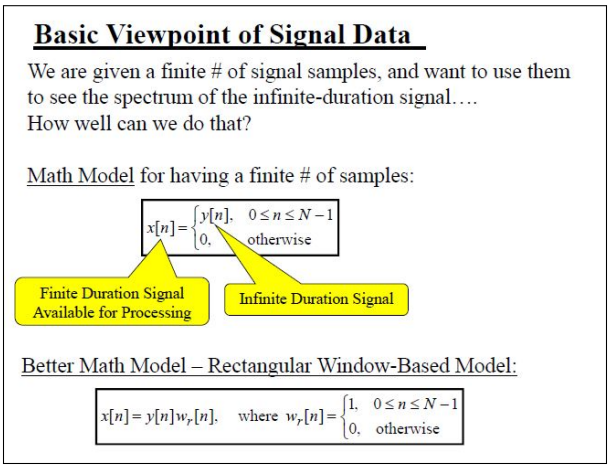


Figure 8.2: Truncation: using the rect window

- We have some infinite signal $y[n]$ we can't collect it (recall from section 8.3 what are the other sources of infinity)
- The actual finite signal we have is $x[n]$
  - A better mathematical modeling is the multiplication by the rect window
- Be aware that in reality, all we do is collecting N samples
  - But to analyze this problem mathematically, we use the windowing idea: all the samples are given into some window and outside this window the values of these samples are 0

The multiplication in time domain give us a convolution in the frequency domain, as shown in Figure 8.3

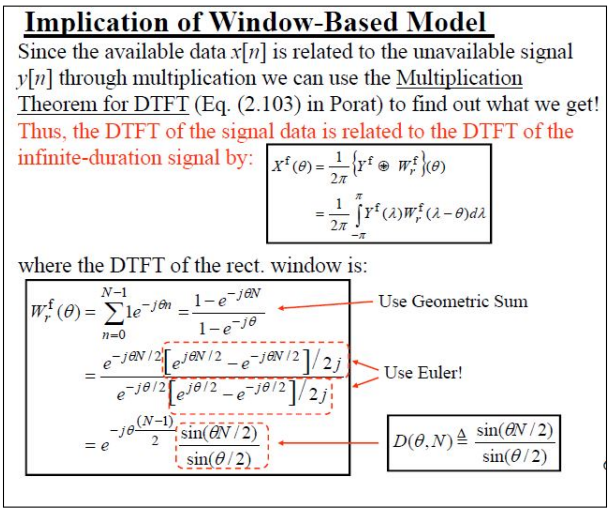


Figure 8.3: Truncation: using the rect window

- The DTFT of a rect is not exactly a sinc function: it is a Drichlet function

In Figure 8.4, we have the amplitude spectrum of the Drichlet for 2 values of N

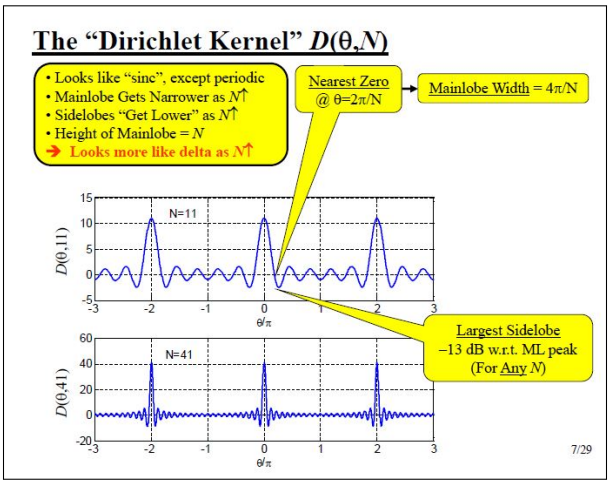


Figure 8.4: Truncation: using the rect window

- for a bigger N,
  - The mainlobe width get's smaller, converging into a delta $\leftrightarrow$ less leakage
  - the maximum get's bigger
  - But the side lobes won't get smaller

Figure 8.5 show an example of the convolution example

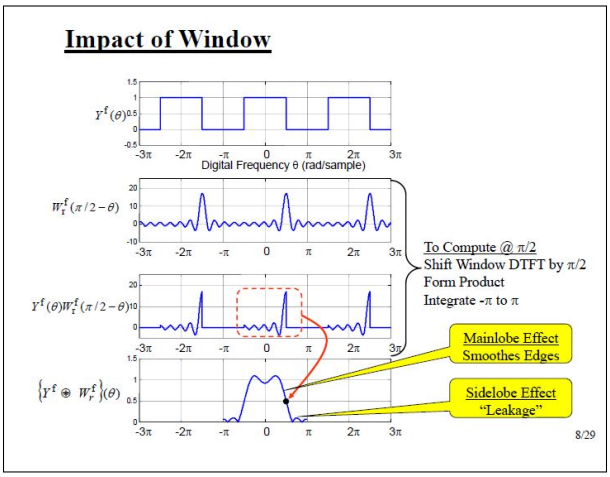


Figure 8.5: Truncation: Exmaple

- $Y(\theta)$ is the true DTFT
- In the 2^{row} we have the Drichlet shifted at $\dfrac{\pi}{2}$
- We do the multiplication and then do the intergral to obtain 1 value at the last line

- if we want to obtaine other values, we repeat the same process (sliding the window) for other values then $\frac{\pi}{2}$
- Notice that if the Drichlet main lobe was thinner (like a Dirac), the transition band would be sharper in the last row

8.5 Example

This section take the causal exponential example, and try to simulate it to illustrate all the points we learned so far, which are summarized in Figure 8.6

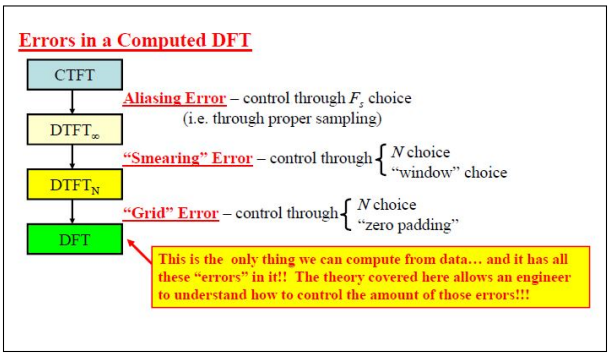


Figure 8.6: Error Types in a DFT Analysis

Remember that in real world, all we can do is the DFT, and all the other things we can't do it since we don't necessarily have theoretical expression for the correspondent FT (CTFT or DTFT).

Chapter 9

DFT V3

9.1 Introduction

This chapter contains note about the DFT taken from chapter 3 in [3].

9.2 Notes

- DFT Equation:

$$X[k] = \sum_{n=0}^{N} x[n] \exp\left(\frac{-j2\pi\, n}{N} k\right) \tag{9.1}$$

  where: $\begin{cases} k : \text{frequency index from } 0 \rightarrow N-1 \\ x[n] : \text{input sequence of length } N \\ n : \text{time domain index for the sequence } x[n] \end{cases}$

- DFT compute the spectrum content of N equally spaced frequency points
- N: is an important parameters since it determine

  1. how much input samples are needed,
  2. the resolution of the frequency domain result
  3. The amount of computation

- The frequency spacing: for a sampling rate F_s and N discrete time input sequence, the fundamental frequency in a DFT analysis is:

$$\text{Fundamental Frequency} = \frac{F_s}{N} \tag{9.2}$$

  All the other frequencies will be multiple of Equation 9.2 $\leftrightarrow k \times \frac{F_s}{N}$.

- Leakage:

  To illustrate leakage, we give some example.

  Suppose we have the following time domain signal:

$$x(t) = \sin(2\pi 1000\, t) + 0.5 \cos\left(2\pi 2000\, t + \frac{\pi}{4}\right) \tag{9.3}$$

  If we take $N = 8$ and $F_s = 8000$ Hz, then the frequencies produced by DFT analysis (using $k \times \frac{F_s}{N}$):0 , 1000 , 2000, $\cdots$.
  Notice that the frequency bins produced by Equation 9.2 coincide with the frequencies of $x(t)$. However, in real word signals we can't control usually this choice. This phenomena is what we call ***leakage***.

  Leakage can be reduced, but can't be eliminated , so we will try to understand it more.

  The leakage problem is also treated from another starting points in different references:

  - For example chapter 7 in [4], they introduce the leakage through the windowing problem, and not directly manipulating the DFT equation
  - Also they provide a good examples to illustrate what happens if the DFT bins doesn't coincide with input signal frequency
  - The best window to use to minimize the effect of leakage and also provide a tradeoff between leakage and resolution is the Hann window

kage:

  to do this in simulation later and ***unify all these concepts***.

Chapter 10

Windowing

Source:

- See chapter 6 in [5]
- See lecture note *Windows for Spectral Analysis of Signals 23* for figures and reference.

10.1 Design Criteria of a Window

- Slide 3: focus on the design of window and what we need as characteristics in some particular window
  - Low side lobes and a rapid rate of decay
  - Narrow main lobe
  - Always a tradeoff between these 2
- Slide 4: The rectangular window has the narrowest main lobe among all the windows we will see
  - Side-lobe drop off rate: -6 dB/octave: octave is the double of the frequency. At each double frequency point, the side-lobe magnitude in dB decrease by 6.

    But around $\pm\pi$, the side-lobes remain constant

10.2 Window Comparison

- Slide 5-6: Barlette Window (known also as triangular)
  - The idea is to lower the side-lobes level
  - So he squared the magnitude of a rect window in frequency domain $\leftrightarrow$ convolution of 2 rect window in the time domain of size $\frac{N}{2}$ per window (to obtain a window of size N)

  - As a result of this squaring, the side lobe ratio $\left(\frac{\text{Main lobe amp}}{\text{side lobe amp}}\right)$ compared to a rect window has decreased
  - Barlett has a continuity near the edge of the window (the continuity will have influence on the rate of decay of the side lobe levels)
  - But the cost is a higher main lobe

- Slide 7-8: Hann window
  - He did a shifting in the frequency of rectangular window to cancell as much as he can the side lobe
  - The largest side lobe level is bettet then Barlette: it is -32 dB vs -27 dB for Barlette
  - And has a faster rate of decay for the side lobes compared to Barlette

- Slide 9-10: Hamming window
  - Hamming took the idea of Hann and play on the amplitude factor
  - He succeeded to obtain a lower side lobe level compared to Hann (- 43 dB compared to -32 dB for Hann)
  - But the decay rate of the side lobes are much slower (6 dB/octave): this is because of the discontinuity of the Hamming at the edge of the window

10.3 Derivative,Continuity and Rate of Decay

e of Decay

Rate of Decay and Derivative concept: to be revisited later to see how to take derivatives as examples.
For now, I will write the main ideas
Slide 11

- In this section, we think in window in continous time to be able to use the concept of derivative
- For a rect: if we integrate we obtain a triangular function which is continuous, so $p = 0$ in this case
  - We can't do derivative because it will give 0
  - trinagular: it will give us some function if we take the derivative, so $p = 1$

Chapter 11

Discrete Time System Implementation

11.1 Introduction

The big question in this chapter is now how to build discrete time systems.

We know that the output of a system at an index k is done via convolution, and there is 2 types of system: FIR and IIR.

$$\begin{aligned} y[\,k\,] &= \sum_{i=0}^{k} h[\,i\,]\,x[\,i\,-\,k\,] \quad \text{IIR} \\ y[\,k\,] &= \sum_{i=0}^{M-1} h[\,i\,]\,x[\,i\,-\,k\,] \quad \text{FIR} \end{aligned} \tag{11.1}$$

- Notice that in IIR, the upper limit in $\sum$ is the same as the index of the output signal y
  - This is a problem, because as k increase, we need to add more terms via $h[i]$
- However in FIR, the upper limit is constant whatever the index k we need to compute for.
  - Each output needs M multiplication and $M\,-\,1$ additions

The FIR filter can be taught as the following block diagram:

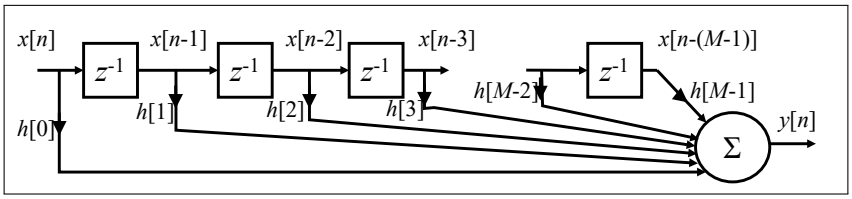


Figure 11.1: FIR Block Diagram

- We are assuming causal signal $x[n]$ and system $h[n]$
- For each clock cycle, so for each value of index n:
  - $y[0] = x[0]\,h[0]$, and all the other terms are 0 since we are causal system/signal
  - In the next clock cycle $y[1]$: $x[0]$ gets shifted (the term $x[n-1]$), and hence for $n = 1$

$$\begin{aligned} y[1] &= x[n]\,h[0] + x[n-1] + h[1] \\ &= x[1]\,h[0] + x[1-1]\,h[1] \\ &= x[1]\,h[0] + x[0]\,h[1] + \cdots \end{aligned} \tag{11.2}$$

  and all the other terms are 0

The FIR block diagram shown in is called ***feeforward*** because everything is coming in forward direction to the $\sum$ block.

11.2 IIR and Recursion

Theoretically, convolution can implement FIR and IIR, but in IIR it poses problem regarding computation.

In this section, we see how we can implement ***some*** IIR system using recursion (feedback).

Consider the accumulator system:

$$y[n] = \sum_{i=0}^{n} x[i] \tag{11.3}$$

which is an IIR system having an impulse response $h[n] = u[n]$. We can rewrite Equation 11.3 as:

$$\begin{aligned} y[n] &= \sum_{i=0}^{n} x[i] \\ &= \sum_{i=0}^{n-1} x[i] + x[n] \\ &= y[n-1] + x[n] \end{aligned} \tag{11.4}$$

So to compute the current output sample $y[n]$, we need to save a value of the past output sample $y[n-1]$ and add the current input sample $x[n]$.

Equation 11.4 lead to the concept of general N^{th} order linear constant coefficient difference equation:

$$y[n] + a_1\, y[n-1] + \cdots + a_N\, y[n-N] = x[n] + b_1\, x[n-1] + \cdots + b_M\, y[n-M] \tag{11.5}$$

Equation 11.5 can be rewritten in recursive form:

$$y[n] = -\sum_{k=1}^{N} a_k\, y[n-k] + \sum_{k=0}^{M} b_k\, x[n-k] \tag{11.6}$$

Equation 11.6 can be used to compute the current output sample $y[n]$ using a finite amount of resources.

11.3 Block Diagrams

Block diagrams are the first step in implementing discrete time systems. They help us to define the internal structure of a discrete time systems, so they can help us when writing code or thinking about a specific hardware to implement and deploy discrete time systems.

We first begin from a specific example, and then take the results to generalize them.

11.3.1 First Order Difference Equation and Direct Form

Suppose we want to construct the block diagram of the following 1st order difference equation:

$$y[n] = -a_1\, y[n-1] + b_0\, x[n] + b_1\, x[n-1] \tag{11.7}$$

- It is called 1st order because the output samples $y[n]$ and $y[n-1]$ differ only by 1 sample
- There are 2 delays: from input and output.

A straightforward block diagram of Equation 11.7 is shown in Figure 11.2.

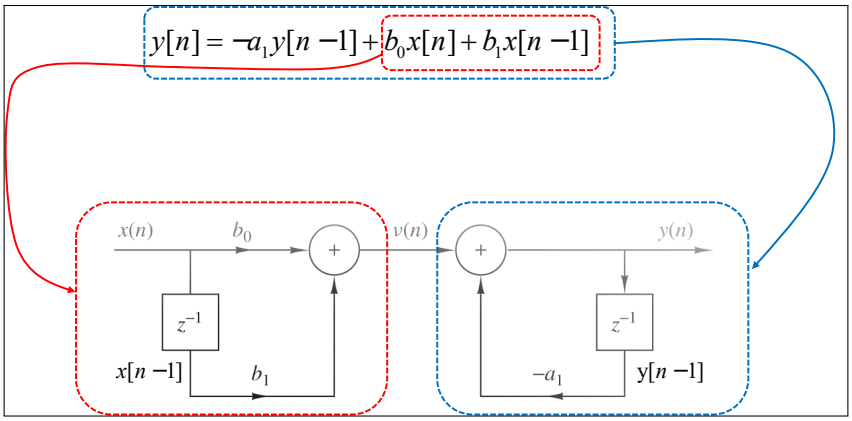


Figure 11.2: Direct Form I for 1st order difference equation

- It is called ***direct form I*** because the delays of input and output are separated

Remember that delays elements are storage units (memory units) which holds previous samples, so the less we have memory the less resources we need.

We can reduce delays elements by doing a handy trick: since we are working in LTI systems, we can ***interchange the order of the systems without affecting the results***. Theses steps are shown in Figure 11.3.

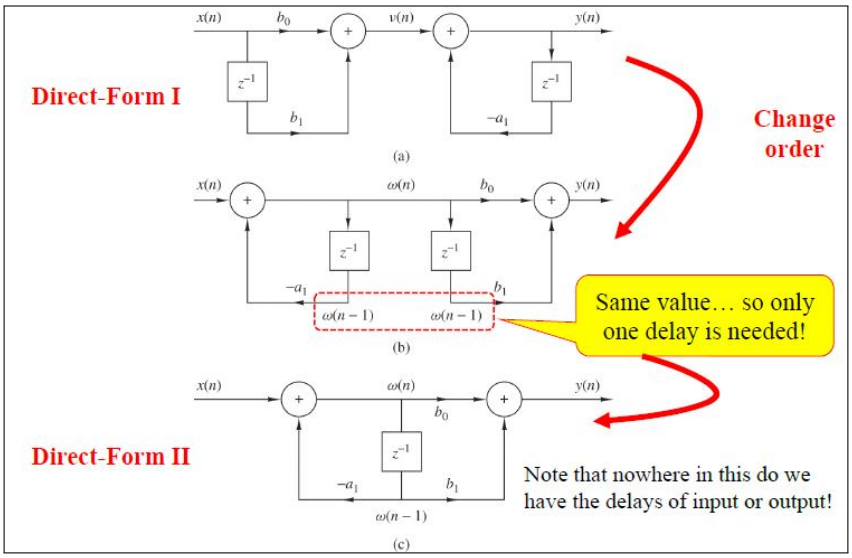


Figure 11.3: Interchanging order to reduce delay elements

- Notice that in b), we have the same ***intermediate number*** $\omega(n)$ as input to the same delays, that's why we need 1 delay element only and not 2

Final results is shown in Figure 11.4.

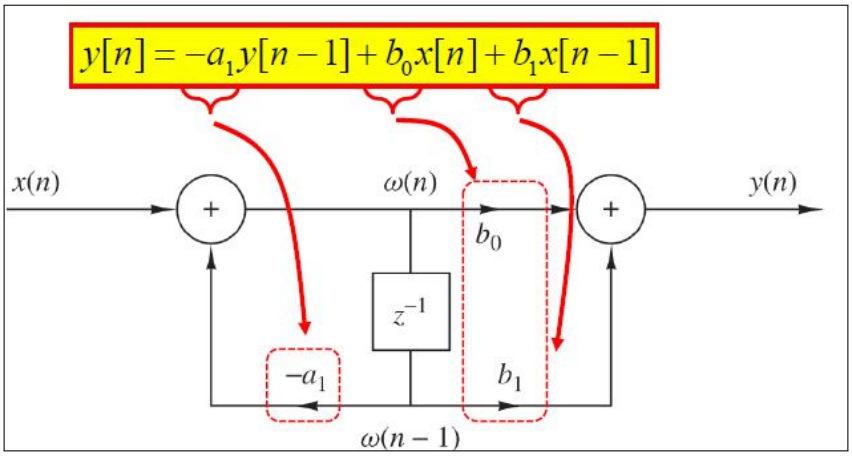


Figure 11.4: Direct form II for 1st order difference equation

11.3.2 General Direct Form

Now we can generalize the direct form 1 and 2 from previous sections.

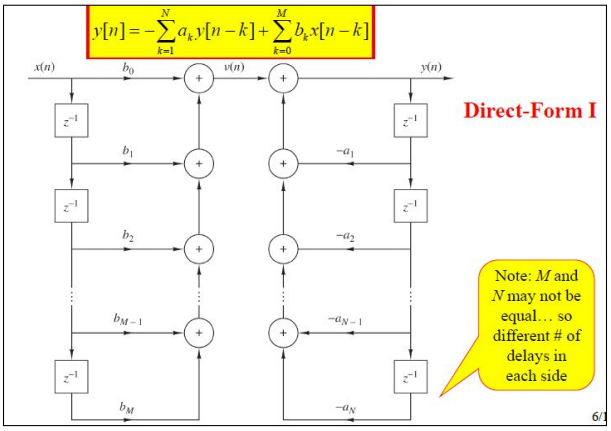


Figure 11.5: Direct form I General Form

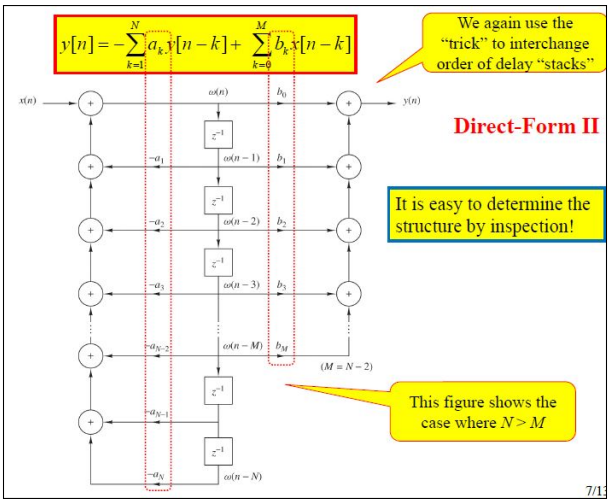


Figure 11.6: Direct form II General Form

- Note that it is not necessary that $N = M$, so we can have different delay elements in direct form 1
- For direct form 2: if $N \neq M$, we don't take branches always

11.3.3 Recursion and Non Recursion

Direct form 2 constitute from 2 parts: the b_k (input coefficients), which are directly fed into the sum blocks, so they are ***non-recursive***

11.4 General Filter Design Concept

Before diving into specific design technique for FIR or IIR, we present some general concept concerning filters in generals.

1. Discrete time filters present much more advantage over analog filters since we can use digital signal processing

2. Ideal response of a filter can't be achievable since the impulse response is a non-causal (sinc function)

3. We can only implement causal system

   - Causal system are proven by Proney-Wiener Theorem, which give us a causal $h[n]$ for square integrable and convergent ln integral
   - But also since we want the integral of the ln to converge, we can't allow for $H_{\text{filter}}(\omega_d)$ to take 0 on some interval, or the integral will blow up since ln 0 tends to $+\infty$, but we can have 0 values for some points $\leftrightarrow$ we can't have perfect stop band

4. Structure of the Filter:

   Proney-Wiener Theorem just says that mathematically we can obtain a causal response $h[n]$, but it doesn't state anything about the structure of the filter.

   Our goal in filter design is to approximate some desired frequency response H_{desired} using a rational transfer function.

   - Why rational ? because using rational transfer function, we can implement filter using conventional block diagrams

5. Practical Specifications: Every filter type have characteristics when designing: pass band, transition band, and ripples shown in

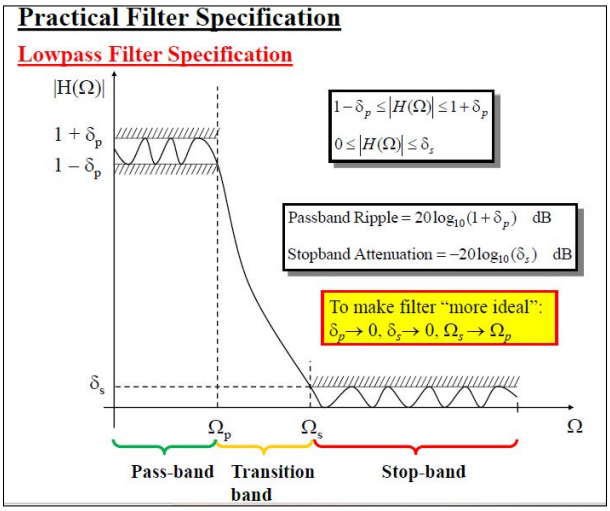


Figure 11.7: Practical Filter Specification

   - The more we approach ideal filter case $\leftrightarrow$ $\delta_p \to 0$, $\delta_s \to 0$ and $\Omega_s \to \Omega_p$ (narrower transition band), the more the designing of the filter (IIR or FIR) become more complex

- Concerning the stop band attenuation: it is $-20\log_10\delta_s$, we put a - to make it positive because when we say attenuation, we say for example attenuation of 40 dB, but in reality it is -40 dB

6. FIR and IIR comparison: To do later.

Chapter 12

FIR Design

12.1 Linear Phase Revisited

Before diving into the design of FIR filters, we revise the concept of linear phase because it will be used later.

If we have some transfer function $H(\omega_d)$, it is called linear phase if the angle spectrum has the form of a straight line.

The linear phase is caused because the amplitude spectrum of $H(\omega_d)$ pass through 0, which causes a flip in the angles, as shown in Figure 12.1.

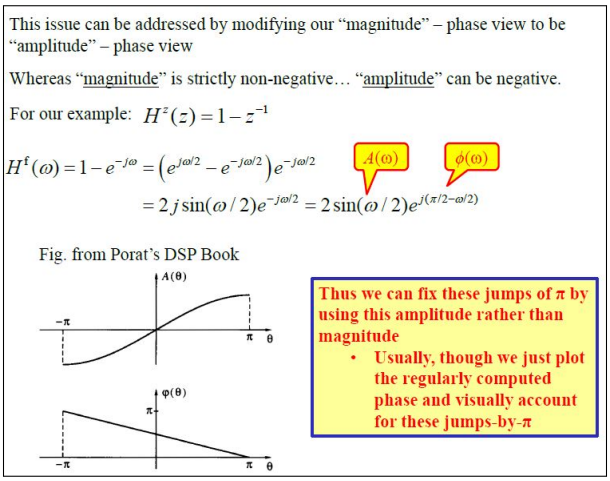


Figure 12.1: An example of linear phase system function

- Notice that we let the amplitude $A(\omega_d)$ be positive and negative
  - Not like MATLAB command `abs` which compute the amplitude spectrum only using positive values

In numerical computation using MATLAB, the angle spectrum will be shift by π, because MATLAB won't allow angle values be bigger then π.

An example is shown in Figure 12.2, where every time the angle will be bigger then $\pm\pi$, MATLAB will do a shift by 2π, and this cause to make the jumps in the plots.

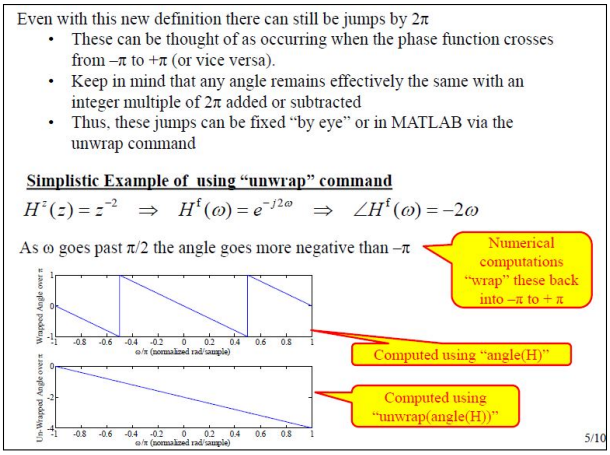


Figure 12.2: Wrapping Error for Phase Spectrum

To fix the 2π jumps, we can use the `unwrap` command. But be aware, that `unwrap` ***fix only*** $\pm 2\pi$ ***jumps, and not the*** $\pm\pi$ ***jumps***. Notice for example that $\omega = 0$, $\phi(\omega) \neq 0$, and this is due to π shift.

Handling π shift is done theoretically or by hand.

12.2 Linear Phase for FIR

12.2.1 Recap for FIR Equations

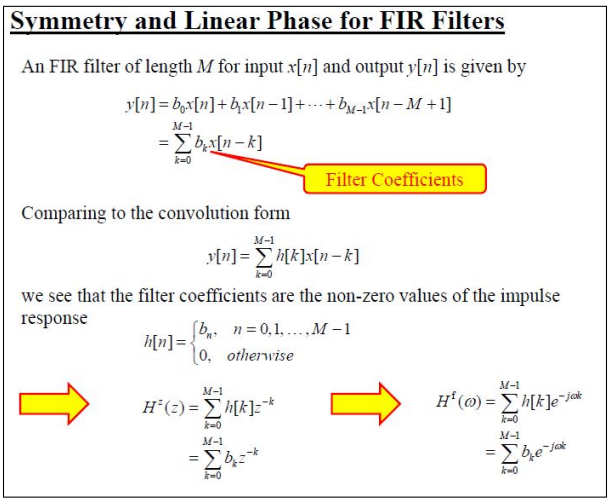


Figure 12.3: FIR Recap

- We have the difference equation of an FIR filter
- The impulse response $h[n]$ is composed from coefficients of the difference equation
- Filter coefficients are called ***taps***

12.2.2 Linear Phase definition for FIR

We say that an FIR is linear in phase if its impulse response has symmetry

$$h[k] = \pm h[M-1-k] \quad \text{for } k = 0, 1, \cdots, M-1 \tag{12.1}$$

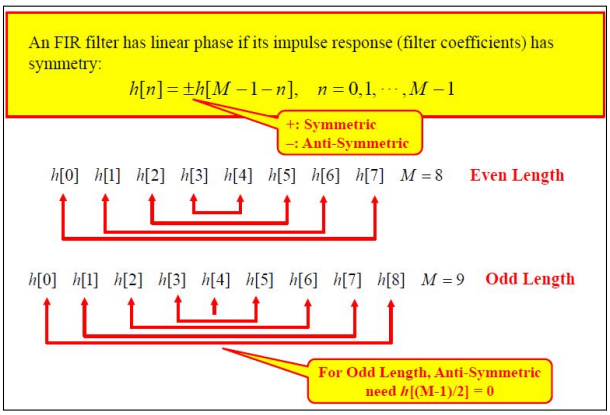


Figure 12.4: FIR Symmetry

Notice that for odd length, we have a constraint for antisymmetric case. In the example of Figure 12.4, if we have in symmetric case, we can say that $h[4] = h[4]$. But if we are in antisymmetric case, the only way we can say that $h[4] = -h[4]$ is if $h[4] = 0$, so we have some constraint to keep in mind in case of odd length antisymmetric case.

Using Equation 12.1, it can be shown that the frequency response $H_{\text{FIR}}(\omega_d)$ can be written in the form of an amplitude times an exponential having a linear phase. For a specific case demonstration, an example is shown in Figure 12.5, where the proof is done for the case of symmetric even length.

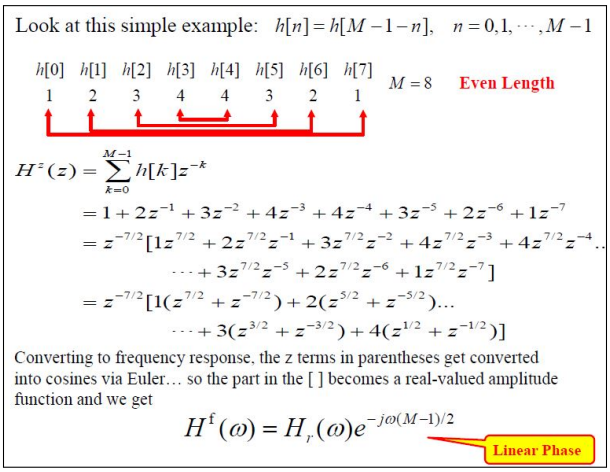


Figure 12.5: FIR Linear Phase Proof by a example

General Formula:

$$\begin{aligned} h[M-1-k] &= \rightarrow H_{\text{FIR}}(\omega_d) = \text{Amp}(\omega_d)\exp\left(\frac{-j\,\omega_d\,(M-1)}{2}\right) \\ -h[M-1-k] &= \rightarrow H_{\text{FIR}}(\omega_d) = \text{Amp}(\omega_d)\exp\left(\frac{-j\,\omega_d\,(M-1)}{2}+\frac{\pi}{2}\right) \end{aligned} \tag{12.2}$$

For antisymmetric case, we have a shift of $\frac{\pi}{2}$ in the phase spectrum.

12.2.3 Zero Pattern

For a linear phase FIR filter, we get an inverse of zero in addition to the initial one.

12.2.4 Use case for Symmetric and Anti-symmetric

Antisymmetric are note suited for LPF, because $H(\omega_d = 0) = 0$, which is not wanted in a LPF frequency response.

12.3 Window Design Technique

This method is not the commonly used one since it is not that effective, but it is a simple one and a good starting point.

In Figure 12.6, we have some LPF with some cut-off frequency to design. The impulse response is a sinc which is non-causal. The phase is linear.

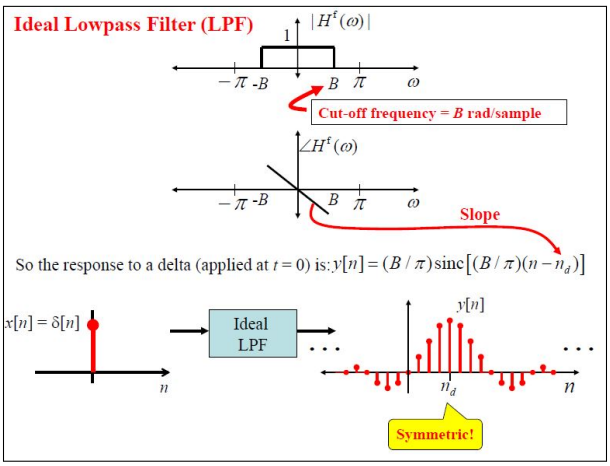


Figure 12.6: LPF FIR

Design Key: Usually we are interested that the phase is linear, but we don't care that much with its slope. We see later how we can achieve certain slope.

The best way to make it causal is to simply truncate the values before 0, and also we truncate the symmetrical part which are after 0 to preserve the symmetry. The filter response after truncation is shown in Figure 12.7.

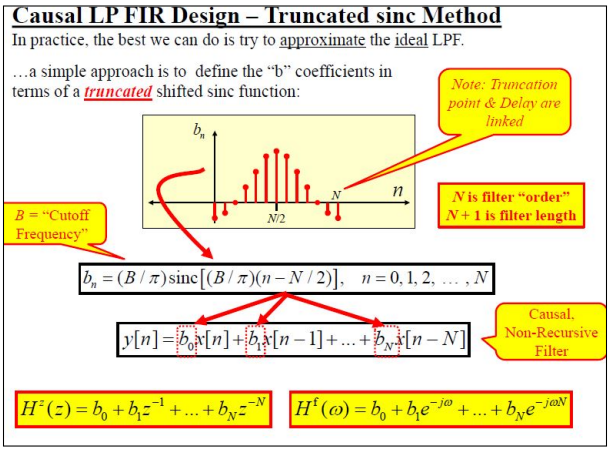


Figure 12.7: Truncation Method

In this type of design to obtain a integer shift, we have to choose N even.

An example is shown in Figure 12.8.

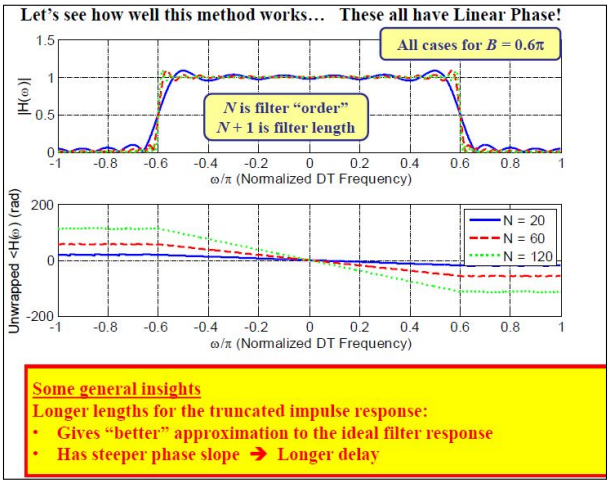


Figure 12.8: Truncation Method

- Notice that when the order N increase, we have a more sharper transition bandwidth.
  - This is true for any desing, and not only this particular desing
- The downside of this is that we don't get to enhance the ripples at the edge of the pass band even when we increase the order

Another example in dB scale is shown Figure 12.9.

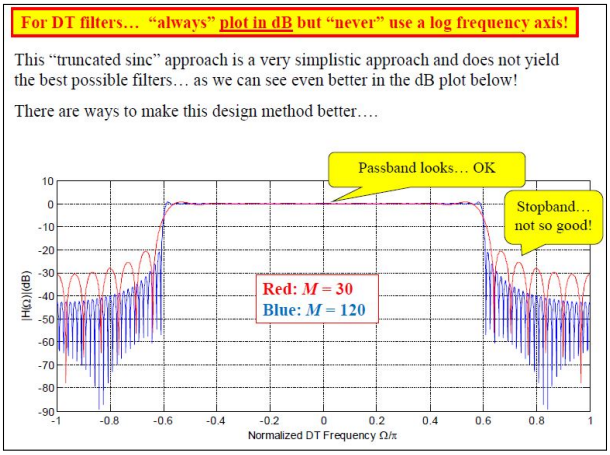


Figure 12.9: Example of design in dB scale: stop band is not good at all

Even with the increase of the order, we have no a big difference the stop band attenuation levels between $M = 30$ and $M = 120$.

12.3.1 The usage of other window

The problem of truncation is can simply formulated by multiplying the desired impulse response by some window, and hence we can use the knowledge of window types and tradeoff to controll the characteristic of the desired filter specification.
The lack of performance in the stop band seen in Figure 12.9 is due to the sidelobes of rectangular window, which don't get lower even when we increase the length (hence the order of the filter).
Recall from Window analysis we have:

- Windows usually cause ***smearing***
- Usually we want some $h_{\text{desired}}[k]$, but we have is some windowed impulse due to windowing
- Mathematically:

$$\begin{aligned} h[k] &= h_{\text{desired}}[k - k_d] \times w[k] \\ H(\omega_d) &= \frac{1}{2\pi} \int_{-\pi}^{+\pi} H_{\text{desired}}(\omega_d)\, W(\omega_d - \omega_0)\, d\omega_d \end{aligned} \tag{12.3}$$

- The sidelobes of the rectangular window will affect the ripples: the pass band (at the edge) and the stop band height
- Whereas the mainlobe will controll the transition bandwidth of the filter

So as a design criteria:

- Choose the window shape to get side lobes we want
- The length of the window control the transition band

12.4 Frequency Sampling Technique

The frequency sampling method rely on DFT computation, and taking finite frequency points at multiple of $\frac{2\pi}{M}$. The steps are shown in Figure 12.10

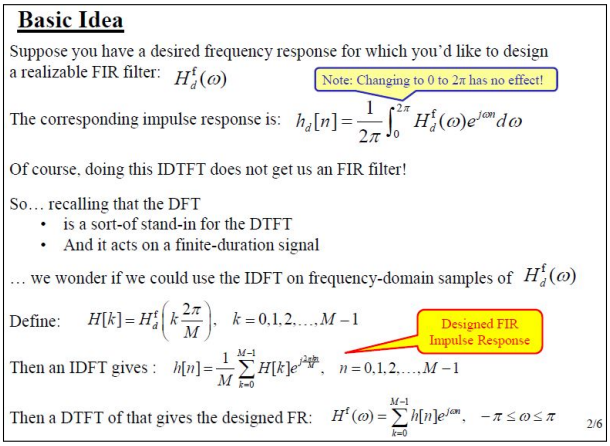


Figure 12.10: FIR using frequency sampling method

- The period of integration in computing $h_{\text{desired}}[n]$ is changed to $0 \rightarrow 2\pi$ instead of $-\pi \rightarrow +\pi$, in order to cover the frequency indices k when computing the frequency response of the desired filter $H_{\text{des}}[k]$ at fininte frequency indices
- Then we apply the inverse DFT to get some finite time domain signal which represent the desired filter (causal and finite)
- As a check, we apply the DTFT again to the computed impulse response to check the filter

Summary results are shown in Figure 12.11.

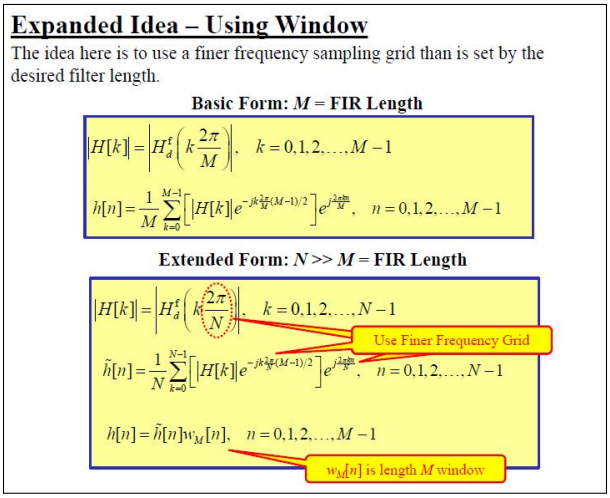


Figure 12.11: FIR using frequency sampling results

- Usually we have $|H[k]|$ are specified

- To ensure a linear phase, we add the term $e^{-j\,k\,\frac{2\pi}{M}\left(\frac{M-1}{2}\right)}$
- The term $e^{j\,\frac{2\pi k\,n}{M}}$ is from the inverse DFT process
- To enhance the design, more frequency sampling points $N >> M$ are taken
  - But this give us a filter bigger then the desired
  - That's why we multiply the $h_d[n]$ by a window

Illustration using MATLAB shown in Figure 12.12

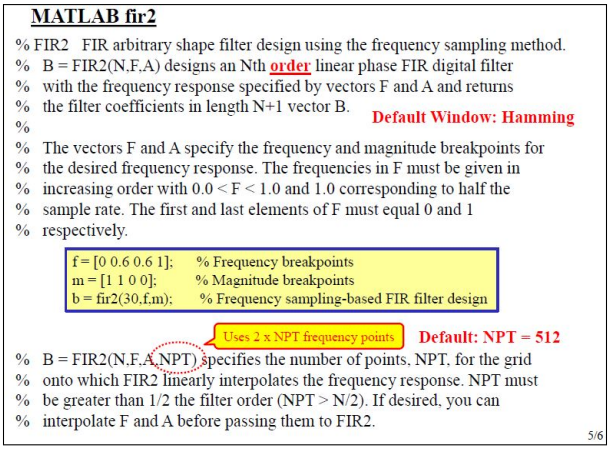


Figure 12.12: FIR using frequency sampling results

- N is the order
- By default, number of frequency points is 2×512
- `fir2` can make arbitrary (some special effect) in some frequency bands, unlike `fir1`
- `f` is inputed as a normalized vector in units of π, and `m` is the desired magnitude
  - In the example of Figure 12.12, from $0 \rightarrow -0.6$, we have our passband since `m` is 1 in this region

12.5 Optimum Equiripple

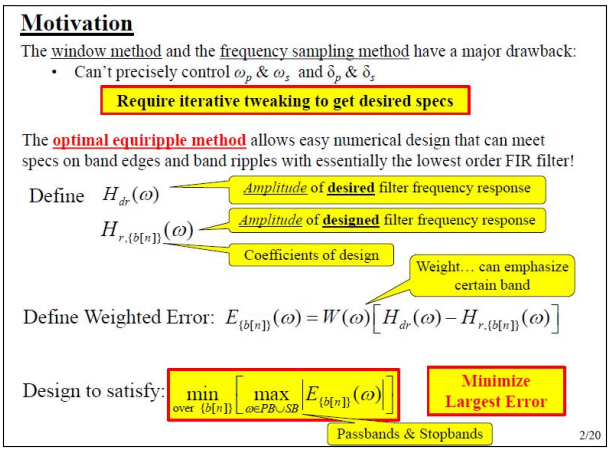


Figure 12.13: FIR using Optimum Equiripple Method

- It was proposed by Parcs and McCellan in 1972, and implemented in FORTRAN
- Drawback of previous designed methods: need to keep iterating to get the specs of the filter, and even after finishing iterating we won't get them all
- The design criteria: minimize the largest error between the desired filter and the designed filter
  - $H_{r\,,b[n]}$: in the designed filter, we explicitly show the coefficient $b[n]$ because there will be used using the design process
  - $\max_{\omega \in PB \cup SB}$: maximize over all the pass band and the stop band $\leftrightarrow$ we don't care what happen in the transition band

12.5.1 MATLAB Example

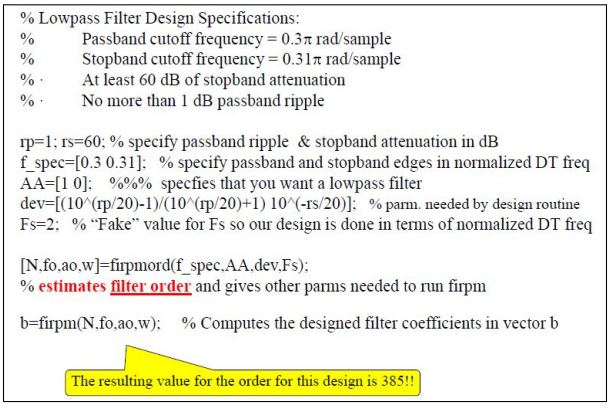


Figure 12.14: FIR using Optimum Equiripple: A MATLAB Example

- First we are specifying the ripples in the pass band and the stop band $\leftrightarrow$ `rp` and `rs`

- Then we set the cuttoff frequencies ***without the*** π (the range goes from $0 \rightarrow 1$), this how MATLAB expects the frequencies
- The sampling rate $F_s = 2$: we choose this value because usually we expect the range of frequencies from $0 \rightarrow \frac{F_s}{2}$.

  If $F_s = 2 \rightarrow \frac{F_s}{2} = 1$, which corresponds to the range of the actual frequencies

  - If we want to desing some pyhsical frequencies in Hz, we need to know the actual sampling rate in hz also.

    As an example: `f-spec = [pass = 1000 , stop = 5000 Hz]` and $F_s = 40$KHz
- The `dev` parameters is for the ripples δ_p and δ_s: we convert them into linear scale
  - Notice the - singe in front of `rs` in the `dev` variable: this is because it is an attenuation

12.5.2 Revisiting the Optimum Equiripple

Design

: To redo this section later.

In a broad view (this is from Dr Barry Veen), the optimum equiripple design consists of:

- An objective function: a minimax criterion, where minimize over the coefficient of the FIR filter, the max absolute error between the desired (ideal) and the designed (actual)
- Take for example 1 form of the linear phase FIR (all of the contains cos terms)
- Use Chebychev polynomial (approximation theory), alternation theorem for the error, then Remez exchange algorithm, all of these methods are grouped under the name Parks-McCellan algorithm, which have used these mathematical tools to form fast, flexible FIR design method
- Unlike the previous 2 methods (window method and frequency sampling), the ripples in the pass and stop band were the same, but in some application we may have some different tolerance.

  The equiripple method can give us different tolerance, but in the expense of having equiripple oscillations (so not perfectly flat as before in window method and frequency sampling)

Chapter 13

IIR Filter

13.1 Introduction

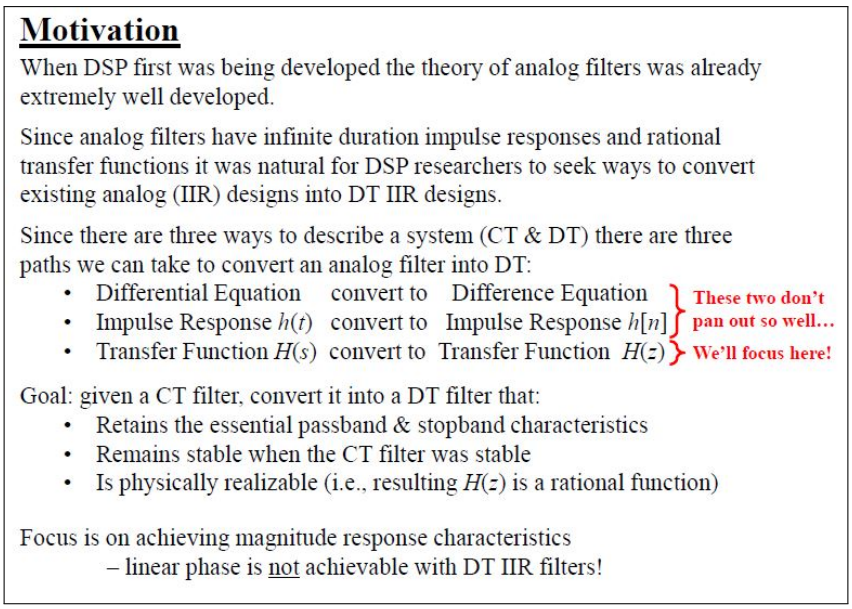


Figure 13.1: IIR Design principle

DSP research started early in 1960 and 1970. Back then, all analog filter were implemented using analog components (resistor,inductor,capacitor,$\cdots$). What the DSP researchers hope to do is to implement these same analog filter using discrete time approach.

A system can be represented using 3 components: differential equation,transfer function and impulse response. What we will do is try to convert each one of them into its discrete time equivalent , and then apply the design.

The first 2 elements (differential equation,impulse response) don't work that much in designing IIR, so we will focus on them briefly and focus on transfer function element.

13.2 Notation

Since we are dealing with transfer functions, and moving from analog to discrete time and frequency, we will have allot of notation illustrated in Figure 13.2.

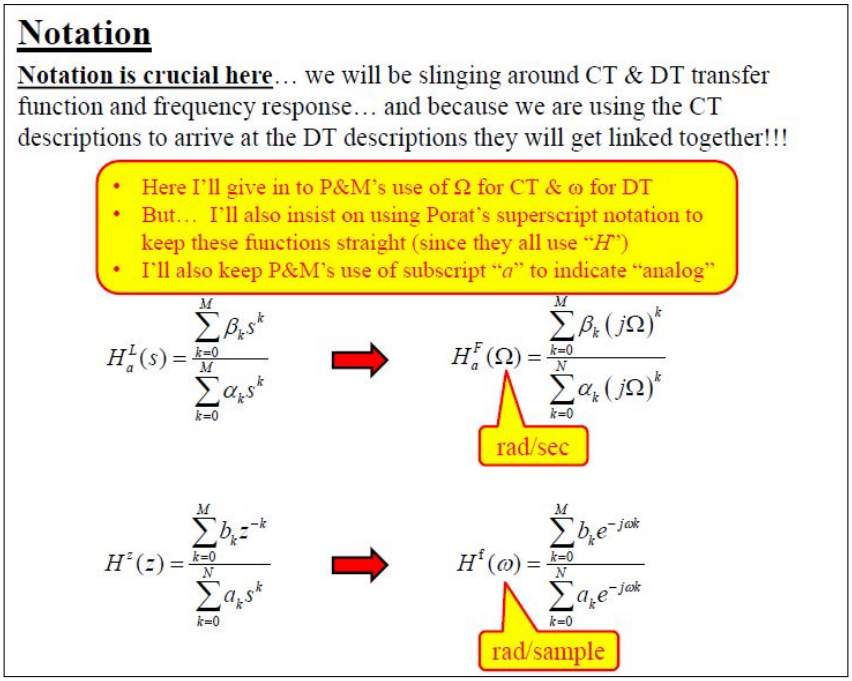


Figure 13.2: Notation Convention

13.3 Difference Equation Approach

In difference equation: we take some differential equation which represent the behaviour of some analog filter, turn it into a difference equation, then apply z-transforms for example to derive $H(z)$.

The conversion from differential equation $\rightarrow$ difference equation is done using approximation of derivative as shown in Figure 13.3

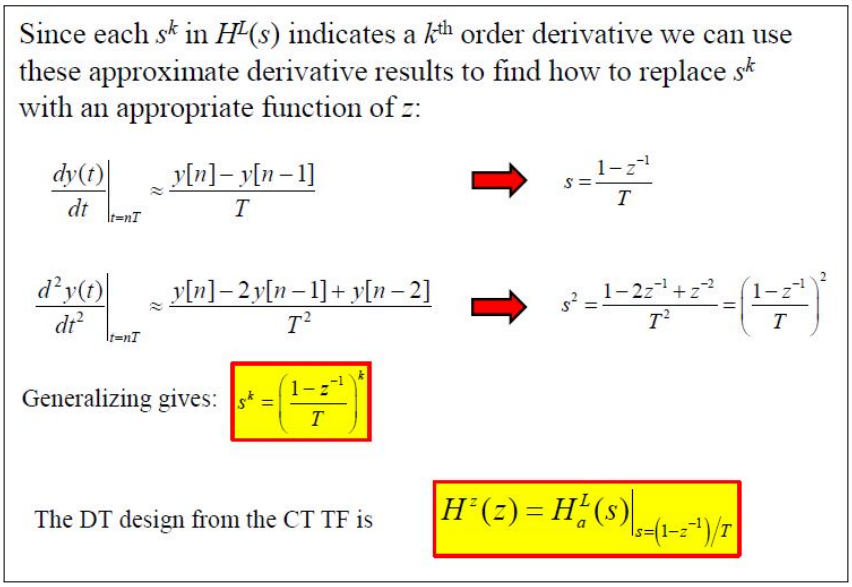


Figure 13.3: Difference equation approach: using derivative approximation to make a change of variable from $s \rightarrow z$

Inverting the relation $s = \dfrac{1 - z^{-1}}{T_s}$ give us $z = \dfrac{1}{1 - sT_s}$. Illustration using s-plane and z-plane is shown in Figure 13.4

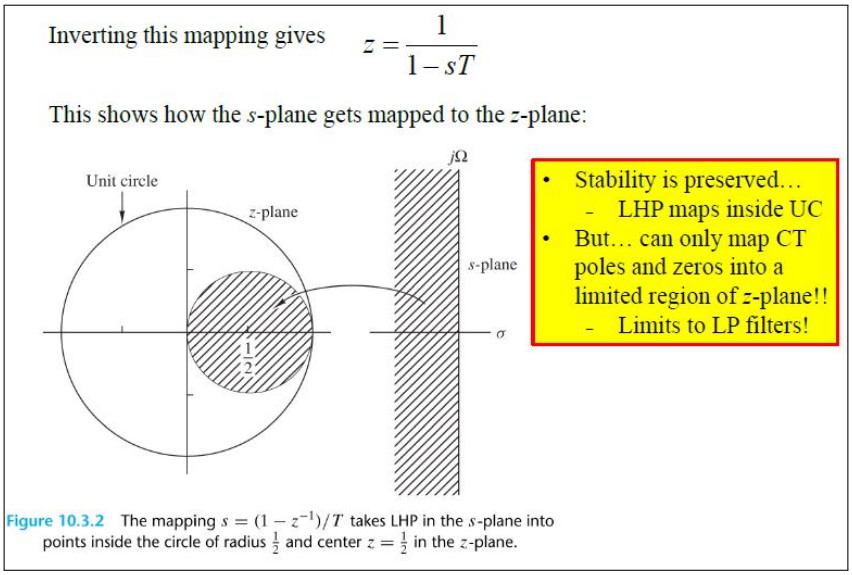


Figure 13.4: Difference equation $s \rightarrow z$ mapping using complex planes

Notice that in Figure 13.4, we don't have all the stable z-plane. So in filter design implication, we are only limited to LPF.

13.4 Impulse Invariance Approach

It takes the impulse function in the *continuous frequency domain*, and turn into a *discrete frequency domain* as shown in Figure 13.5.

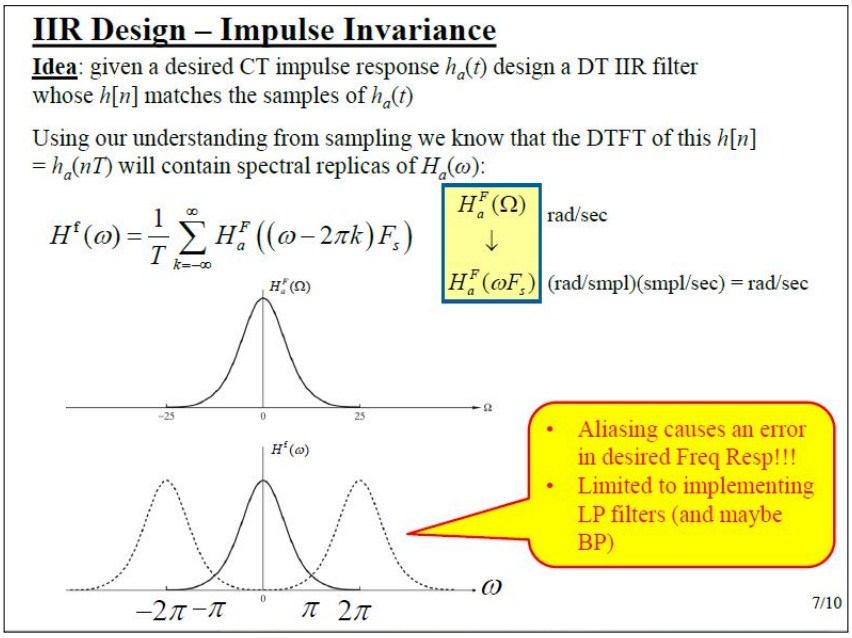


Figure 13.5: Impulse Invariance Approach: Sampling the continuous transfer function

Nevertheless, in Figure 13.5 it is not very clear how to use this fact to build some discrete time system and convert it to rational transfer function. This is illustrated in Figure 13.6.

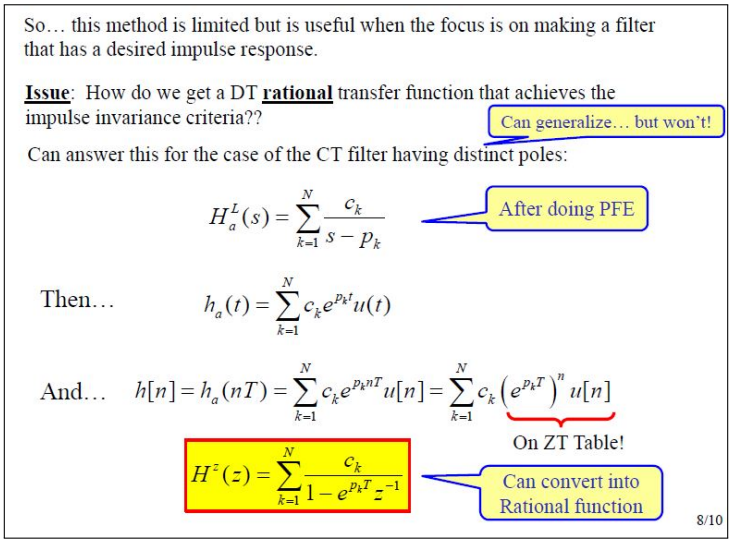


Figure 13.6: Impulse Invariance Approach: converting to a rational transfer function

13.4.1 Example

An example is shown in Figure 13.7.

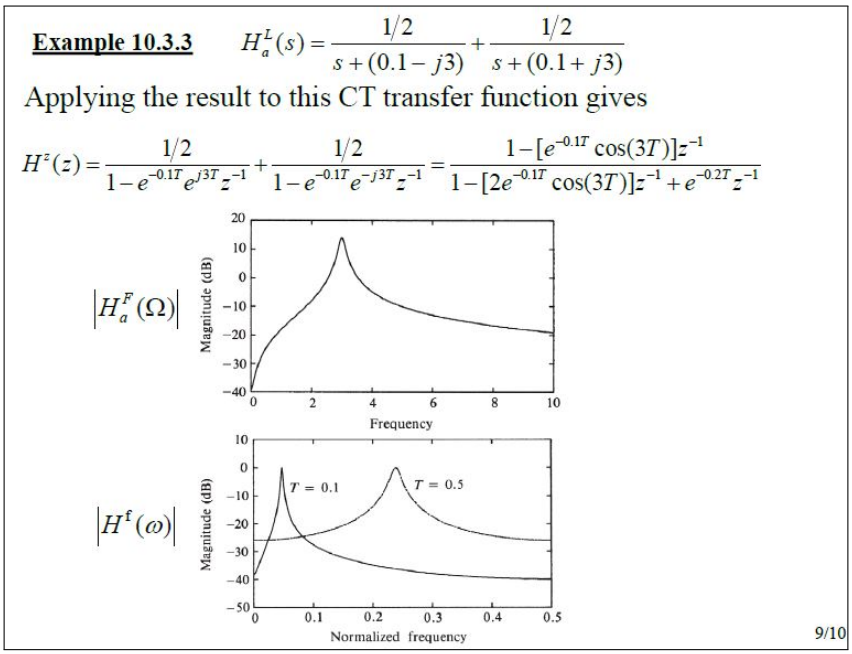


Figure 13.7: Impulse Invariance Approach Example

For a bigger T_s, we get shift in the resonance frequency.

13.5 Bilinear Transformation

Note: here the term *transformation* doesn't mean as in the context of Fourier or Laplace transform, but it means in the context of mapping. But this name has stuck in it in the history.

As a first step, what we have done before is the usage of transfer function concept. This is illustrated in Figure 13.8.

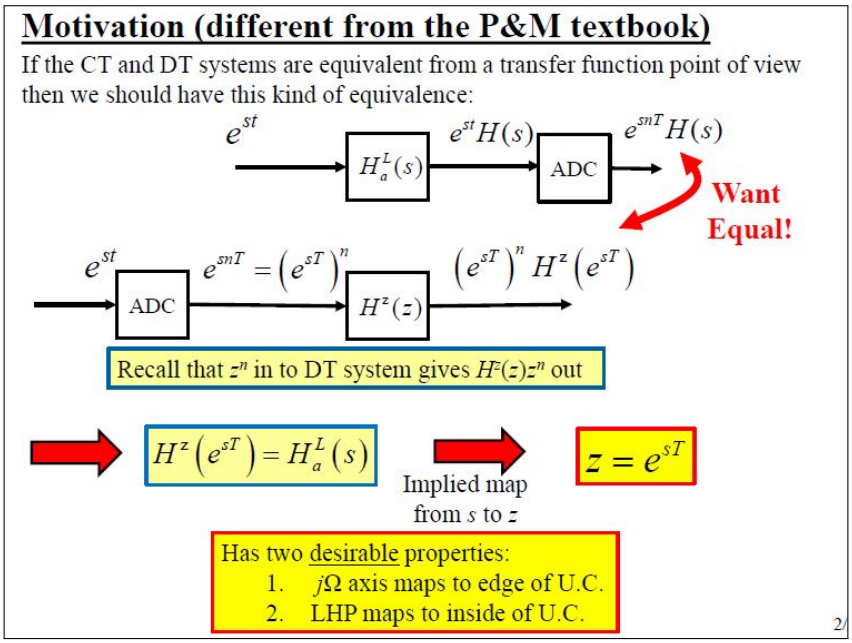


Figure 13.8: Using Transfer function concept

If we want to compare, we need to use and ADC in the front end (as shown in the 2nd row of Figure 13.8). This lead to derive the 1st mapping relation:

$$z = e^{sT} \tag{13.1}$$

Equation 13.1 have some nice properties, like stability (left half plain mapped into the uniform circle of $z-$plane). However, it causes some wrapping (which is similar to aliasing), as shown in Figure 13.9.

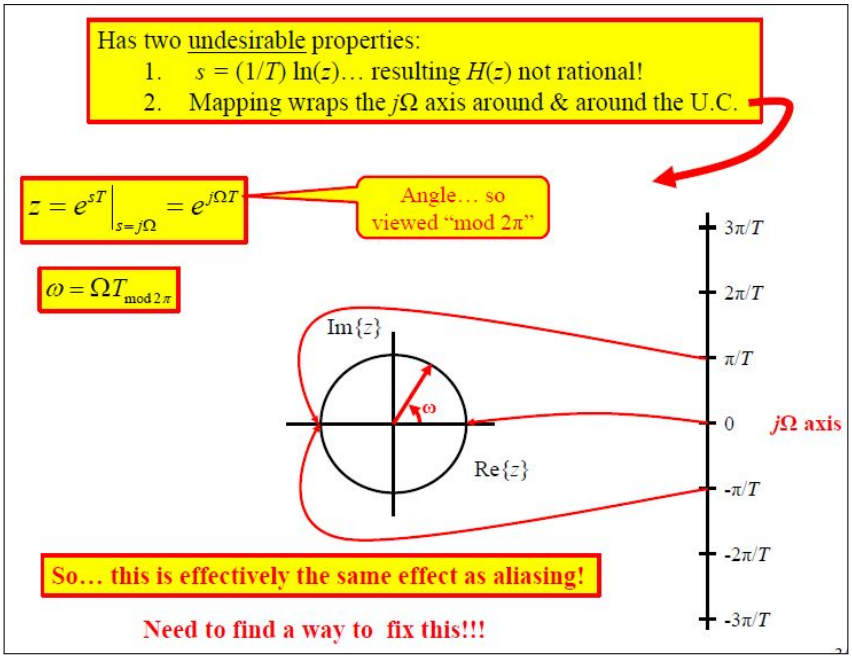


Figure 13.9: Using Transfer function concept

In order to solve this problem, we need to modify the mapping. The final result will be:

$$s = \left(\frac{2}{T}\right)\frac{z-1}{z+1} \tag{13.2}$$

The proof of is shown in Figure 13.10.

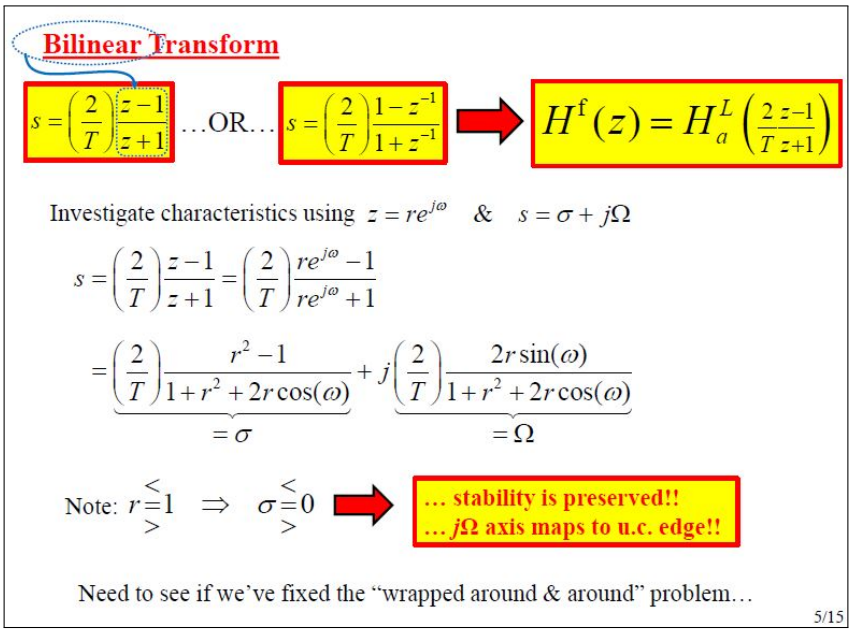


Figure 13.10: Proof of Mapping

Concerning this proof, it isn't very clear how the researcher start with it or what is the motivation behind it, but it works.

Now the next step is to set $r = 1$ and $\sigma = 0$, so we can at the circle and the LHP of the s-plane, plugging in these values in Equation 13.5 we get:

$$\omega = 2 \tan^{-1}\left(\frac{\Omega T}{2}\right) \tag{13.3}$$

An example of Equation 13.3 is shown in Figure 13.11.

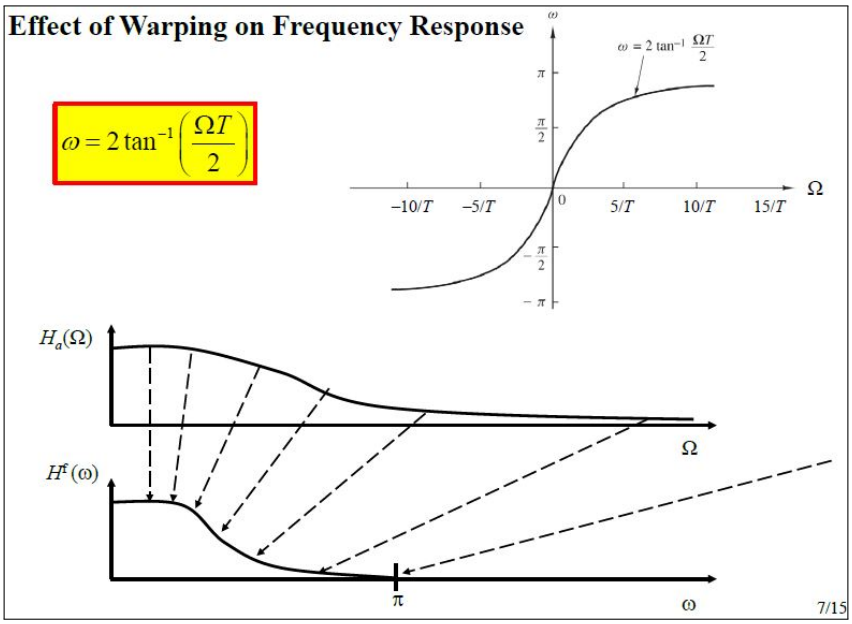


Figure 13.11: Effect Of Warping frequencies

We can see that the continuous transfer function $H(\Omega)$ is spread out over all Ω, and then the discrete transfer function $H(\omega)$ is squish to π, and this compression is controlled by T_s.
The higher the T_s, the more compression we get.

Chapter 14

System Realization

Chapter 15

Multirate Signal Processing

15.1 Reference Used

Chapter 12 from [6], and chap from [4].

15.2 Downsampling

We will see now what happen when we do sampling at T_s then do downsampling at a rate of $M\,T_s$ (downsample)

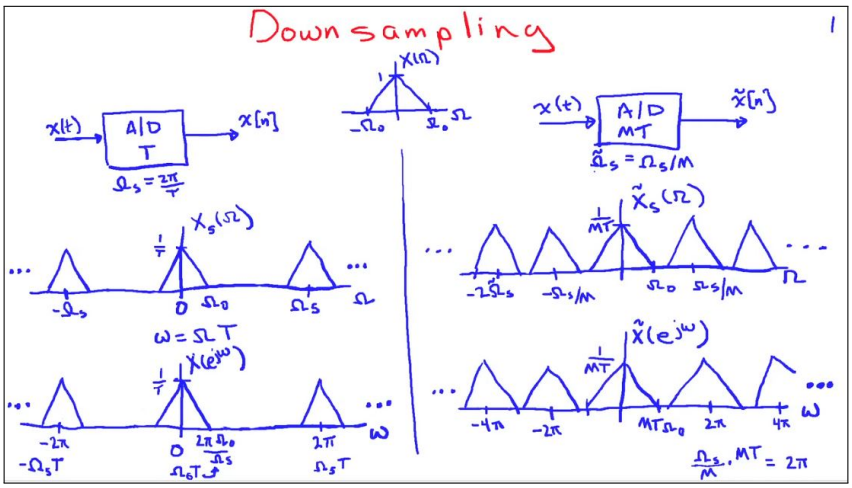


Figure 15.1: Downsampling effect in the frequency domain

- At a rate of T_s, the spectrum get replicated at a multiple of ω_s.
  - When we do the frequency mapping $\omega_d = \omega_c \times T_s$, the spectrum get replicated at a multiple of 2 π
- When doing downsampling:
  - $T_{\text{downsample}} = M \times T_s$ where $M > 1 \in \mathbb{N}$ and $\omega_{\text{downsample}} = \dfrac{\omega_s}{M}$
  - The mapping now in frequency is $\omega_d = \omega_c \times T_{\text{downsample}}$

  - The frequency spectrum in ω_c now is getting shifted at a multiple of $\frac{\omega_s}{M}$
  - In the discrete domain of $-\pi \leq \omega_d \leq \pi$, the amplitude is scaled down by $\frac{1}{M\,T_s}$ and the bandwidth is expanded by a factor of $M\,T_s$

- Aliasing:

  If the signal to be downsampled (that is the original signal) has frequency components larger than the new folding frequency $\leftrightarrow f > \frac{f_s}{2D}$ aliasing noise will be introduced into the downsampled data.

  - To avoid aliasing, the anti-aliaising filter should have a stop frequency edge at $\frac{f_s}{2\,D}$ (or $\frac{\pi}{D}$ radian in discrete frequency)

15.2.1 Implementing Downsampling

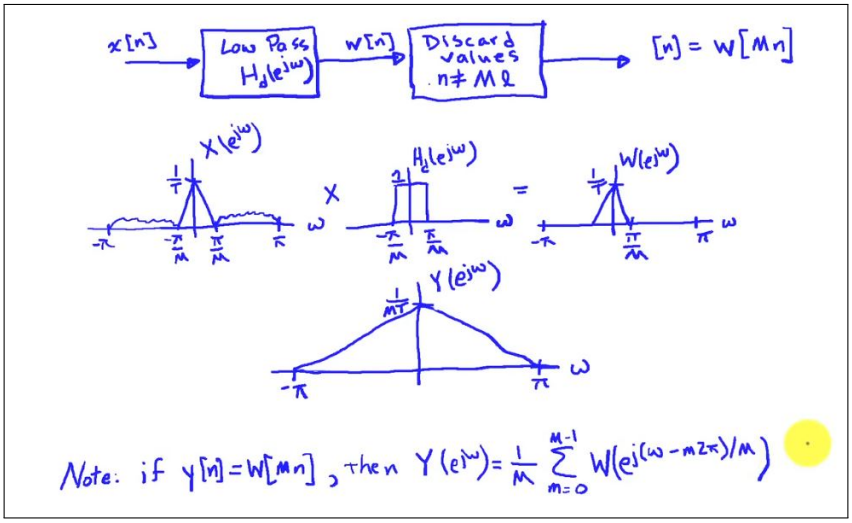


Figure 15.2: Implementation of downsampling

1. Input the discrete signal $x[k]$ in a low pass filter to prevent aliasing
2. Then we throw the values that are not multiple of M

   - $y[k] = w\,[M\,k]$: scaling the discrete time axis by a factor of M
   - We compress the signal in time which means we stretch it in frequency domain: the amplitude spectrum of the signal has stretch out from $\frac{\pi}{M} \rightarrow \pi$
   - DTFT property for scaling:

     $$\text{if } y[k] = w\,[M\,k] \rightarrow Y(\omega_d) = \frac{1}{M}\sum_{m=0}^{M-1} W\left[\exp\left(\frac{j\,(\,\omega_d - m\,2\,\pi\,)}{M}\right)\right]$$

15.3 Upsampling

Upsampling is also know as discrete time interpolation.
Effect on spectrum both in continuous time frequency and discrete time frequency is shown in Figure 15.3.

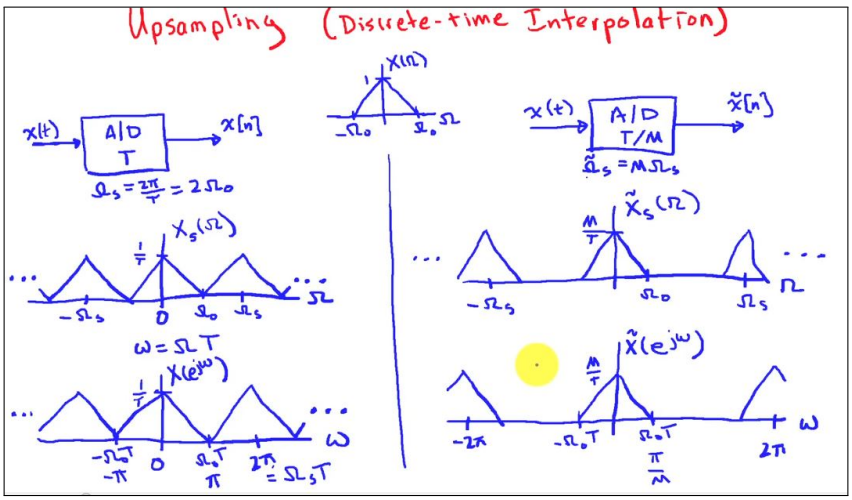


Figure 15.3: Upsampling effect in the frequency domain

- Opposite to downsampling operation, upsampling compress the spectrum

Example of upsampling in discrete time domain is shown in Figure 15.4.

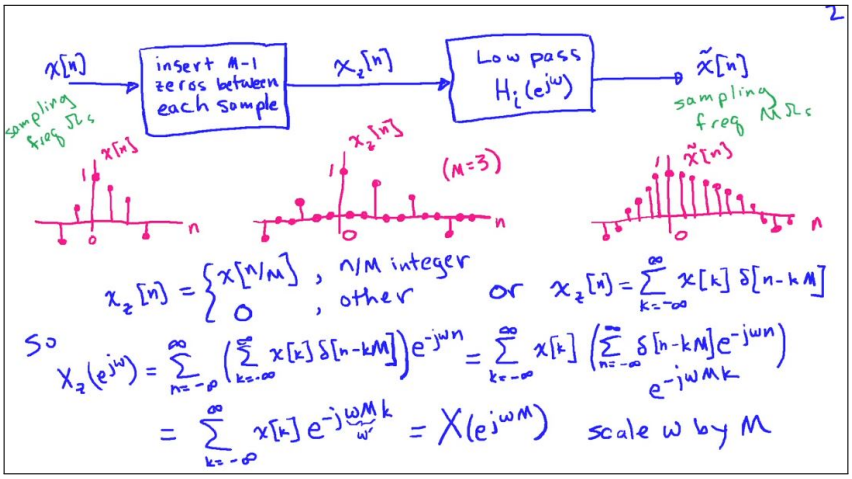


Figure 15.4: Upsampling Example

- We add 0 between each sample of the original discrete time $x(k)$ signal such that $\frac{n}{M} \in \mathbb{N}$
- Then we filter the signal by a low pass filter

Example of the process in the DTFT spectrum is shown in Figure 15.5.

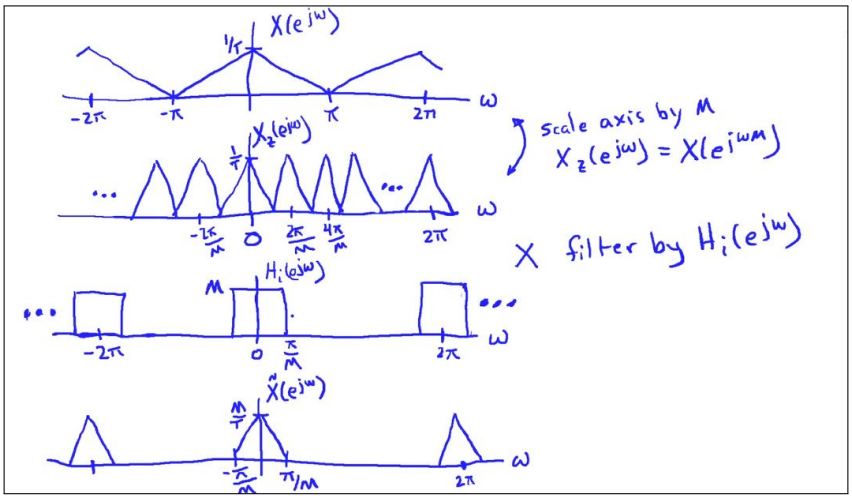


Figure 15.5: Upsampling Example in DTFT spectrum

- The spectrum at π in $X(\omega_d)$ is now shown at $\frac{\pi}{M}$ in $X_z(\omega_d)$
- We filter the signal $X_z(\omega_d)$ using a LPF

Chapter 16

Multirate Sampling by Dr.Michael Lustig Berkely

16.1 Motivation

16.2 Downsampling

- The input of the dowsampling operation is a discrete signal $x[k]$
- What happens when we do dowsampling ?

  The answer is in the spectrum.

  - Recall that sampling in time give us replicate in frequency
  - DTFT for $x[k]$ and $x_{\text{decimated}}[k] = x[k\,M]$ where M is the dowsampling factor
  - Equations:

$$\begin{aligned} X(\omega_d) &= \frac{1}{T_s}\sum_n X_c\left[j\,(\omega_d - n\,\omega_s)\right] \\ X_{\text{decimated}}(\omega_d) &= \frac{1}{M\,T_s}\sum_n X_c\left[j\left(\frac{\omega_d}{M} - n\,\frac{\omega_s}{M}\right)\right] \end{aligned} \tag{16.1}$$

    Equation (**??**) says that if we compress in time, we get more replicate in the frequency

- Next step: is to find the relation between $X_{\text{DTFT}}(\omega_d)$ and $X_{\text{decimated}}(\omega_d)$ without using the CTFT spectrum $X_{\text{CTFT}}(\omega_c)$

  - Relation:

$$X_{\text{decimated}}(\omega_d) = \frac{1}{M}\sum_{i=0}^{M-1} X_{\text{DTFT}}\left(\frac{\omega_d}{M} - \frac{2\pi}{M}\,i\right) \tag{16.2}$$

  - Equation (16.2) tells us that our bandwidth will expand (stretch) by a factor of M.

16.3 Aliasing Revisited

Recall that in aliasing , if we have $f \geq \frac{F_s}{2}$, then aliasing.

In undersampling (1 of Bandpass Sampling techinques:)

- Rule: The sampled spectrum must fall in a 1 Nyqyuist Zone only

Note: figure undersampling Nyquist zone not found

16.4 Upsampling

So much like digital to analog conversion (to a certain point they have similarities, but not the same).

Chapter 17

Fred Harris talk about Multirate 2020

Goal: document and write important points during the talk

-

Chapter 18

Compressed Sensing

Source: Lecture by Prof Michael Lusting, Berkeley University.

18.1 Introduction and Motivation

- Usually when we store information (images captured by camera, some audio files, $\cdots$), we store them in a compressed way, as shown in Figure 18.1

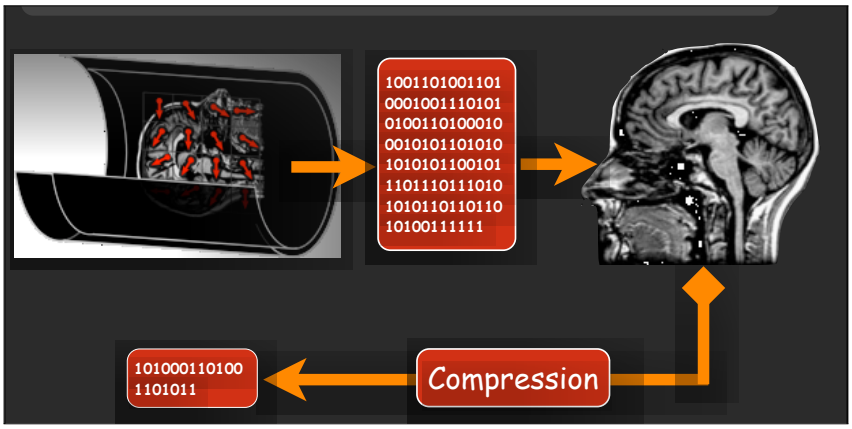


Figure 18.1: Compressing MRI: first we collect, then we compress

  - In other words, depending on Figure 18.1, we usually spend allot of time constructing the data (the measurements), construct the image (the MRI), then inputted to compression algorithm to compress it and ***throw away allot of measurements we spend time to collect them !***.

- As a side note for compression: we usually compress data depending on human perception (hearing or seeing), in such a way we restore only valuable information (so not all the data) and we can't see or hear (in case of audio) the difference

- But what if ***we can sense directly*** the information, as shown in Figure 18.1

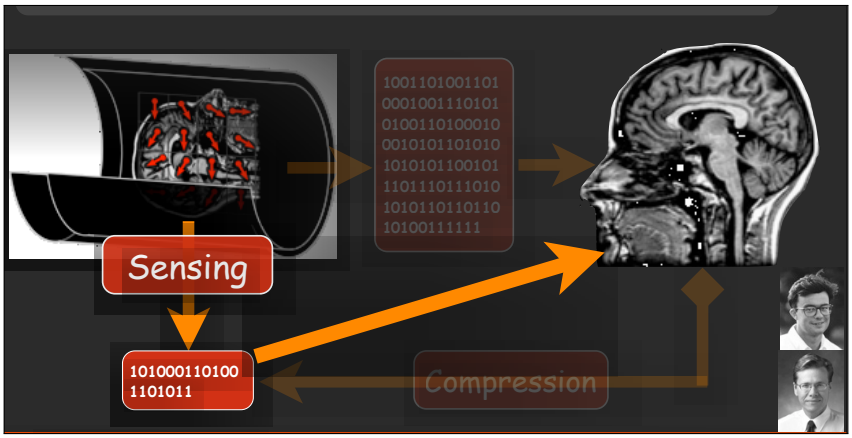


Figure 18.2: Sensing Concept

The approach shown in Figure 18.1 is very helpful because by this way, we don't need to throw allot of information we gathered as in Figure 18.1, and directly collect the valuable information needed for reconstruction.

18.2 Example of Compression and Interpretation

- First let's see some audio signal in Figure 18.3

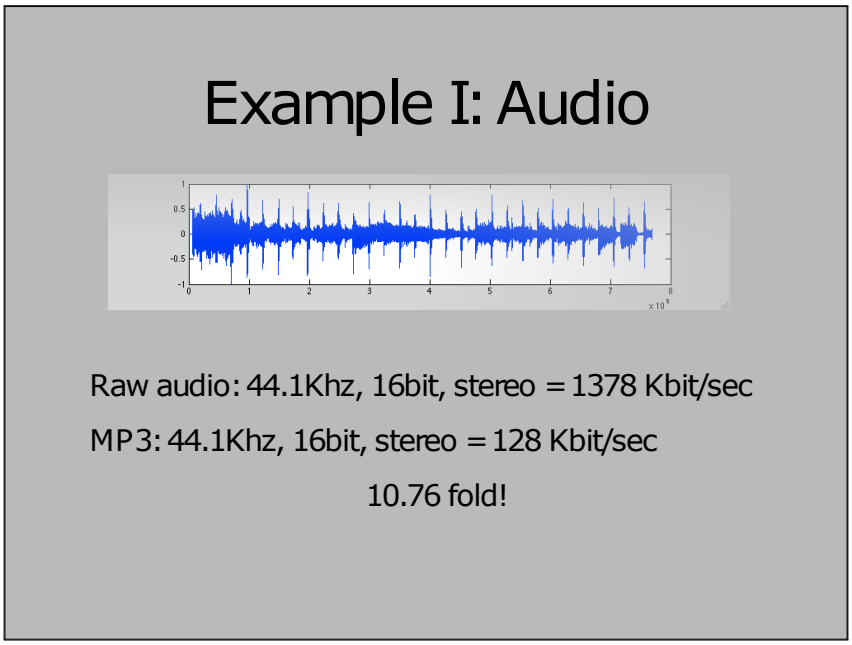


Figure 18.3: Audio Compression

  - As interpretation, if we listen at a rate of 128 Kbit/s, it would make no difference if we here in original rate

- What about images ? are images compressible like audio.

Images are generally more complex type of signals, because they are higher in dimension.

An example is shown in Figure 18.4.

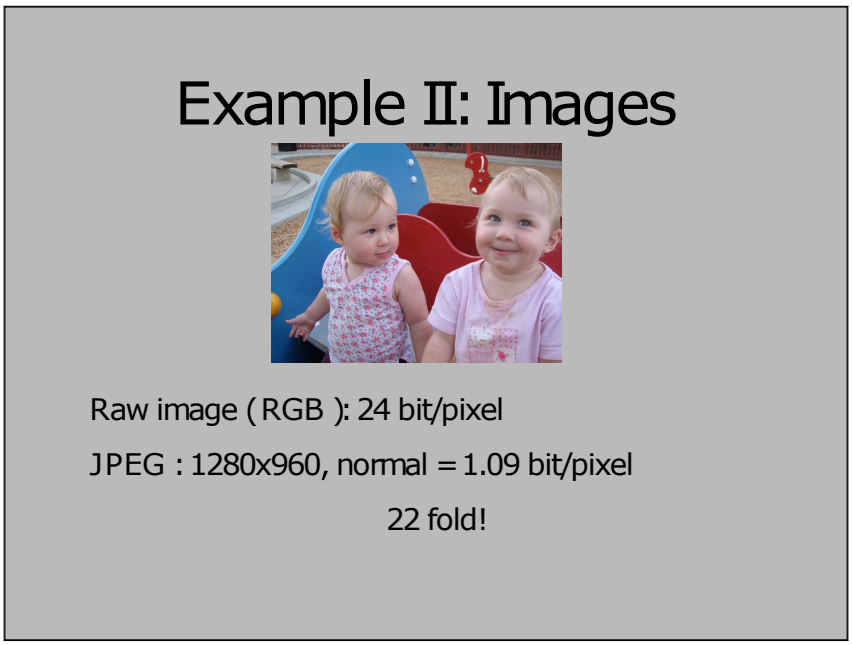


Figure 18.4: Image Compression

- It turns out that we can compress with higher compression ration
- It is true that dimension increase, but the information doesn't necessary increase also as the dimension (power of 2 in this casee)
- The thing is there is allot of correlation we can use in the channel

- Same concept applied to video signals, shown in Figure 18.5.



Figure 18.5: Video Compression

18.3 Steps of Compression

General steps of compression are shown in Figure 18.6.

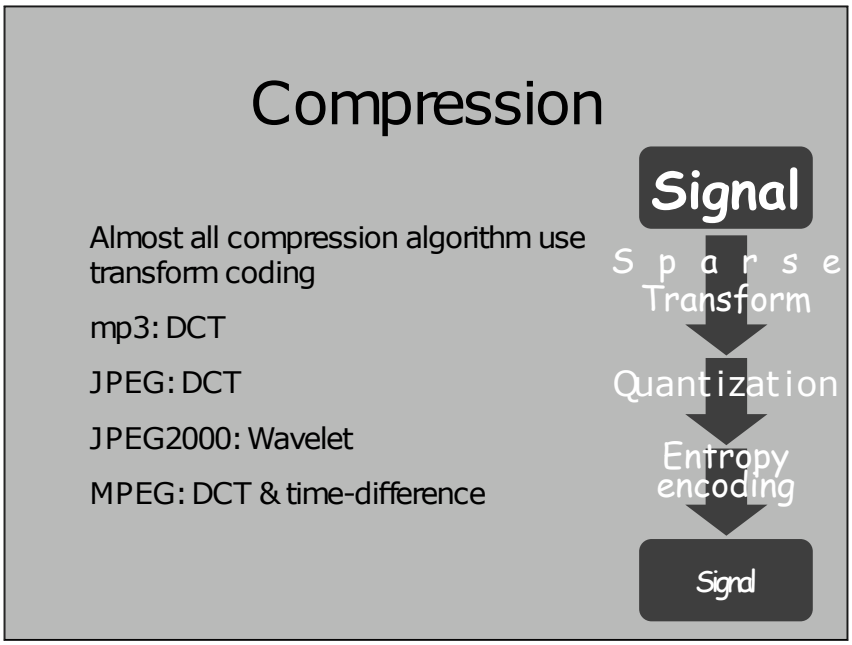


Figure 18.6: Compression Steps

- We apply some sparse transformation
  - Sparse transformation decompose the signals in some set of few coefficients
- We quantize these coefficients and do some entropy coding

18.4 Sparsity and Noise

Usually, when we have a sparse signal with noise, we can identify the signal directly by simple threshold operation. So in other words, sparse signals (or sparse representation in some other domains) have the ability to separate between the useful components of the signals (non zero component with large values usually) and with noisy components which tends to have small values. This is actually known as denoising by using sparsity. These concepts are illustrated in Figure 18.7.

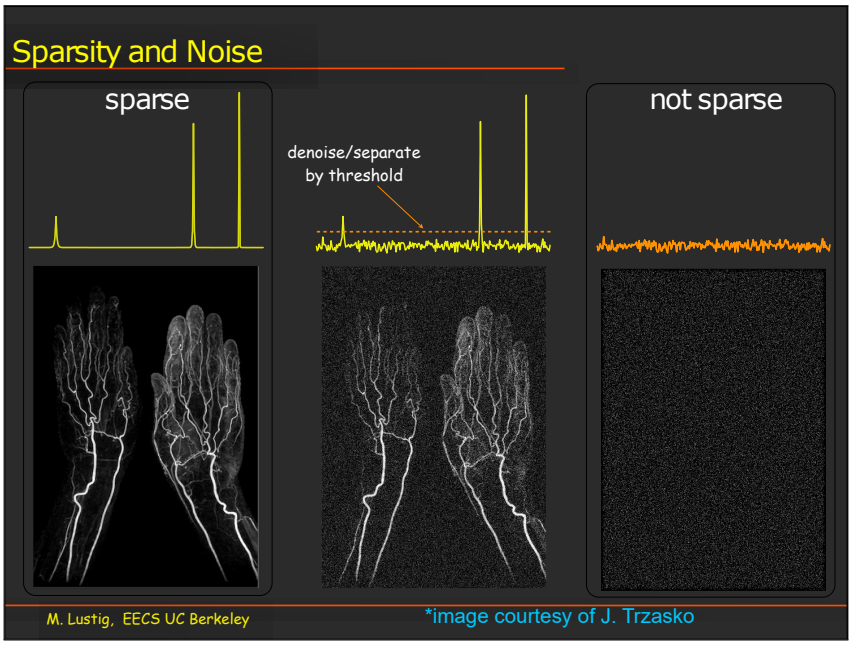


Figure 18.7: Noise Reduction taking advantage from sparse nature of signal

Some example of brain image using wavelet decomposition shown in Figure 18.8

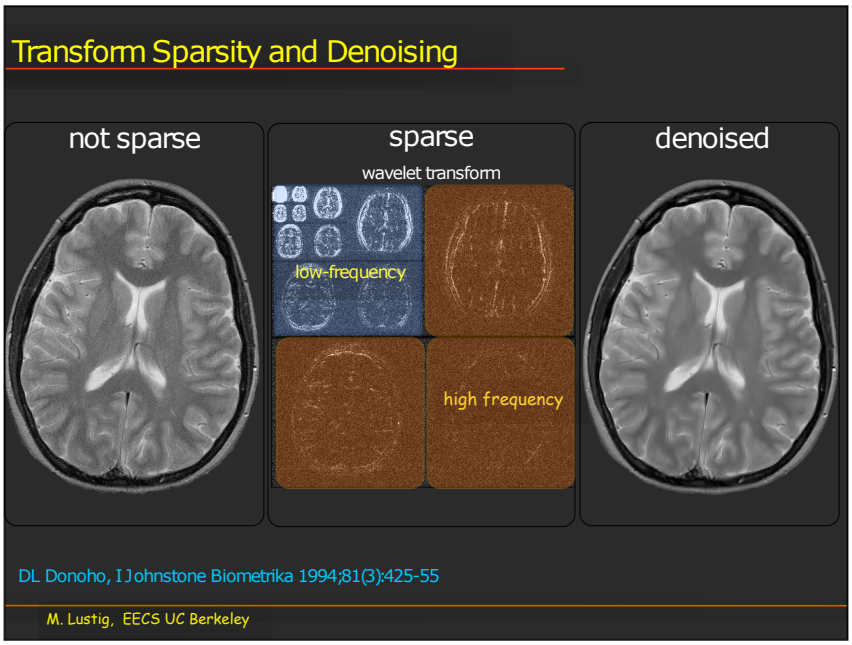


Figure 18.8: Noise Reduction using wavelet transform

- It is true that the image is not sparse. However its wavelet decomposition is sparse and hence we can use thresholding to reduce the noise level and reconstruct the image

18.5 Moving to Compressive Sampling

We have seen in sections before that sparsity help us with noise reduction and compression, but we didn't see its relation with compressive sampling, yet !.

Let's see the concepts of compressive sampling shown in Figure 18.9

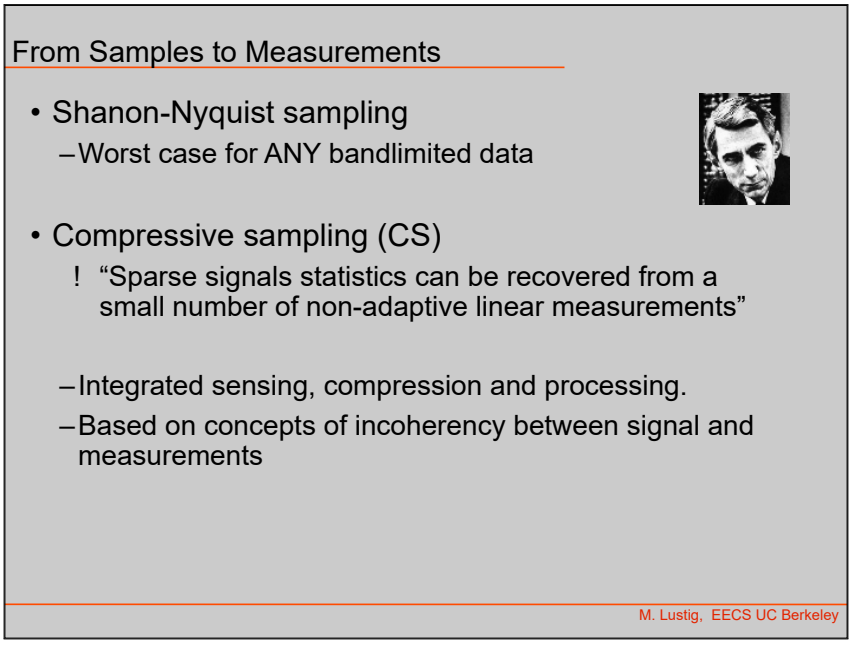


Figure 18.9: Compressive Sampling Concept

- Usually by following Nyquist theorem, we can reconstruct the full signal
- However in compressive sampling, we reduce the number of samples and still able to do reconstruction

Let's begin first by the ***classical sensing*** case, shown in Figure 18.10

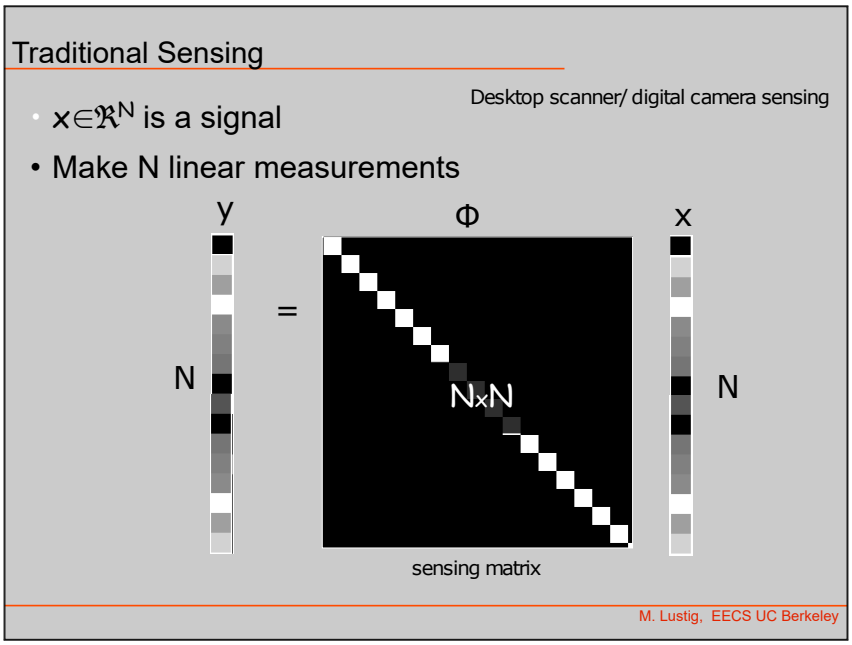


Figure 18.10: Classical Sensing Concept

- We have some signal $x \in \mathbb{R}^N$ and we need to recover it
- What we can do is to construct some N independent linear measurement vector y using a sensing matrix Φ
  - The matrix Φ shown in Figure 18.10 is an identity matrix related to single pixel camera experiment

- The best sensing matrix is the orthogonal matrix: $\Phi \times \Phi^T = \mathrm{I}$
  - Any orthogonal matrix can be used as a sensing matrix. Another example of sensing matrix Φ is shown in Figure 18.11, where it is the FFT matrix used in MRI

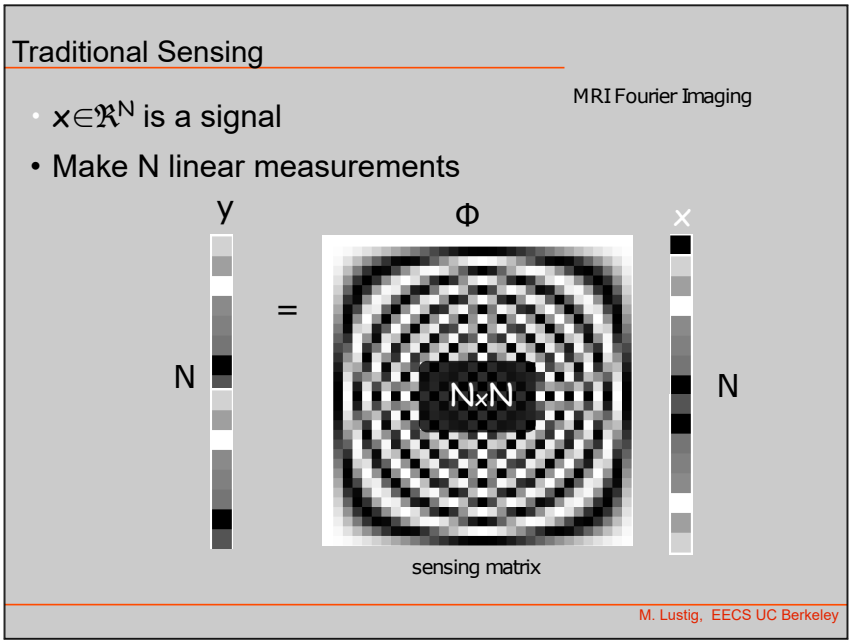


Figure 18.11: Classical Sensing: FFT matrix in MRI

So the classical case for estimating some unknown vector $x \in \mathbb{R}^N$, is to take the same number N on liner independent measurement.

Now let's shift the gears to the sparse case shown in Figure 18.12.

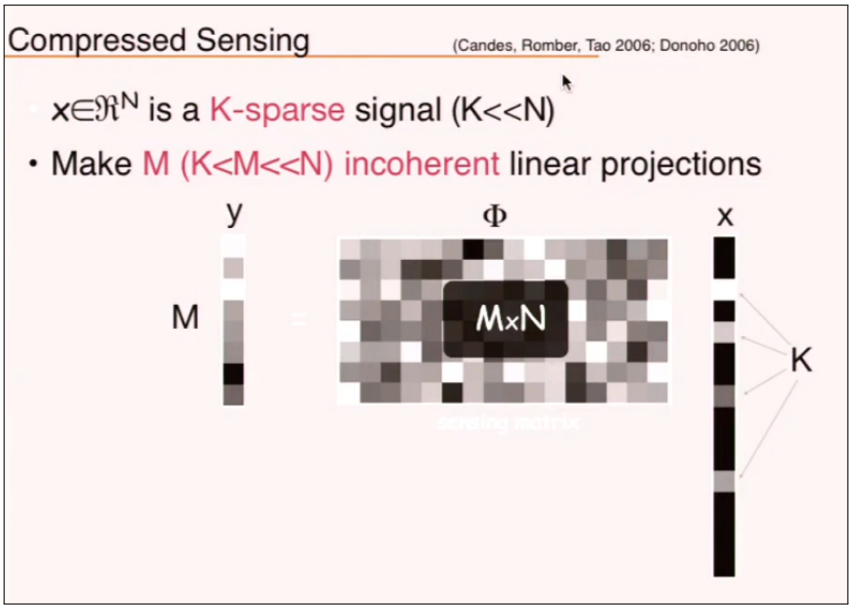


Figure 18.12: Sparse Sensing

- If my signal x is sparse, then we don't need to do N linear independent measurement
  - What we do is take M measurements such that M is far less than N, and little big than K, $\rightarrow$ $K < M << N$
- In the case of sparse also, the sensing matrix Φ becomes flat (rectangular), so it will not become fully orthogonal, but approximately orthogonal
  - What approximately orthogonal means that the columns of Φ will try to be orthogonal as much as they can
  - And also the product $\Phi \times \Phi^T$ will give an approximate identity matrix, where the elements off the diagonal tends to 0, but not exactly 0

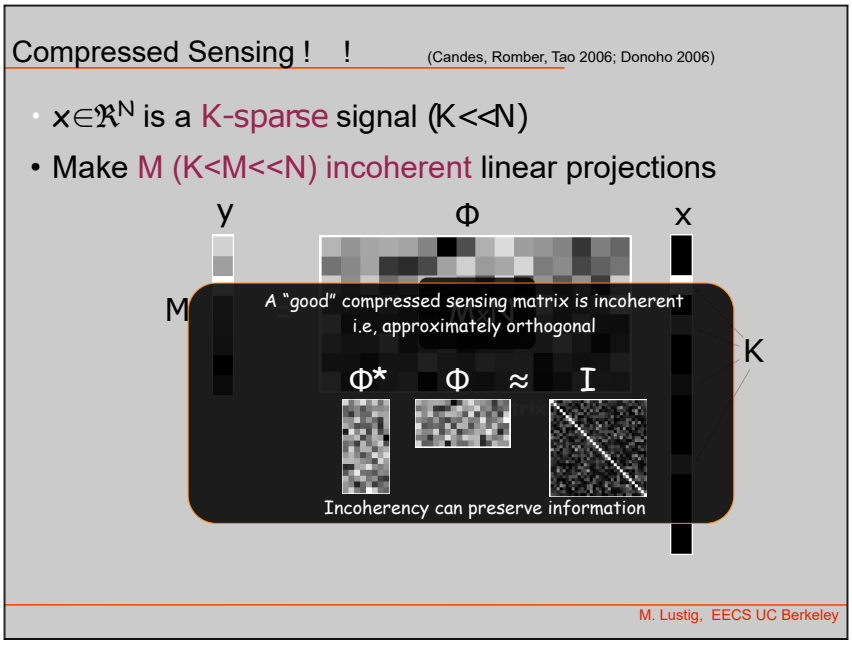


Figure 18.13: Sparse Sensing matrix Φ

If we look again at Figure 18.12, we can do the following:

- If we know where the non zero coefficient are in x, so we can recover x by eliminating the unnecessary columns in Φ which multiply the 0 elements
- In relation with band limited signals, we know where our frequency content is concentrated (the analogy with location of non zero components in signal x), so we can design some sampling criteria lower than the Nyquist rate in which we can recover the signal with no aliasing

However, we usually don't know where these non zero values are, so that's a problem.

However, we can benefit from the constraint that x is sparse, and try to formulate it in some optimization formulation. There are many ways to try:

- Some natural intuition is because x is sparse, we can try to minimize the energy subject to $y = \Phi x$, which can be written as:

$$\begin{aligned} &\min \| x \|_2 \\ &\text{s.t } y = \Phi x \end{aligned} \tag{18.1}$$

Chapter 19

Compressed Sensing 2: Dr Steve Brunton

19.1 References

Read chapter 3 in [7].

19.2 Notes

- A sparse representation: $x_{N\times 1} = \Psi_{N\times N} \times s_{N\times 1}$

  where $\begin{cases} s \text{ is a sparse vector} \\ x \text{ is the natural signal in the intial domain (time or spatial)} \end{cases}$

- Even if we compress and we are able to return x from x, the compresses sensing framework let us choose only few measurements and restore s.

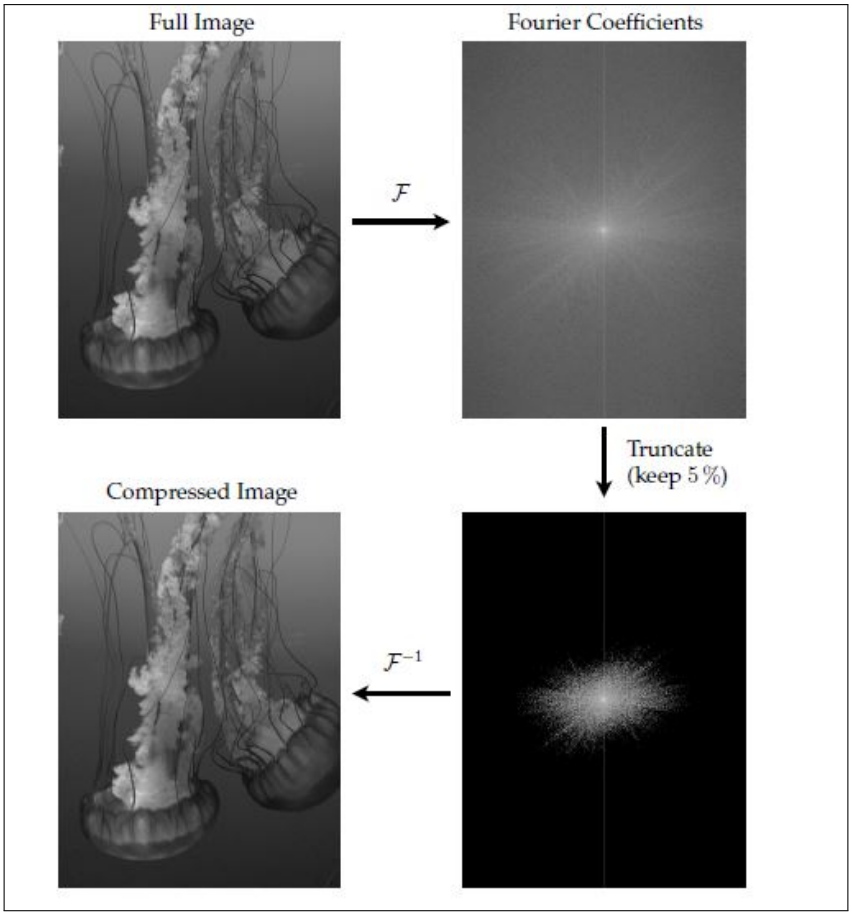


Figure 19.1: Compression using Fourier Basis

- However, in compression, we are making many measurements then throwing away most of these values
  - So what if we can directly make few measurements, recover the sparse vector s, then take the inverse Fourier to restore the original signal x
  - Mathematically, this can be done:

$$\begin{aligned} y_{m\times 1} &= C_{m\times N} \times x_{N\times 1} \\ &= C_{m\times N} \times \Psi_{N\times N} \times s_{N\times 1} \\ &= \Theta \times s_{N\times 1} \end{aligned} \tag{19.1}$$

- Mathematics:
  - The system of equation by (19.1) is known to be an undertimined system
    - There are infinite number of s vectors which satisfy equation (19.1)
  - However, we know that s should be sparse (a constraint which we need to satisfy), hence we add a regularization term to the objective function
    - This the work of many genius people in the field of optimization, statistics and programming
  - For many years, people have used the ℓ_2 norm

* ℓ_2 norm doesn't produce a sparse solution
* It minimize the radius of the circle (the energy) but all the component of the signal (vector) are still active

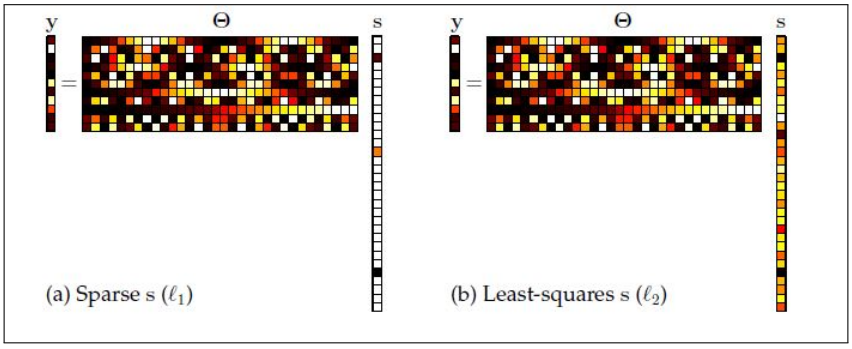


Figure 19.2: ℓ_1 and ℓ_2 norms effects

– Objective Function writing for this problem:

$$\min || C_{m \times N} \times \Psi_{N \times N} \times s_{N \times 1} - y ||_2 + \lambda ||s||_1 \tag{19.2}$$

Or we can formulate it as in (19.3):

$$\min || s ||_1 \text{ subject to } || C_{m \times N} \times \Psi_{N \times N} \times s_{N \times 1} - y ||_2 \leq \epsilon \tag{19.3}$$

where ϵ is a tolerance we set due to some noise in the measurement

19.3 Conditions

Recalling the matrix formulation:

$$\begin{aligned} y_{m \times 1} &= C_{m \times N} \times x_{N \times 1} \\ &= C_{m \times N} \times \Psi_{N \times N} \times s_{N \times 1} \\ &= \Theta \times s_{N \times 1} \end{aligned} \tag{19.4}$$

In order to solve the optimization problem, we need to be aware of several conditions to be respect:

- C is called the measurement matrix, where is has to satisfy some conditions

  1. Incoherence with respect to Ψ: C needs not be parallel
  2. Good measurement matrices can be Gaussian distributed, Bernoulli,$\cdots$.
     So they are random. Put picture here.
  3. A bad measurement matrix example: if we take Ψ^T.
     If we do so, we won't capture all the frequencies (or features) of the sparse vector s. Put the picture here.
  4. Note: it is usually that mathematicians love Gaussian distributed matrices because they are robust mathematically. However, this not usually true in engineering problems. For example, the single pixel camera measurement matrix is used often (to do this later)

- Number of measurements p : they have to be $\propto O\left(k \log\left(\frac{n}{k}\right)\right)$

Bibliography

[1] B. Lathi, *Signal Processing and Linear Systems*, ser. Oxford series in electrical and computer engineering. Oxford University Press, 2010.

[2] J. G. Proakis and D. K. Manolakis, *Digital Signal Processing (4th Edition)*. USA: Prentice-Hall, Inc., 2006.

[3] R. G. Lyons, *Understanding Digital Signal Processing*, 1st ed. USA: Addison-Wesley Longman Publishing Co., Inc., 1996.

[4] D. Manolakis and V. Ingle, *Applied Digital Signal Processing: Theory and Practice*. Cambridge University Press, 2011.

[5] B. Porat, *A Course in Digital Signal Processing*. Wiley, 1997.

[6] L. Tan and J. Jiang, *Digital signal processing : Fundamentals and Applications*, 3rd ed. USA: Academic press, 2018.

[7] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, 1st ed. USA: Cambridge University Press, 2019.